Le développement piloté par les spécifications à l'ère des agents IA

Déléguer le code, garder le contrôle

Olivier Dahan

Introduction - Déléguer le code, garder le contrôle

Je n'écris pas ce livre pour décider si les IA génératives, les agents de code ou les LLM qui les sous-tendent sont bons ou mauvais pour le développement logiciel. Cette question appelle trop facilement une réponse générale à partir d'une expérience locale. Dans une équipe, un agent IA peut faire apparaître en quelques minutes une première proposition de code, de test ou de migration qui aurait demandé davantage de temps. Dans une autre, il peut transférer le travail vers la recherche de contexte, la relecture, l'attente, la correction et la coordination. Les deux expériences peuvent être réelles sans se contredire.

J'emploie ces mots avec précision. Une IA générative produit notamment du texte ou du code. Beaucoup d'agents contemporains s'appuient sur un LLM, un grand modèle de langage, pour interpréter une demande et préparer une réponse ; l'agent ajoute des outils, des accès et une capacité à enchaîner des actions. Le LLM ne porte pas seul les conséquences d'un changement. Elles appartiennent aussi au système où il agit et aux personnes qui choisissent de l'y employer.

Ce qui m'intéresse est le changement de situation qu'elles rendent visible. Lorsque produire des réponses devient moins coûteux, une équipe ne reçoit pas seulement plus de code. Elle reçoit plus tôt des options qui ont déjà une forme, des tests qui paraissent précis, des plans qui semblent complets et des tâches prêtes à être distribuées. Cette abondance peut être utile. Elle peut, hélas, aussi rendre plus facile le passage silencieux entre une proposition plausible et une décision que personne n'a vraiment formulée.

J'ai choisi de partir de ce passage plutôt que d'une moyenne de productivité issue de statistiques et d’études qu’on pourrait discuter des heures. *Une moyenne ne dit pas à elle seule ce que votre équipe a gagné, ce qu'elle a déplacé, ni quelle conséquence elle doit accepter*. Elle ne peut pas décider si une contrainte de confidentialité résiste à une solution séduisante, si une information historique est assez sûre

pour une migration, si un test répond réellement au risque ou si la personne qui autorise une activation peut en porter les effets.

Lorsque le code devient plus facile à produire, qu'est-ce qui doit rester explicitement sous jugement humain ?

Ma réponse est simple, sans être simpliste : l'intention, les contraintes, la connaissance disponible, les preuves et les responsabilités ne disparaissent pas dans l'automatisation. Ils deviennent souvent plus importants à rendre visibles. Un outil peut proposer un chemin. Il ne sait pas, par la seule qualité apparente de sa réponse, quel résultat votre organisation cherche, quelle limite elle ne peut pas franchir, quelle preuve est suffisante dans ce contexte ni quel risque résiduel une autorité peut accepter.

Cela ne fait pas des agents des adversaires du jugement. Je les considère comme des moyens puissants de préparer, comparer, explorer, rédiger et vérifier certaines choses. Leur utilité augmente même lorsque leur place est claire. Une équipe peut leur demander plusieurs variantes afin de mieux voir les compromis. Elle peut leur confier la préparation de scénarios pour consacrer son attention à une propriété métier. Elle peut les employer pour chercher une incohérence ou rendre une trace plus lisible. Mais elle perd quelque chose si la réponse produite prend, par sa seule présence, le statut de réponse autorisée.

Le paradoxe n'est pas la vitesse

Les travaux disponibles donnent de bonnes raisons de résister aux slogans. Sur une tâche autonome et clairement évaluée, une assistance peut réduire fortement le temps nécessaire à une première solution. Dans des travaux de maintenance réels, les gains attendus peuvent au contraire se dissoudre dans le contexte implicite, la revue ou la reprise. Je ne tire pas de ces résultats une loi sur l'avenir des équipes. J'en retiens une prudence plus utile : le mot « productivité » rassemble des réalités différentes.

Un délai plus court ne mesure pas toujours un effort cumulé plus faible. Un gain local de production peut accroître le travail de la personne qui

doit intégrer, expliquer ou maintenir le résultat. Une implémentation livrée plus vite peut créer de la valeur, ou déplacer vers plus tard une décision qui deviendra plus coûteuse à reprendre. La qualité, la confiance du client, la réversibilité, le coût de support et la capacité à comprendre le système font aussi partie du résultat.

Je ne demande donc pas de ralentir par principe. Une équipe qui hésite devant chaque correction locale perdrait sa capacité à apprendre et à servir ses utilisateurs. Je propose plutôt de distinguer ce qui peut être confié à un agent IA de ce qui doit être assumé par les personnes mandatées. *Une production rapide est une ressource. Elle ne devient une décision acceptable que lorsqu'elle reste reliée à ce qu'elle cherche à produire, aux limites qu'elle respecte, aux éléments qui l'étayent et au mandat qui en accepte les conséquences*.

Cette distinction protège également la vitesse quand elle compte réellement. Une décision explicite évite de refaire plus tard une enquête coûteuse pour comprendre pourquoi un comportement a été choisi. Une contrainte écrite avant la construction évite de découvrir après intégration que la solution la plus simple exposait une donnée ou détruisait une propriété. Une preuve placée avant la propagation évite que plusieurs équipes apprennent chacune, à leurs frais, la même incompatibilité. Le contrôle n'est pas l'opposé du travail confié à un agent IA. Il lui donne un périmètre défendable.

Le métier ne disparaît pas : son centre de gravité se déplace

Je crois que c'est ici que le changement devient personnel pour les informaticiens. Une part du métier a longtemps consisté à transformer une intention imparfaite en une forme exécutable : comprendre un système, choisir une structure, écrire du code, corriger ses défauts, puis l'expliquer à celles et ceux qui devront le faire évoluer. Cette part demeure. Mais lorsqu'un agent peut produire rapidement une implémentation plausible, elle n'occupe plus toujours la même place dans la journée ni dans la valeur que l'équipe apporte.

Le travail se déplace d'abord vers la formulation. Une demande vague produisait autrefois un délai qui laissait parfois l'ambiguïté apparaître

pendant la construction car celle-ci prenait du temps. Une réponse générée très vite par un Agent IA peut au contraire donner l'impression que la demande était suffisamment précise. La solution est là sous vos yeux avant-même qu'une controverses puisse naître... Et on oublie que ce n'est qu'une solution et non pas forcément la solution. L'informaticien ne devient pas seulement la personne qui traduit une solution en code. *Il devient plus visiblement la personne qui aide à distinguer le problème à résoudre* de la première solution proposée, qui fait préciser le résultat attendu et qui repère la contrainte absente avant qu'elle ne devienne une dépendance du système.

Il se déplace aussi vers le jugement de l'intégration. Du code qui compile et dont les tests locaux passent reste une proposition située dans une architecture, une histoire de données, des contrats, des usages et des responsabilités. Il faut comparer ce qu'il modifie avec ce que d'autres services supposent, reconnaître une règle métier enfouie dans un ancien comportement et décider si le coût de maintenance ou de support est acceptable. L'agent peut préparer cette enquête. Il ne supprime pas la connaissance tacite ni l'obligation de la mettre à l'épreuve.

La preuve change elle aussi de statut. Quand produire une variante est coûteux, l'effort visible est souvent celui de la construction. Quand les variantes se multiplient à la vitesse des tokens par seconde, l'effort décisif peut devenir celui qui permet de les départager. *L'informaticien conçoit alors des scénarios, cherche le cas frontière, délimite ce qu'un test établit et refuse qu'un indicateur vert couvre par raccourci une question qu'il ne pose pas*. Ce n'est pas un recul vers une fonction de contrôle administratif. *C'est une compétence d'ingénierie* : relier un comportement observé à la propriété que le système doit préserver.

Enfin, le métier devient plus explicitement collectif. Quelqu'un doit savoir qui peut choisir un compromis de disponibilité, accepter une diffusion réduite, modifier un engagement envers un client ou différer une règle que l'équipe ne connaît pas encore. Ces décisions étaient déjà présentes dans le logiciel. Une production accélérée rend simplement plus risqué le fait de les laisser dispersées entre le code, les messages et les habitudes. L'informaticien n'est pas tenu de porter

seul tous les risques. Il doit pouvoir les rendre intelligibles à la personne ou au groupe qui a le mandat de les assumer.

Je ne présente pas ce déplacement comme une promotion abstraite, ni comme l'annonce que chacun devrait devenir architecte ou manager. Écrire, déboguer, tester et maintenir du code restent des activités de jugement technique profond. Le point est différent : la qualité du travail ne se mesure plus seulement à la difficulté de produire une solution. Elle se mesure aussi à la capacité de rendre cette solution compréhensible, vérifiable, intégrable et assumable. Les agents IA peuvent élargir la part de production qui leur est confiée. Ils donnent d'autant plus de valeur aux personnes qui savent garder ces liens vivants.

Pourquoi parler de développement piloté par les spécifications

Le titre de ce livre nomme le développement piloté par les spécifications, ou SDD. Je ne l'emploie pas comme le nom d'une méthode unique. Je ne prétends pas non plus qu'un ensemble de fichiers transforme mécaniquement une organisation en équipe prudente. Le SDD m'intéresse parce qu'il donne une forme visible à des objets qui risquent sinon de rester dispersés entre une discussion, une pull request, un ticket et la mémoire de quelques personnes.

Une constitution peut rendre visibles des principes qui ne doivent pas être érodés par une solution locale. Une spécification peut séparer le résultat attendu de la manière de le construire. Un plan peut exposer les choix de conception au lieu de les enfouir dans le code. Des tâches peuvent conserver le lien entre un travail distribué et la décision qui le justifie. Une clarification peut empêcher une supposition de prendre l'apparence d'un fait. Une analyse ou une convergence peut révéler qu'un critère, une preuve et une décision ne racontent pas la même chose.

Je vois le SDD comme un harnais, non comme un frein. Il attache l'action d'un agent IA aux points d'ancrage que l'équipe doit conserver : une intention, des contraintes, des critères observables, des preuves et

un mandat. Le harnais ne produit pas le mouvement à la place de l'agent, pas plus qu'il ne décide de la destination. Il évite que la puissance de production se détache de ce qui lui donne un sens et une limite.

Dans certaines pratiques contemporaines, ces objets portent les noms de `constitution.md`, `spec.md`, `plan.md` et `tasks.md`. Spec Kit est un exemple de boîte à outils qui donne à un agent de code un contexte structuré autour de ces artefacts : une constitution de projet, puis une spécification, un plan, des tâches, une implémentation et une convergence pour une fonctionnalité. Une clarification, une checklist ou une analyse de cohérence peut compléter ce parcours lorsqu'une question le justifie. Je les mentionne pour que le lecteur qui les rencontre puisse reconnaître ce dont il est question. Je ne donne ici ni commandes, ni modèle imposé, ni enchaînement à suivre. L'artefact n'est pas la décision. Il rend une décision plus facile à discuter, à vérifier et à reprendre ; il ne choisit pas à la place de l'équipe.

Cette frontière est essentielle. Une `spec.md` peut écrire une règle qui reste ambiguë. Un plan peut décrire avec précision une architecture qui ne sert pas l'intention réelle. Une checklist peut être entièrement cochée et ne pas couvrir le risque qui importe. Un fichier contenant une approbation ne suffit pas à établir que la bonne autorité a accepté le bon périmètre. Je préfère des traces courtes qui aident à décider à des documents complets qui donnent seulement l'impression du contrôle, ou pire, qui comme l’ancienne méthode en V noyait les équipes sous des tonnes de documentation que plus personne ne lisait…

Sources et portée

Les études sur la productivité assistée servent ici à borner un paradoxe, non à établir une loi générale. Le chapitre 1 présente un résultat sur une tâche autonome ; le chapitre 2 compare plus directement les indicateurs, les contextes et les coûts déplacés. Les sources documentaires ne valident pas empiriquement à elles seules la discipline proposée dans ce livre. Celle-ci est un cadre de lecture que j'examine au moyen d'arguments et de scénarios construits.

Je vous propose donc de regarder ce que la vitesse rend plus visible dans votre propre travail : *les décisions qui précèdent le code, les limites qu'une solution doit respecter et les preuves qui permettent encore de la contredire*. Le SDD fournira des points d'ancrage concrets. Le jugement restera ce qui leur donne une direction.

Plan de lecture - Une progression pour garder le jugement visible

Ce plan ne remplace pas un sommaire. Il donne la logique du parcours avant d'entrer dans les chapitres. Le livre ne propose pas une méthode à appliquer dans le même ordre à toutes les équipes. Il examine ce que les IA génératives et les agents de code, souvent appuyés sur des LLM, changent dans le travail logiciel. Il construit progressivement les questions qui permettent de juger le travail confié à ces agents : ce qui change dans le travail, ce qui doit être rendu explicite, ce qui doit être établi avant de propager un résultat et la manière dont une organisation peut accepter une décision à sa juste échelle.

Chaque partie ajoute une capacité de lecture. Les chapitres suivants ne refont pas le travail des précédents : ils s'appuient sur lui. L'interlude pratique donne une forme tangible aux artefacts SDD sans transformer l'ensemble en manuel d'outillage.

Partie I - Le déplacement du travail

Cette première partie ne cherche pas à décider si l'IA est, en général, productive. Elle fait apparaître les situations dans lesquelles une production plus rapide déplace le travail vers l'intégration, la revue, la compréhension et le jugement.

Chapitre 1 - Le code devient abondant

Le point de départ est la multiplication possible de propositions plausibles. Le chapitre distingue une implémentation disponible d'une solution réellement choisie. Il montre que plusieurs réponses qui compilent peuvent engager des promesses produit, des coûts et des responsabilités différents.

Chapitre 2 - La fausse productivité

Le deuxième chapitre sépare le gain local de production de la performance d'une équipe. Il compare délai, effort cumulé, qualité et

valeur afin d'éviter qu'un temps économisé à un endroit masque une charge de revue, de maintenance ou de coordination ailleurs.

Chapitre 3 - La dette de décision

Le troisième chapitre nomme le coût créé par des choix implicites, insuffisamment justifiés ou reportés sans condition de reprise. Il distingue cette dette de la dette technique tout en montrant comment elles peuvent se renforcer lorsque personne ne peut retrouver le sens d'une décision passée.

Partie II - Reprendre la main

Après avoir identifié le déplacement du travail, le livre revient aux objets qu'une équipe doit garder sous contrôle humain explicite. Cette partie ne prescrit pas une architecture ; elle établit la grammaire qui permet de discuter une solution avant que son code ne fige une décision.

Chapitre 4 - Commencer par l'intention

Ce chapitre sépare le résultat recherché de la première solution proposée. Il montre comment une intention suffisamment claire permet de discuter la valeur attendue et les conséquences pour l'utilisateur, sans confondre une demande initiale avec le moyen de la réaliser.

Chapitre 5 - Rendre les contraintes visibles

Le cinquième chapitre distingue les exigences impératives, les arbitrages autorisés et les préférences de conception. Il examine pourquoi une solution techniquement commode peut rester inacceptable lorsqu'elle compromet confidentialité, disponibilité, réversibilité ou une autre limite qui s'impose.

Chapitre 6 - Spécifier ce qui doit être vrai

Le chapitre 6 transforme une attente en propriétés observables, scénarios et cas frontières. Il montre qu'une spécification vérifiable guide la production sans choisir nécessairement son implémentation,

et qu'une condition inconnue doit rester visible plutôt que recevoir une réponse pratique par défaut.

Partie III - Produire avec des preuves

Une intention, une contrainte et une spécification ne suffisent pas à établir qu'un résultat est acceptable. Cette partie traite du moment où la preuve intervient et de la personne qui peut décider lorsque les conséquences dépassent le geste technique local.

Chapitre 7 - Valider avant que l'erreur ne se propage

Ce chapitre situe les contrôles avant les points où une erreur devient plus coûteuse à comprendre ou à réparer. Il distingue ce qu'un test, une revue ou une observation peut réellement établir, et examine comment une même hypothèse peut contaminer à la fois le code et ses tests.

Chapitre 8 - Organiser les responsabilités

Le huitième chapitre rend explicite la distribution des mandats, des droits de décision et des escalades. Il défend une autonomie réelle : une équipe peut agir vite dans son périmètre, mais ne doit pas accepter seule un risque produit, réglementaire ou inter-organisationnel qu'elle ne peut pas porter.

Interlude pratique - Un dossier SDD minimal

L'interlude montre, sur un seul scénario tarifaire construit, comment `constitution.md`, `spec.md`, `plan.md` et `tasks.md` peuvent garder reliés les principes, les exigences, les choix, les tâches, les preuves et l'acceptation. Les extraits sont des fichiers Markdown lisibles dans leur syntaxe source. Ils illustrent une forme possible de dossier, non un modèle officiel à copier ni une procédure Spec Kit.

Partie IV - Choisir une posture

La dernière partie met la grammaire du livre à l'épreuve de situations différentes. Elle ne cherche pas une décision unique. Elle montre plutôt pourquoi une même discipline peut conduire à poursuivre, réduire,

différer ou escalader selon les preuves, les contraintes et les mandats disponibles.

Chapitre 9 - Quatre scénarios d'équipe

Quatre équipes rencontrent des limites distinctes : un engagement client, une connaissance historique insuffisante, une contrainte de protection et une responsabilité répartie. Le chapitre compare leurs décisions sans effacer leurs coûts, ni prétendre que la prudence exige toujours de renoncer à la livraison.

Chapitre 10 - La discipline de la délégation

Le dernier chapitre rassemble la posture durable du livre. Il propose de diriger, d'évaluer et d'accepter ou d'escalader comme gestes liés aux objets déjà acquis, sans les transformer en nouvelle grille obligatoire. Il insiste sur la proportion de l'effort, la révision des décisions et la nécessité de ne pas confondre autonomie avec absence de limites.

Conclusion - Déléguer sans abandonner le jugement

La conclusion revient à la promesse initiale sans résumer les chapitres un à un. Elle rappelle que confier du travail à un agent IA ne signifie pas laisser cet agent décider du résultat recherché, des contraintes qui s'imposent, de la portée des preuves ou du risque résiduel. L'objectif du parcours est une capacité durable : produire avec ces outils sans perdre la possibilité de comprendre, de justifier et de reprendre les décisions qui comptent.

Chapitre 1 - Le code devient abondant

Une équipe reçoit une demande familière : un client veut modifier un créneau de livraison déjà réservé lorsqu'il signale assez tôt son indisponibilité. Il faut préserver la confiance du client, ne pas désorganiser inutilement la planification et permettre au service de répondre sans multiplier les exceptions.

Jusqu'ici, la difficulté commençait souvent par la production d'une première piste. Quel domaine modifie-t-on ? Quelle interface expose-t-on ? Comment les tests décrivent-ils le nouveau comportement ? Avec un agent IA, l'équipe obtient en peu de temps disons trois propositions cohérentes. Elles compilent, leurs tests locaux passent et chacune semble répondre à la demande.

La première modifie directement le créneau dans la commande. La deuxième enregistre une demande traitée ensuite par le planificateur. La troisième conserve le créneau initial et propose au client de choisir une nouvelle réservation. Nous avons soudain davantage de code, davantage de tests et davantage de décisions de conception visibles.

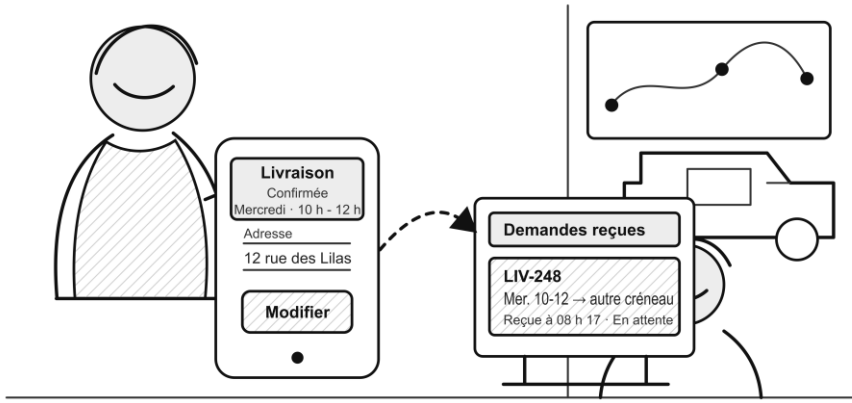


Figure 1.1 - Scénario construit. Le client formule une demande de changement tandis que le créneau initial reste confirmé et que le planificateur doit encore traiter la demande.

Mais avons-nous davantage de réponses ?

Qu'est-ce qui change réellement lorsque produire une première version de code demande moins d'effort ?

Le code ne devient pas abondant partout, ni dans le même sens pour toutes les équipes. Une base ancienne, une intégration mal documentée ou une exigence de sécurité peuvent conserver une grande part de leur difficulté. L'outil ne fait pas disparaître la nécessité de comprendre. Il peut cependant réduire, dans certaines situations, le coût d'apparition d'une première proposition : une fonction, une interface, un correctif, une migration, une explication ou plusieurs variantes d'une même idée.

Ce déplacement mérite notre attention. Une proposition disponible plus tôt n'est pas encore une solution choisie. Lorsque les premières versions se multiplient, le travail ne disparaît pas : il se rend plus visible ailleurs, dans la compréhension du contexte, la comparaison des compromis et l'intégration d'un changement réellement acceptable.

Trois façons de changer une livraison

Reprenons notre scénario construit. Le client a déjà reçu une confirmation. Il indique qu'il sera absent au créneau prévu et demande une modification. L'équipe veut lui permettre de faire cette demande sans ouvrir systématiquement un échange avec un opérateur humain.

La mise à jour directe paraît la plus simple. Une action remplace l'ancien créneau par le nouveau, puis une notification informe les parties concernées. La proposition est nette, son code est court et les tests montrent qu'une commande modifiée affiche désormais la nouvelle date.

Mais cette simplicité porte un engagement précis : le système traite le premier créneau comme s'il n'avait plus de valeur propre. Que devient la confirmation déjà reçue ? Quel historique l'équipe conserve-t-elle lorsqu'un transporteur, un client ou un service de support doit comprendre ce qui a été modifié ? Le problème n'est pas que la mise à jour directe serait toujours mauvaise. Elle peut être exactement la bonne réponse dans un produit où le créneau est une préférence

révisable. Le point est que cette proposition de réponse par l'Agent IA a déjà choisi un sens pour l'engagement initial sans que personne n'y ait réfléchi ni qu'une autorité n'ait validé les conséquences.

La deuxième proposition de l'agent est la demande asynchrone qui fait un choix différent. Le client formule son souhait ; le planificateur l'examine avec les autres contraintes de tournée ; le système accepte, refuse ou propose une alternative. Cette option conserve la trace de la demande et peut mieux respecter la réalité d'une planification partagée.

Elle ne résout pas tout par elle-même. Si aucun accusé de réception n'est prévu, le client ne sait pas si sa demande est arrivée. Si aucun délai de réponse n'est annoncé, l'attente devient une nouvelle incertitude. Une architecture plus souple n'est pas automatiquement un meilleur service. Elle déplace les questions vers les attentes que le produit choisit d'assumer.

La proposition d'une nouvelle réservation conserve, elle aussi, une cohérence locale. Le créneau initial reste intact ; le client reçoit une proposition qu'il peut accepter ; l'équipe évite de modifier un engagement déjà établi. Mais le système peut bloquer temporairement deux créneaux. Dans une période de forte demande, cette précaution peut empêcher un autre client de réserver. Là encore, ce n'est pas un défaut universel. C'est un compromis qui devient visible.

Les trois propositions sont plausibles. Aucune n'est encore acceptable par le seul fait qu'elle compile, paraît lisible ou passe des tests locaux. Le code produit rend les options plus concrètes ; *il ne nous dit pas quel engagement envers le client, quelle responsabilité de planification ou quel coût temporaire l'équipe doit accepter*.

Le contraste devient net lorsque l'équipe répond à des questions ordinaires. Si le client modifie son créneau, reçoit-il une nouvelle confirmation ou une simple information ? Si le planificateur refuse la demande, qui l'explique et dans quel délai ? Si une seconde réservation bloque une capacité rare, à partir de quel moment cette précaution devient-elle un coût injustifié pour les autres clients ?

Ces questions ne sont pas des détails ajoutés après l'architecture. Elles constituent le comportement du produit. La mise à jour directe privilégie une apparence de simplicité. La demande asynchrone rend plus visible la responsabilité de la décision. La nouvelle réservation conserve l'engagement initial, mais introduit une rareté temporaire. L'équipe ne choisit pas entre trois styles de code ; *elle choisit entre trois manières d'assumer une promesse*.

L'agent IA peut rendre cette comparaison plus rapide. Il peut produire une chronologie, proposer les états de la demande, faire ressortir les champs modifiés et préparer les tests qui illustrent chaque option. Ces apports sont utiles parce qu'ils aident l'équipe à voir le choix. Ils ne lui évitent pas de décider ce que le client doit pouvoir attendre.

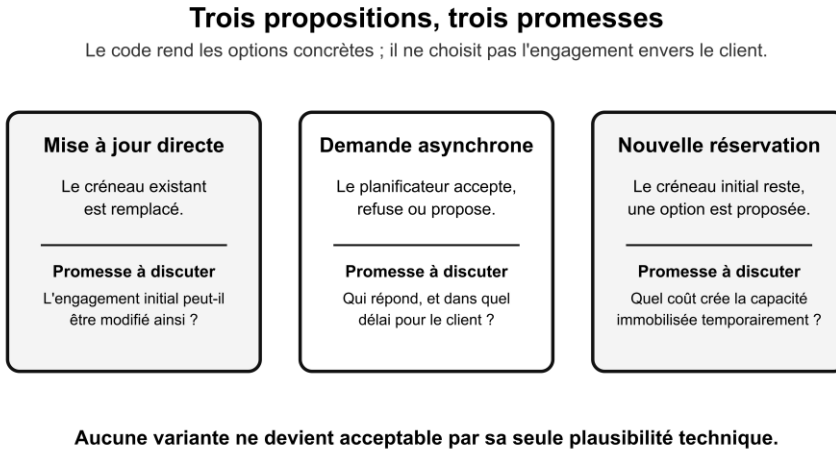


Figure 1.2 - Scénario construit. Trois réponses plausibles peuvent porter des engagements distincts.

Une proposition n'est pas encore une réponse

Cette différence peut sembler évidente lorsque nous prenons le temps de la formuler. Elle l'est moins devant une pull request complète. Une implémentation présente donne immédiatement des objets auxquels réagir : classes, endpoints, captures d'écran, tests et commentaires. Elle fait avancer une conversation. Sa présence peut aussi donner l'impression que la question a été résolue.

L'agent IA n'a pas besoin de mal raisonner pour créer cette confusion. À partir de la demande fournie, il peut produire une réponse très compétente. Si nous lui demandons de modifier un créneau, il peut choisir une stratégie habituelle, anticiper des erreurs et générer une suite de tests. Il ne sait pourtant pas, par cette seule demande, si le créneau est un engagement contractuel, une préférence, une ressource rare ou une information que d'autres systèmes ont déjà exploitée.

Le code semble parfois plus certain que le contexte qui l'a fait naître. Une fonction porte une signature, un test fournit un résultat attendu, une interface présente une action. Les éléments absents sont moins visibles : une règle métier jamais écrite, une exception connue du support, une attente du client ou un compromis que personne n'a encore accepté. Lorsque la première proposition demande plus d'effort, cette absence peut se manifester avant que le changement ne soit prêt. Lorsqu'elle arrive vite, l'équipe doit créer volontairement l'espace où ces questions peuvent apparaître.

Ce livre ne demande pas de soupçonner toute sortie d'IA. Une première proposition est souvent une aide réelle. Elle permet de comparer des options, de révéler une ambiguïté ou de préparer une discussion plus vite. Elle devient problématique seulement si la disponibilité d'une réponse est prise pour la preuve que la bonne question avait été posée.

Le code disponible très tôt attire naturellement la discussion vers ses détails visibles : un nom de classe, un endpoint, la couverture d'un test. Ces détails comptent. Ils peuvent néanmoins prendre tout l'espace avant que l'équipe ait discuté l'engagement créé par la solution. La conversation semble avancer parce qu'elle devient concrète, alors qu'elle a seulement déplacé hors champ la décision la plus coûteuse à reprendre. Les informaticiens aiment le code, et considèrent souvent que les discours trop théoriques sont une perte de temps. Ce qui arrive et qui va bouleverser leur pratique c’est que ces valeurs sont inversées par l’arrivée des assistants IA et des harnais.

Prendre un temps de comparaison ne revient pas à suspendre toute production jusqu'à une certitude impossible. L'équipe peut conserver plusieurs variantes, demander une information au support, construire un prototype ou limiter un premier périmètre. Ce qui importe est que

ces gestes gardent le statut d'une exploration. Ils ne doivent pas devenir, par l'entrée d'une pull request, une réponse que plus personne ne sait attribuer.

Ce qui peut devenir abondant

Le mot « abondance » ne désigne pas un compteur de lignes de code. Il désigne une baisse du coût de production initiale dans certaines situations. Une équipe peut obtenir plus facilement une analyse d'un module, une variante de conception, une implémentation, des tests, une documentation ou un correctif. Elle peut surtout obtenir plusieurs versions suffisamment crédibles pour mériter une comparaison.

Cette possibilité est documentée dans des tâches précises. En 2022, Peng et ses collègues ont mené un essai contrôlé auprès de 95 développeurs recrutés, chargés d'implémenter un petit serveur HTTP en JavaScript. Parmi les personnes ayant terminé la tâche, le groupe ayant accès à GitHub Copilot a mis en moyenne 71,17 minutes, contre 160,89 minutes pour le groupe contrôle - soit 55,8 % de temps en moins.[^peng]

Le résultat ne répond pas à la question de ce livre dans son ensemble. La tâche était autonome, courte et évaluée par douze tests fournis. Elle ne demandait pas d'intégrer une modification à un produit ancien, de négocier un engagement client ni d'assumer une conséquence de production. Elle rend néanmoins crédible un fait important pour notre raisonnement : dans certaines conditions, une première version fonctionnelle peut apparaître beaucoup plus vite.

Nous n'avons pas besoin d'en faire une loi générale pour prendre ce changement au sérieux. Une réduction du coût de production initiale suffit à modifier une conversation d'équipe. Au lieu de discuter une seule option construite progressivement, nous pouvons recevoir trois ou quatre options prêtes à être commentées. L'attention se déplace alors vers ce qui permet de les départager et non la difficulté de les coder.

Ce déplacement ne suppose pas une performance identique des outils ou des équipes. Une personne peut gagner du temps en demandant

une explication d'un module, une autre en préparant une migration, une troisième en générant des cas de test. Le mécanisme commun n'est pas une vitesse universelle. C'est la diminution possible du temps séparant une intention encore floue de plusieurs formes plausibles de cette intention.

Le chiffre ne donne pas une vitesse à réclamer à chaque développeur. Il ne fournit pas davantage un multiplicateur à inscrire dans un plan de capacité. Il rappelle seulement qu'une tâche précisément définie, isolée et testée peut être résolue beaucoup plus vite avec une assistance. *Cette possibilité suffit à changer la forme du travail qui l'entoure*. Elle rend moins rare la première matérialisation d'une idée. *Elle ne rend pas nécessairement moins rare le discernement qui permet de l'adopter*.

Il faut donc résister à deux lectures symétriques. La première serait de conclure que toute production de code est désormais bon marché et que le développement se réduit à la revue. La seconde serait de refuser ce que des études aux périmètres limités et l'expérience quotidienne rendent visible : dans certains cas, obtenir une première réponse demande effectivement moins d'effort. Le chapitre suivant examinera pourquoi ces deux constats ne suffisent pas à mesurer la productivité d'une équipe. Pour l'instant, retenons une conséquence plus simple : ***le nombre de propositions auxquelles nous pouvons donner une forme augmente parfois plus vite que notre capacité à les juger***.

Ce que l'automatisation ne décide pas

Parler d'abondance peut inquiéter les développeurs. Si le code devient plus facile à produire, qui devra encore le lire, le modifier et le maintenir ? La réponse honnête est que les agents peuvent eux aussi accomplir une part croissante de ce travail. C'est même la tendance qui rend ce livre nécessaire. Nous ne devons pas bâtir notre raisonnement sur la promesse que l'humain écrira toujours chaque ligne, ni que la compétence manuelle restera l'unique condition du contrôle. D’ailleurs quand plus personne n’écrira tous les jours du code, qui pourra affirmer avoir la compétence pour vérifier une pull request produite par une IA ? Personne.

Le point décisif est différent. Un système continuera de porter des comportements, des contrats, des données, des opérations et des conséquences réelles. Quelqu'un doit pouvoir dire ce qui est confié à l'agent IA, ce qui est attendu du résultat, ce qui reste inconnu et qui peut accepter le risque d'un changement. Cette personne n'a pas nécessairement à réécrire elle-même l'implémentation. Elle doit pouvoir évaluer le résultat à son niveau de responsabilité, demander une preuve adaptée ou savoir quand faire intervenir l'expertise qui lui manque.

Ce qui change est donc la rareté qui organise l'attention. Quand une première version demandait beaucoup de temps, la capacité à la produire était un goulot évident. Cette capacité demeure utile, notamment pour comprendre un domaine, choisir une architecture ou diagnostiquer une erreur difficile. Mais elle n'est plus toujours le seul accès à la production. Lorsque plusieurs premières versions deviennent accessibles, leur seule existence distingue moins les options. L'attention se porte davantage sur la capacité à reconnaître ce que chacune suppose et ce qu'elle coûtera une fois intégrée.

Cette capacité n'est pas un supplément abstrait appelé « gouvernance ». Elle est faite de gestes concrets : retrouver le contrat qu'un changement affecte, demander à qui une confirmation doit être envoyée, comprendre pourquoi une règle existe, comparer deux effets de bord, vérifier un comportement et décider que le risque résiduel est acceptable. Les chapitres suivants donneront des noms plus précis à ces gestes. Ici, il suffit de constater qu'ils ne sont pas produits automatiquement avec les classes et les tests.

Une maxime peut nous aider à sentir la limite : « Ne demande jamais à une IA ce que tu ne saurais faire toi-même. » Prise au pied de la lettre, elle est un peu trop stricte. Les IA nous servent aussi à découvrir de nouveaux domaines, si on en fait l’effort, ce qui n’est pas forcément le cas de tout le monde. Mais sous cette forme la maxime nous oblige encore à savoir écrire le code que la machine produit. C’est une situation temporaire, un avantage porté par les séniors qui ont 20 à 40 années de code derrière eux. Les prochaines générations n’auront plus cette expérience. Au même titre que rares sont les séniors

d’aujourd’hui capables d’écrire et maintenir du code assembleur. Même du code CIL de .NET.

La version utile ici est donc plutôt celle-ci quand on déplace la connaissance du code vers la connaissance des décisions : ne confions pas à une IA une décision que personne ne saura définir, évaluer ou reprendre. Le contrôle ne repose donc pas sur l'obligation de savoir tout coder soi-même. Il repose sur une chaîne de jugement : une intention explicite, des limites connues, une manière de vérifier et une personne ou une équipe capable d'escalader lorsque ses compétences ne suffisent plus.

Le scénario des créneaux l'illustre sans opposer les personnes à l'agent IA. L'agent peut aider l'équipe à localiser les usages du créneau, à préparer les variantes et à générer des tests. Un développeur expérimenté peut voir plus vite que la mise à jour directe écrase un engagement ou que la seconde réservation immobilise une capacité rare. Mais ni l'un ni l'autre ne peut déduire du code seul quelle expérience le produit doit promettre au client. *Cette question doit être assumée et rendue visible avant que l'implémentation ne la fixe à bas bruit*.

Ce qui ne devient pas automatiquement abondant

La première chose qui ne devient pas abondante est la connaissance d'un système donné. Un agent IA peut retrouver les usages d'un champ, résumer des journaux ou pointer des dépendances. Il peut raccourcir une enquête. Il ne transforme pas automatiquement un commentaire ambigu ou une documentation incomplète en fait établi.

Dans le scénario, il faut encore savoir ce que le premier créneau signifie pour le client et pour le planificateur. Les réponses peuvent être dispersées dans les conditions de vente, les habitudes de l'équipe support, les contrats d'intégration ou la mémoire d'une personne. Certaines informations sont peut-être absentes. La production d'un endpoint ne les recrée pas.

La deuxième chose qui reste rare est la capacité de choisir. Plusieurs options peuvent être techniquement défendables tout en engageant

des compromis incompatibles. Choisir ne consiste pas seulement à préférer le code le plus court, le plus élégant ou le plus rapide à obtenir. C'est décider quel résultat nous cherchons, ce que nous ne pouvons pas compromettre et quelle incertitude nous acceptons provisoirement.

La troisième est la capacité d'intégrer. Une solution générée, lue une première fois ou démontrée localement n'est pas encore devenue une partie du produit. Elle doit rencontrer les contrats existants, les comportements réels et les personnes qui devront la faire vivre. Elle doit être vérifiée à un niveau proportionné à ce qu'elle change. Elle doit enfin être acceptée par celles et ceux qui portent les conséquences de son entrée dans le système.

Ces trois travaux ne sont pas les restes ingrats d'une production devenue automatique. *Ils donnent sa valeur au code produit*. Ils ne demandent pas toujours plus de temps qu'avant ; ils peuvent eux aussi être assistés. Mais ils ne disparaissent pas dans la même proportion que la première version d'une fonction perdue au fond d’un code source généré par un agent.

Il est utile de voir comment ils s'enchaînent. Comprendre ne signifie pas tout connaître avant de commencer. L'équipe peut découvrir une information en construisant un prototype ou en comparant deux options. Choisir ne signifie pas qu'une seule personne impose une solution. Il peut s'agir de rendre explicite un compromis, puis de demander à la personne qui en porte les conséquences de l'accepter ou de l'écarter. Intégrer ne signifie pas non plus allonger chaque changement d'un cérémonial identique. Le niveau de vérification dépendra du risque, de la réversibilité et de la portée du changement. La nouvelle approche ne doit surtout pas devenir l’occasion de recréer une bureaucratie infernale !

Mais aucune de ces nuances ne permet de traiter la première proposition comme la décision elle-même. Une équipe peut avancer avec une connaissance incomplète, une solution provisoire et un risque limité. Elle doit pouvoir dire que c'est ce qu'elle fait. À défaut, elle ne gagne pas vraiment du temps : elle laisse simplement au système, à

ses utilisateurs ou à l'équipe suivante le soin de découvrir le compromis qui n'avait pas été choisi.

L'attention départage les propositions

Lorsque plusieurs premières versions deviennent disponibles, leur seule existence ne permet plus de les distinguer. L'attention se porte sur leur sens : l'engagement qu'elles créent, la connaissance qu'elles supposent et la conséquence qu'elles laissent à l'équipe. Cette attention n'est pas une étape administrative ajoutée au code. Elle est ce qui permet de reconnaître qu'une proposition répond réellement à la situation.

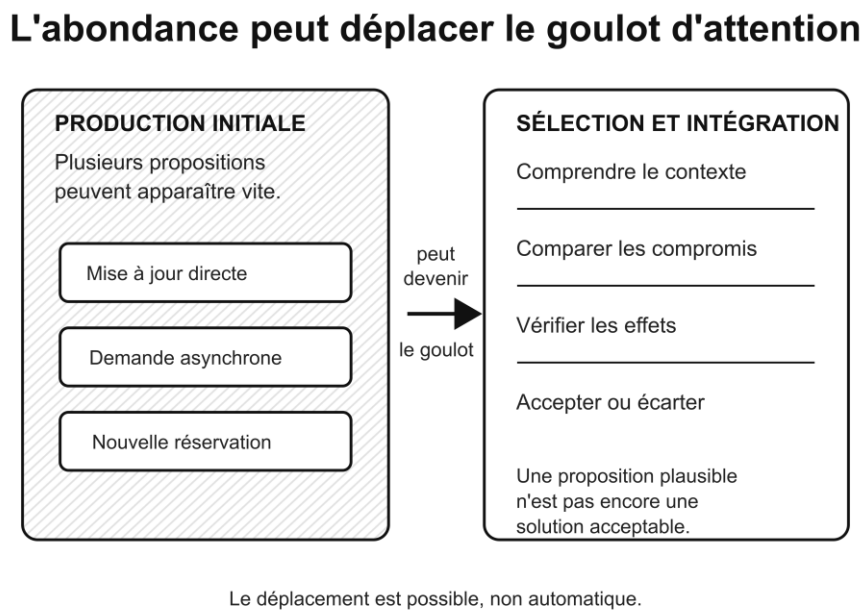


Figure 1.3 - Scénario construit. L'abondance de propositions déplace l'attention vers leur sens et leur intégration.

Ce déplacement n'est pas une loi mécanique. Une équipe peut encore manquer de compétences, d'accès ou de capacité de conception, et une proposition générée peut être trop faible pour mériter une comparaison. Le chapitre suivant examinera seulement les situations

où une accélération locale existe réellement, afin de distinguer ce gain du bilan complet du travail.

Le choix reste devant l'équipe

Revenons aux créneaux de livraison. L'équipe ne choisit pas encore entre la mise à jour directe, la demande asynchrone et la nouvelle réservation. Elle n'a pas assez d'éléments pour le faire correctement. Elle sait cependant que ces trois propositions n'ont pas le même sens et que produire une quatrième variante ne résoudrait pas, à lui seul, l'arbitrage.

Cette constatation évite de traiter l'IA comme une promesse de réponse immédiate ou comme une source de bruit à contenir. L'outil a fourni de la matière utile. Il a rendu plus perceptibles les décisions qui attendaient derrière la demande. L'équipe peut maintenant chercher ce qu'elle doit comprendre, formuler les contraintes pertinentes et décider quel résultat elle veut réellement rendre possible.

Le code devient parfois abondant. La capacité à savoir quel code mérite d'entrer dans le système ne devient pas abondante par le même mouvement.

> **Dans une pratique SDD** `constitution.md` fixe les principes du projet et `spec.md` porte l'intention et les critères d'une fonctionnalité. `plan.md` et `tasks.md` peuvent ensuite dériver du besoin plutôt que de la première solution disponible.

La prochaine question est donc moins « à quelle vitesse pouvons-nous produire ? » que « quel travail avons-nous réellement accéléré, et quel travail avons-nous seulement déplacé ? » C'est la question de la fausse productivité.

Sources et portée

Sources et portée

peng. Peng, S., Kalliamvakou, E., Cihon, P. et Demirer, M., *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*, février 2023 ; texte intégral, vérifié le 19 septembre 2026. L'étude mesure une tâche JavaScript standardisée et son temps d'achèvement ; elle ne mesure ni la qualité en production ni la productivité d'une équipe sur la durée.

Chapitre 2 - La fausse productivité

Les pull requests arrivent plus vite. Sur le tableau de l'équipe, les tickets quittent la colonne « à faire », les premières versions sont disponibles le jour même et les demandes qui restaient autrefois plusieurs jours sans réponse ont désormais une proposition concrète.

Ce signal mérite d'être regardé. Il peut correspondre à un gain réel. Une personne peut chercher moins longtemps, préparer plus vite une migration, écrire une première suite de tests ou explorer une variante qui aurait auparavant attendu. Il serait absurde de nier cette accélération locale au seul motif qu'elle ne résout pas tout.

Mais que permet-elle de conclure sur la capacité de l'équipe à livrer une valeur fiable ?

Produire une première version et livrer un changement compris, accepté et durable ne sont pas la même activité. Entre les deux, il reste du travail : comprendre le contexte, vérifier ce que le changement engage, corriger ce qui ne tient pas, coordonner son intégration et répondre de ses effets. Ce travail peut diminuer lui aussi. Il peut aussi devenir le nouveau goulot. La fausse productivité apparaît lorsque nous prenons le débit visible pour le bilan complet.

Le kit qui avance en surface

Considérons une équipe qui maintient un kit de développement partagé par plusieurs produits internes. Le kit fournit des conventions d'authentification, de journalisation et de gestion d'erreurs. Il évolue avec les versions des dépendances et les usages de ses produits consommateurs.

Avec un agent IA, les développeurs produisent rapidement des correctifs de compatibilité, améliorent les exemples, adaptent des versions et ajoutent des tests. Les pull requests sont plus nombreuses. Certaines sont utiles immédiatement. Un exemple manquant est clarifié, un avertissement de compilation disparaît, une dépendance obsolète est remplacée. Le débit visible n'est pas fictif : davantage de matière est bien produite.

Le problème apparaît au moment de l'intégration. Les quelques mainteneurs qui connaissent les contrats réels du kit doivent reconstituer l'intention de chaque changement. Un correctif de compatibilité paraît isolé, mais modifie peut-être le comportement attendu par un produit plus ancien. Deux pull requests paraissent indépendantes, mais elles ne peuvent pas être fusionnées dans n'importe quel ordre. Une variante déjà proposée ailleurs revient sous une forme différente, sans que son auteur ait connaissance de la décision précédente.

Les mainteneurs ne sont pas devenus plus lents par défaut. Ils assurent un travail que le débit des pull requests ne montre pas : relier une proposition locale à des versions, des contrats et des usages qui ne sont pas tous présents dans le ticket. Leur connaissance est une capacité rare du système de travail. Lorsque les premières propositions deviennent abondantes, cette capacité peut être sollicitée plus vite qu'elle ne se renouvelle.

Le tableau de suivi affiche donc deux histoires. La première est vraie : les premières versions arrivent plus tôt. La seconde est vraie aussi : la file d'attente s'est déplacée vers la compréhension, la revue et l'intégration. Aucune n'autorise, seule, un verdict sur la productivité. On se rendra d’ailleurs peut-être compte qu’il faudra réorganiser les équipes pour gagner en productivité : moins de profils débutants qui fabriquent le code, et plus de profils plus capés pour superviser.

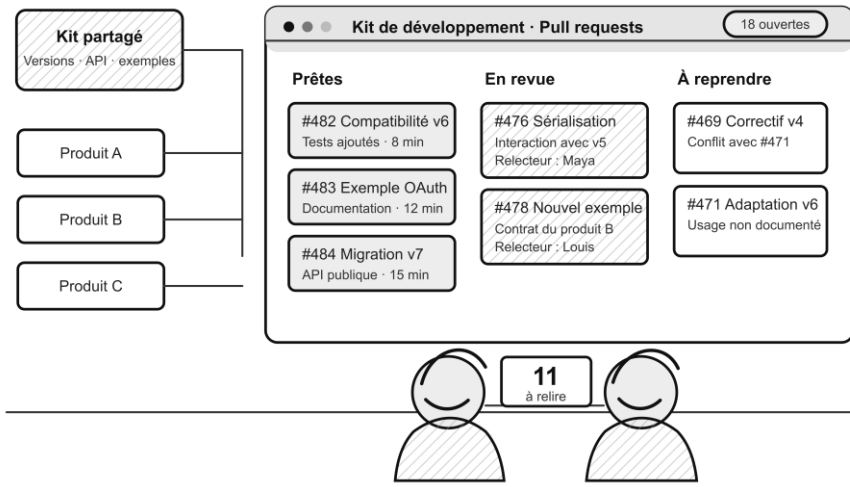


Figure 2.1 - Scénario construit. Les pull requests s'accumulent plus vite que les mainteneurs ne peuvent relier chaque changement aux usages du kit partagé.

Prenons une correction de compatibilité apparemment modeste. Son auteur a retrouvé l'API modifiée, appliqué la mise à jour recommandée et ajouté un test qui passe. Le mainteneur doit encore savoir si tous les produits consommateurs utilisent la même version, si un comportement ancien était volontairement conservé et si les exemples publics du kit créent une attente qui dépasse le module modifié. Il ne relit pas seulement une différence de code. Il relie une proposition à une histoire de décisions dont une partie n'est inscrite ni dans le ticket ni dans le dépôt.

Cette activité peut être réduite : une meilleure description de la demande, une compatibilité déjà documentée ou une proposition qui rend ses hypothèses visibles peuvent éviter une partie de l'enquête. Elle peut aussi augmenter lorsque plusieurs variantes arrivent sans contexte comparable. Nommer cette possibilité évite d'opposer les auteurs des pull requests aux mainteneurs. Les uns et les autres contribuent à un même changement, mais ils ne rendent pas visible le même coût.

Deux diagnostics insuffisants

Le premier diagnostic serait de dire : « Nous ouvrons davantage de pull requests, donc nous sommes plus productifs. » Il confond l'apparition d'une proposition avec le résultat du système. Une pull request ouverte ne dit ni quand le changement sera intégré, ni combien de reprises il demandera, ni si le produit consommateur en retirera une valeur utilisable.

Le diagnostic inverse serait tout aussi rapide : « Les mainteneurs sont saturés, donc l'outil ne produit que de la dette. » Lui aussi confond un signal avec une conclusion. Une revue supplémentaire peut absorber tout le gain local, mais elle peut également en absorber une partie seulement. Elle peut même faire apparaître une incompatibilité avant sa diffusion et améliorer la qualité du changement.

Le déplacement d'un coût n'est pas encore son bilan. Il devient dangereux lorsque l'équipe cesse de le voir, ou lorsqu'elle fait peser sans le dire sur quelques personnes le travail nécessaire pour garder le système cohérent. Ce déplacement des valeurs, ces changements de perspectives devront d'ailleurs probablement obliger les DSI et les entreprises à revoir leur grille de lecture pour juger de la qualité et de la productivité d'une équipe.

Ce que le bilan doit rendre visible

Il n'existe pas ici de score universel. Les dimensions suivantes ne sont pas des KPI à déployer ni un tableau de bord à remplir. Elles aident à ne pas donner le même nom à des réalités différentes.

Dimension	Question utile	Ce qu'elle ne permet pas de conclure seule
Effort cumulé	Combien de travail ont nécessité production, compréhension, revue, correction et coordination ?	Le calendrier de livraison, car une partie du travail peut être parallèle ou attendre.

Dimension	Question utile	Ce qu'elle ne permet pas de conclure seule
Délai	Combien de temps s'écoule entre le besoin et le changement intégré ?	La valeur ou la qualité du changement.
Qualité	Le changement respecte-t-il les propriétés attendues et évite-t-il une reprise prévisible ?	Que l'effort engagé était excessif ou inutile.
Valeur	Le changement permet-il réellement le résultat recherché pour les personnes concernées ?	Quel mécanisme a produit ce résultat ni si le gain se répète ailleurs.

L'effort cumulé rend visible ce qui disparaît des statistiques de saisie. Dans le scénario, il comprend le temps pour préparer une proposition, mais aussi celui des mainteneurs qui cherchent les usages d'une option, lisent une modification, expliquent une incompatibilité et accompagnent une reprise. Le délai rend visible une autre réalité : une proposition créée en une heure peut rester plusieurs jours dans une file de revue. La qualité oblige à demander ce qui est effectivement préservé, plutôt que de traiter chaque changement intégré comme une victoire. La valeur rappelle enfin que même un changement rapide et correct peut ne pas résoudre le besoin qui l'avait motivé.

Ces dimensions se contraignent sans se remplacer. Diminuer le délai en supprimant une vérification peut dégrader la qualité. Ajouter une vérification peut augmenter l'effort tout en évitant une reprise très coûteuse. Livrer moins de changements peut parfois accroître la valeur si l'équipe a cessé de produire des variantes sans usage. Une équipe qui veut comprendre son gain doit donc regarder les tensions de son propre travail, pas additionner des indicateurs hétérogènes jusqu'à obtenir un nombre rassurant.

Le délai mérite une attention particulière parce qu'il mélange production, attente et coordination. Une première version préparée en trente minutes peut rester bloquée trois jours parce qu'une seule personne connaît le contrat concerné. Réduire le temps de saisie ne

réduit pas automatiquement cette attente. Inversement, une revue plus longue peut raccourcir le délai global si elle révèle assez tôt une incompatibilité qui aurait imposé une reprise après publication. L'équipe ne doit pas confondre cette observation avec un argument pour ralentir chaque changement ; elle doit localiser l'attente qui limite réellement sa livraison.

La qualité ne se réduit pas non plus à l'absence d'une erreur immédiatement visible. Dans le kit partagé, une modification peut préserver les tests du module tout en introduisant une divergence qui ne se révélera que chez un produit consommateur. À l'inverse, une revue attentive peut découvrir une incompatibilité, imposer une reprise et améliorer le changement avant son intégration. Si l'équipe ne regarde que le temps passé, elle peut traiter ce travail comme une perte. Si elle ne regarde que le nombre de changements intégrés, elle peut ignorer le coût qui aurait été évité. Il ne faudrait pas non plus réduire les agents IA à des machines à produire du code. Les IA frontières vont tellement loin qu'elles savent aussi trouver des failles ignorées des informaticiens, des bogues potentiels qui seraient restés là encore longtemps. Ces agents peuvent donc, à l'extrême, faire « perdre du temps » parce qu'ils découvrent bien plus de problèmes avérés ou potentiels qu'une équipe de bon niveau. Ainsi le temps peut s'étirer, et selon les critères habituels, la productivité peut sembler chuter. Mais au bénéficie d'une qualité améliorée. La productivité n'est donc pas le seul indicateur, la qualité du produit final joue, ou devrait jouer, pour une part encore plus importante. Produire plus vite de mauvais softs ou plus lentement de meilleurs ? Si c'était la seule alternative qu'est ce que votre direction choisirait, et vous ? Heureusement, les choses ne sont pas si radicales, et la réalité est faite de tous ces mouvements divergents et alternatives opposées à la fois, la grille de lecture permettant de savoir si le job a été fait est donc bien plus subtile qu'une productivité mesurée même finement…

La valeur pose une question encore différente. Une amélioration d'exemple, une correction de compatibilité ou un test supplémentaire ne contribuent pas tous de la même manière à la capacité des produits consommateurs à avancer. Une équipe peut produire plus de modifications et livrer moins de valeur si les changements les plus

importants attendent derrière une file de compréhension. Elle peut également choisir de différer quelques variations faciles afin de libérer la capacité rare qui permet d'intégrer un changement déterminant. Le but n'est pas de hiérarchiser chaque pull request par une formule. Il est de rendre visible que le débit ne contient pas à lui seul cette hiérarchie.

Le goulot ne disparaît pas, il change de forme

Quand la première version d'une modification demandait plusieurs jours, la production pouvait constituer le goulot évident. Avec l'assistance, ce goulot peut se déplacer vers la personne capable de relier le changement à l'historique du kit, aux contrats des consommateurs et aux décisions déjà prises. Cette personne n'est pas un contrôle bureaucratique ajouté à la fin. Elle détient, parfois de manière incomplète, une connaissance dont dépend la possibilité d'intégrer sans surprendre les produits utilisateurs.

Le déplacement devient visible dans des gestes ordinaires. Le mainteneur demande pourquoi une option a été introduite, retrouve une incompatibilité qui avait été acceptée pour un produit ancien, explique qu'un exemple public est devenu un engagement ou constate que deux correctifs se contredisent. Aucun de ces gestes n'est spectaculaire. Ensemble, ils peuvent faire de la revue l'endroit où une équipe gagne ou perd sa capacité à livrer.

La réponse n'est pas de réserver toute production aux mainteneurs. Elle peut consister à rendre les hypothèses plus visibles dans la proposition, à documenter les contrats récurrents, à réduire le périmètre d'une évolution ou à donner aux auteurs un retour qui rend la prochaine proposition plus facile à examiner. Le système peut apprendre. *Mais il doit d'abord reconnaître l'endroit où l'attention rare est réellement consommée*.

Un coût déplacé n'est pas forcément une perte nette

Imaginons deux variantes fictives d'un changement comparable. Dans chacune, l'assistance IA fait économiser quatre heures de production initiale. Dans la première, la proposition demande une heure

supplémentaire de revue et de coordination. À qualité comparable, il reste trois heures d'effort cumulées économisées. Le gain local a été réduit, non annulé.

Dans la seconde, la proposition demande six heures de revue, d'explication et de reprise. À qualité comparable, l'effort total augmente de deux heures. Le gain de production existe toujours comme phénomène local, mais il ne devient pas un gain pour l'ensemble du travail.

Ces exemples ne mesurent pas une équipe réelle. Ils séparent seulement trois possibilités trop souvent confondues : un gain préservé, un gain réduit et une perte nette. Ils séparent aussi l'effort du délai. Six heures réparties entre plusieurs personnes peuvent ne pas ajouter six heures au calendrier ; une heure de revue qui attend une personne rare peut, elle, retarder fortement une livraison. La qualité et la valeur restent encore d'autres questions.

L'intérêt de cette distinction n'est pas de faire calculer chaque changement. Elle rend une conversation plus honnête. Si la revue devient le goulot, faut-il améliorer les informations données avec la proposition, réduire le périmètre du changement, retrouver une règle manquante, demander un autre avis ou accepter que ce sujet ne soit pas encore prêt à être intégré ? Le bon geste dépend de la limite réellement rencontrée.

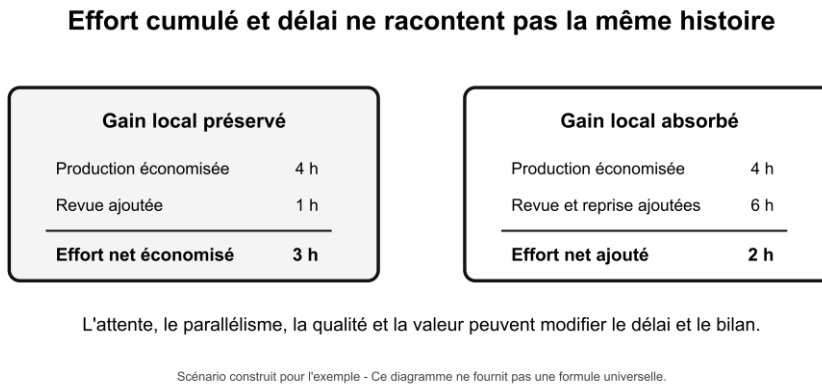


Figure 2.2 - Scénario construit. Un coût déplacé peut réduire, préserver ou dépasser un gain local.

Les études créent une tension, pas une moyenne

Les travaux disponibles rendent cette distinction difficile à ignorer, mais ils ne fournissent pas une moyenne que l'on pourrait appliquer à toute équipe.

Dans un essai contrôlé mené en 2022, Peng et ses collègues ont demandé à 95 développeurs qu'ils avaient recrutés d'implémenter un petit serveur HTTP en JavaScript. Parmi les personnes ayant achevé la tâche, le groupe disposant de Copilot a mis en moyenne 71,17 minutes, contre 160,89 minutes pour le groupe contrôle.[^peng] Le résultat renseigne une tâche autonome, courte et évaluée par douze tests fournis. Il ne mesure ni la maintenance d'un système mature ni la capacité durable d'une équipe à intégrer des changements.

Un essai contrôlé mené auprès de 96 ingénieurs Google sur une tâche expérimentale complexe de niveau entreprise a, lui aussi, estimé une réduction centrale du temps consacré à la tâche, d'environ 21 %. Les auteurs signalent un intervalle de confiance large et ne généralisent pas ce résultat à d'autres outils, périodes ou organisations.[^paradis] Il prolonge l'idée qu'un gain est possible dans une tâche plus complexe, sans mesurer pour autant l'histoire complète d'un produit en production.

Trois expériences randomisées menées dans le travail ordinaire de Microsoft, d'Accenture et d'une entreprise du Fortune 100 élargissent encore le tableau. En réunissant 4 867 développeurs, leurs auteurs estiment une hausse de 26,08 % du nombre de tâches achevées, avec une erreur standard de 10,3 % et des résultats variables selon les expériences.[^cui] L'échelle et le contexte industriel donnent du poids au gain observé. L'indicateur reste toutefois un nombre de tâches achevées : il ne contient pas, à lui seul, leur valeur ni leur coût de maintenance futur.

Le contraste est net avec une expérience de METR publiée en 2025. Seize mainteneurs open source expérimentés ont réalisé 246 tâches réelles dans des dépôts matures qu'ils connaissaient. Avec les outils IA étudiés, le temps de réalisation a été estimé 19 % plus long. Les auteurs observent moins de temps de codage et de recherche directe,

mais davantage de temps de prompting, d'attente et de revue.[^metr] Ce résultat ne signifie pas que l'IA ralentit tout développement. Il indique qu'une tâche réelle, riche de connaissance tacite et intégrée à un dépôt connu, ne réagit pas nécessairement comme une tâche autonome.

Une prépublication de Xu et ses collègues va dans une direction voisine, avec un autre dessin d'étude. Elle associe l'adoption de Copilot dans des projets open source à une production accrue chez des contributeurs périphériques, tandis que les contributeurs centraux voient augmenter leur charge de revue et de maintenance. Son statut de prépublication et son contexte observationnel imposent une prudence particulière.[^xu] Elle ne démontre pas que l'outil produit toujours de mauvais changements ni qu'une cause unique explique la charge des mainteneurs. Elle rend toutefois plausible la tension du kit : un gain local peut coexister avec une pression accrue sur les personnes qui garantissent la cohérence.

Une expérience contrôlée publiée en 2026 a suivi un autre effet aval : 151 participants, dont 95 % de professionnels, ont d'abord fait évoluer une application Java avec ou sans agent IA, puis d'autres participants ont repris ces solutions sans assistance. Les auteurs ne détectent pas de différence significative dans le temps d'évolution ultérieur ni dans la qualité mesurée.[^borg] Ce résultat ne prouve pas l'absence de tout coût différé. Il empêche en revanche de traiter la dégradation de la maintenabilité comme une conséquence automatique de l'assistance.

METR a publié en février 2026 une mise à jour d'une expérience ultérieure dont les estimations centrales suggéraient plutôt une accélération. Les intervalles incluaient l'absence d'amélioration, et l'organisation a souligné des biais de sélection ainsi que la difficulté de mesurer du travail agentique parallèle.[^metr-update] Cette évolution est importante. Elle interdit de transformer le ralentissement de 2025 en vérité permanente. Elle renforce une conclusion moins spectaculaire et plus utile : l'effet dépend de la tâche, de l'outil, des personnes et du système de travail.

Pourquoi aucune moyenne ne se transpose

Demander quelle étude « a raison » suppose qu'elles observent le même objet. Ce n'est pas le cas. Une tâche autonome avec un résultat défini, une tâche expérimentale de niveau entreprise, des maintenances réelles dans des dépôts connus et une observation de communautés open source ne rendent pas le même travail visible. Les indicateurs diffèrent également : temps d'achèvement, temps consacré à une tâche, contribution produite, charge de revue ou reprises.

Cette diversité ne rend pas les études inutilisables. Elle interdit de les convertir en une moyenne rassurante. Un gain mesuré sur une tâche bornée ne devient pas une promesse de livraison durable. Un ralentissement observé chez des mainteneurs expérimentés ne devient pas un procès de toute assistance. Ensemble, ces résultats rendent plus difficile la réponse automatique et plus nécessaire l'examen du travail concret de chaque équipe.

Une revue systématique consacrée à la productivité des développeurs avec ChatGPT a recensé 26 articles et les a organisés autour des usages, de la collaboration humain-IA et de leurs effets sur la productivité.[^ebrahim] Elle confirme l'existence d'un champ empirique, mais ne fournit pas une méta-analyse quantitative dont l'effet agrégé serait transposable à une équipe. Son périmètre centré sur ChatGPT ne couvre d'ailleurs pas à lui seul tous les agents de développement actuels.

Le scénario du kit ne cherche pas à reproduire leurs chiffres. Il donne une forme pédagogique à une tension qu'elles rendent plausible : un gain dans la production initiale peut coexister avec une pression accrue là où une connaissance rare doit interpréter, vérifier et intégrer. Cette pression peut être réduite par de meilleures conditions de travail. Elle peut aussi absorber le gain. Le chapitre ne peut pas conclure lequel de ces résultats prévaudra pour une organisation donnée.

Une observation qui mène à une décision

Regarder le système n'implique pas d'installer une surveillance de chaque minute ou de transformer les équipes en producteurs d'indicateurs. Une observation utile commence souvent par quelques changements représentatifs. L'équipe peut se demander lesquels ont été intégrés sans reprise, lesquels ont attendu une information détenue par une personne précise, lesquels ont révélé une incompatibilité après une première revue et lesquels ont créé une valeur effectivement utilisée par un produit consommateur.

Ces questions peuvent changer une décision de travail. Si les mainteneurs découvrent toujours trop tard les mêmes hypothèses, l'équipe peut améliorer la description de la demande ou demander que les alternatives soient présentées avec leurs engagements. Si la file provient d'un contrat rarement explicité, elle peut documenter ce contrat ou isoler une migration. Si l'effort de revue augmente alors que les reprises diminuent fortement, elle peut reconnaître que ce coût supplémentaire protège un gain réel au lieu de l'appeler indistinctement « perte de productivité ».

L'observation n'a pas à produire un verdict définitif. Elle doit permettre de choisir ce qui mérite d'être changé dans le système de travail. Une équipe qui constate un goulot de connaissance ne répond pas nécessairement par plus de réunions. Elle peut décider de limiter une évolution, d'associer plus tôt la personne concernée, de créer une preuve plus proche du contrat ou de reconnaître qu'un gain local ne doit pas encore être diffusé à toute l'organisation.

Regarder le système sans accuser l'outil

La question utile n'est donc pas de savoir si une équipe doit aimer ou refuser l'assistance IA. Elle est de localiser ce qu'elle accélère et ce qu'elle limite maintenant. Le goulot peut se trouver dans le cadrage d'une demande, dans une connaissance dispersée, dans la capacité de revue, dans l'intégration entre systèmes, dans la correction d'une erreur ou dans l'accès à la personne qui peut accepter un compromis.

Cette lecture n'impose pas une revue exhaustive de toute ligne générée, ni un cérémonial identique pour chaque changement. Une modification triviale, réversible et bien comprise n'appelle pas le même effort qu'une évolution qui affecte plusieurs produits. Il s'agit d'abord de rendre visible la nature de la limite, afin de ne pas répondre à une saturation de revue par une pression supplémentaire sur la production.

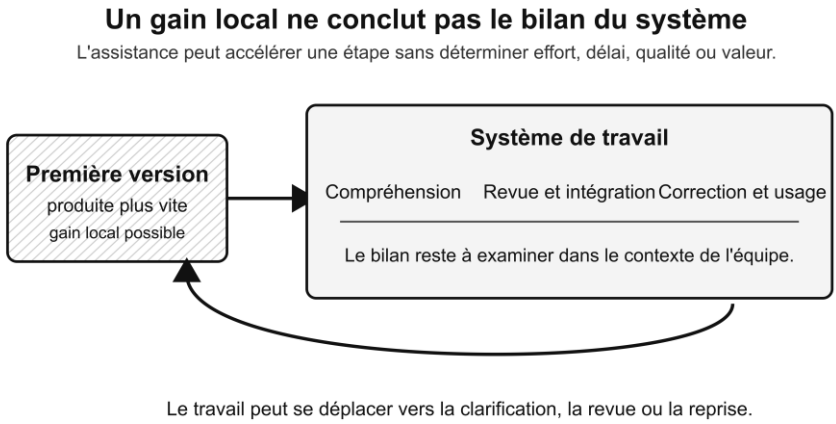


Figure 2.3 - Cadre proposé par le livre. Une accélération peut porter sur une étape sans déterminer le résultat du système de travail.

Une équipe ne gagne pas en maturité parce qu'elle a inventorié tous ses coûts. Elle progresse lorsqu'elle peut dire, dans une situation précise, ce qui a réellement disparu, ce qui s'est déplacé et ce qui reste à vérifier. Cette conversation peut être brève. Elle devient nécessaire lorsque le débit donne une impression de progrès que la capacité d'intégration ne confirme pas.

Le gain n'a pas besoin d'être total pour compter

Reconnaître un coût déplacé ne demande pas de refuser les gains partiels. Une proposition plus rapide peut libérer du temps pour comprendre une dépendance, préparer une preuve ou comparer plusieurs solutions. Une revue qui absorbe une part de ce temps peut rester un bon échange si elle rend un changement plus sûr ou évite une reprise coûteuse. Le problème apparaît lorsque l'équipe appelle

indistinctement « gain de productivité » ce qui n'est encore qu'une accélération dans une étape particulière.

Cette distinction protège aussi contre une réaction de rejet. Si les mainteneurs sont sous pression, l'équipe peut être tentée de réduire l'usage de l'assistance sans examiner le mécanisme. Elle perdrait peut-être un gain réel dans les tâches où la proposition initiale est bornée et facilement vérifiable. À l'inverse, si elle ne regarde que le débit, elle peut étendre un usage qui crée une file d'attente durable chez les personnes les plus rares. Dans les deux cas, la décision doit partir du travail observé, non d'une position générale sur l'outil.

Le gain qui subsiste peut prendre des formes différentes. Il peut réduire l'effort cumulé, raccourcir le délai, améliorer la qualité à effort comparable ou libérer l'attention nécessaire à un changement plus important. Dire lequel de ces effets est recherché rend la discussion plus utile. Cela évite de demander à un seul nombre de représenter l'ensemble de la valeur produite par une équipe.

Du coût déplacé au choix implicite

Les reprises ne sont pas toujours seulement des heures perdues. Elles révèlent parfois qu'une décision avait été prise sans être formulée. Dans le kit, une pull request peut supposer qu'un comportement ancien est abandonné, qu'une compatibilité ne compte plus ou qu'une équipe consommatrice acceptera une migration. Le code donne une forme à cette hypothèse avant que les personnes concernées aient eu l'occasion de la confirmer ou de la contester.

La revue découvre alors deux choses à la fois : une modification à corriger et un choix à éclaircir. Si ce choix reste invisible, il reviendra sous une autre pull request, une exception, une note de support ou une régression. Le coût futur ne vient pas seulement du code à reprendre. Il vient du temps nécessaire pour reconstruire pourquoi une décision avait été prise, quelle information manquait et qui pouvait la trancher.

Le débit est un signal, pas une conclusion. Il peut être le début d'un gain réel, la source d'une file d'attente ou les deux à la fois. Pour le

comprendre, il faut regarder le travail entier plutôt que la seule étape devenue facile à observer.

> **Dans une pratique SDD** `spec.md`, `plan.md` et `tasks.md` rendent traçable le passage de l'attente au choix de conception puis au travail produit. Une étape `clarify` ou une `checklist` peut faire apparaître avant la revue une compatibilité ou une question, afin que le gain local reste interprétable à l'échelle du système de travail.

La question suivante est donc celle des choix qui prennent la forme du code avant d'être suffisamment explicites. Lorsqu'ils s'accumulent, le coût déplacé devient *une dette de décision*.

Sources et portée

Sources et portée

peng. Peng, S., Kalliamvakou, E., Cihon, P. et Demirer, M., *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*, février 2023 ; texte intégral, vérifié le 19 septembre 2026. L'essai porte sur une tâche JavaScript standardisée, non sur un système maintenu dans la durée.

paradis. Paradis, E. et al., *How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial*, *ICSE-SEIP 2025*, vérifié le 20 septembre 2026. L'étude estime le temps consacré à une tâche expérimentale complexe, non la performance durable d'une organisation.

cui. Cui, K. Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S. et Salz, T., *The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers*, *Management Science*, publication en ligne le 27 février 2026, vérifié le 20 septembre 2026. Les expériences mesurent les tâches achevées dans trois entreprises ; elles ne mesurent pas directement la maintenabilité à long terme.

metr. Becker, J., Rush, N., Barnes, B. et Rein, D., *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*, juillet 2025, vérifié le 17 septembre 2026. L'essai porte sur des mainteneurs open source expérimentés et des tâches réelles de leurs dépôts ; il ne mesure pas la gouvernance d'une organisation complète.

xu. Xu, F., Medappa, P. K., Tunç, M. M., Vroegindeweij, M. et Fransoo, J. C., *AI-Assisted Programming Decreases the Productivity of Experienced Developers by Increasing the Technical Debt and Maintenance Burden*, version 3, 28 janvier 2026, prépublication recontrôlée le 20 septembre 2026. La page signale des présentations en conférence, mais aucune publication en revue ; le résultat observationnel reste à corroborer.

borg. Borg, M. et al., *Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability*, *Empirical Software Engineering*, vol. 31, article 161, 2026, vérifié le 20 septembre 2026. L'expérience porte sur deux évolutions bornées d'une application Java et précède l'essor des agents de codage plus autonomes.

metr-update. Becker, J., Rush, N., Cunningham, T., Rein, D. et Mahamud, K., *We are Changing our Developer Productivity Experiment Design*, février 2026, vérifié le 17 septembre 2026. La mise à jour décrit des estimations difficiles à interpréter, notamment en raison de biais de sélection et du travail parallèle.

ebrahim. Ebrahim, S. et Tanner, M., *A Systematic Literature Review on the Use of ChatGPT to Support Productivity of Software Developers*, *AMCIS 2025 Proceedings*, article 17, vérifié le 20 septembre 2026. La revue porte sur 26 articles, sur ChatGPT et sur plusieurs dimensions de la productivité ; ce n'est pas une méta-analyse quantitative.

Chapitre 3 - La dette de décision

Une solution peut être propre, cohérente, testée et pourtant nous engager plus loin que nous ne le croyons.

Ce danger n'a pas attendu les assistants de code. Les systèmes accumulent depuis toujours des conventions, des raccourcis, des réglages et des choix qui n'ont jamais été vraiment posés comme tels. L'IA ne crée pas cette histoire. Elle peut en modifier le rythme : une interprétation plausible devient plus vite une proposition complète, puis une pull request convaincante, puis une dépendance dont il semble coûteux de discuter.

Le problème n'est pas qu'une équipe se trompe. Une équipe sérieuse se trompe parfois, même lorsqu'elle enquête, compare et vérifie. Le problème commence plus tôt : *lorsqu'une hypothèse utile à la production prend la forme d'une décision sans avoir été reconnue, discutée ou assumée*.

J'appelle ici **dette de décision** le coût futur d'un choix important resté implicite, reporté sans condition de réexamen, faiblement justifié ou fondé sur une connaissance incomplète prise pour acquise. Ce n'est pas une nouvelle catégorie de faute. C'est un angle pour voir ce qui se passe quand le code transforme une réponse provisoire en contrainte pour le reste du système.

Un nettoyage qui paraît sans risque

Le scénario qui suit est construit. Il ne décrit ni un client réel ni un incident documenté.

Une équipe prépare l'arrivée d'un nouveau partenaire d'intégration. Plusieurs services représentent une référence client de manières différentes. Les données contiennent `P-0042`, `42`, parfois `00042`. Les conversions sont dispersées : un service retire le préfixe, un autre garde les zéros initiaux, un troisième transmet la valeur telle qu'il la reçoit.

La demande paraît raisonnable : uniformiser les références avant d'ouvrir le nouvel échange. Un agent IA de code repère les transformations, propose une fonction commune, adapte les appels et génère les tests. La pull request supprime de la duplication. Les tests existants passent. Les valeurs envoyées au partenaire deviennent régulières.

Il serait facile de considérer le sujet comme réglé. Après tout, une référence plus homogène est plus simple à lire, à rechercher et à transmettre.

Mais un commentaire ancien attire l'attention : dans un canal historique, le préfixe et les zéros auraient pu distinguer une catégorie particulière de client. Personne ne sait encore si cette règle est toujours active. Les documents actuels ne le disent pas. Une partie des systèmes ignore déjà cette distinction ; une autre semble encore la conserver.

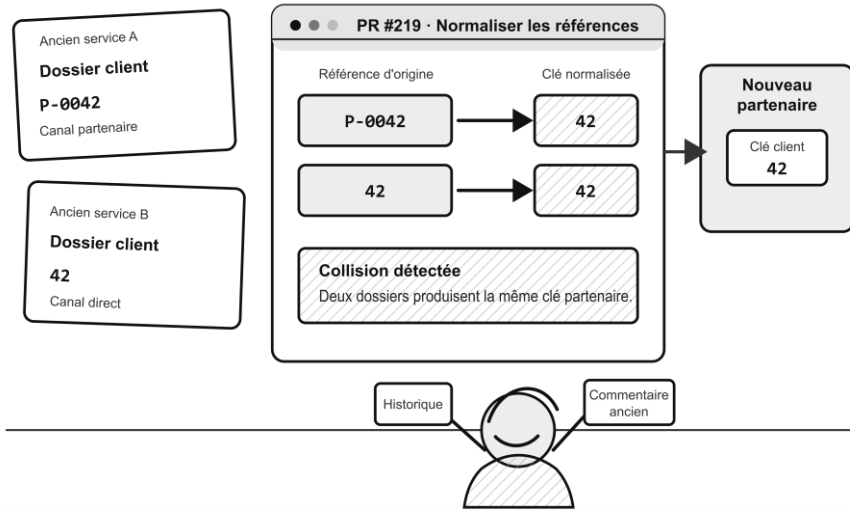


Figure 3.1 - Scénario construit. Deux références distinctes produisent la même clé avant que leur équivalence métier soit établie.

La fonction proposée fait ceci :

Référence reçue	Référence normalisée	Ce que le calcul établit	Ce qu'il n'établit pas

Référence reçue	Référence normalisée	Ce que le calcul établit	Ce qu'il n'établit pas
`P-0042`	`42`	La transformation produit `42`.	Que `P-0042` et `42` désignent le même client métier.
`42`	`42`	La valeur est déjà sous la forme attendue.	Que cette forme est suffisante dans tous les échanges.
`00042`	`42`	Les zéros initiaux sont retirés.	Que les zéros n'ont jamais porté de distinction utile.

Le nouveau partenaire n'utilise que cette valeur normalisée comme identifiant. Les références d'origine restent dans l'historique, mais cette conservation ne répare pas automatiquement une association déjà propagée chez le partenaire. Une représentation dérivée qui garderait la valeur originale serait une autre décision de conception ; elle ne démontrerait pas, à elle seule, l'absence de collision.

La proposition est donc techniquement plausible. Elle peut aussi être dangereuse. Non parce qu'elle est manifestement mauvaise, mais parce qu'elle encode une affirmation silencieuse : les différences de format ne comptent plus pour l'identité.

Quand une hypothèse devient une contrainte

Le code a une propriété redoutablement utile : il rend les choses exécutables. Cette même propriété lui donne un poids particulier dans une équipe. Dès qu'une règle est mise en œuvre, elle devient plus facile à réutiliser, à copier dans un test, à décrire dans une API et à traiter comme un acquis.

Dans notre scénario, personne n'a écrit : « Nous décidons que `P-0042` et `42` désignent toujours le même client. » *Pourtant, intégrer la normalisation et la donner au partenaire produit pratiquement cet effet*.

La suite est prévisible, sans être certaine. Un second service s'appuiera sur la clé normalisée. Un test vérifiera qu'elle est envoyée. Une

documentation d'intégration l'appellera peut-être « identifiant client ». Dans quelques mois, un changement nécessitera de comprendre pourquoi deux enregistrements différents se comportent de la même manière. L'équipe devra alors retrouver non seulement une règle technique, mais aussi le choix métier que cette règle a rendu implicite.

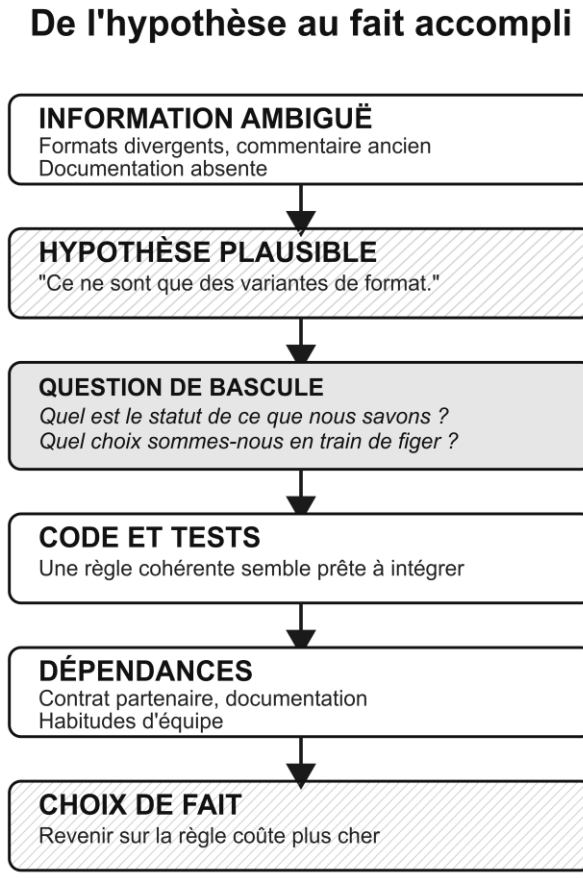


Figure 3.2 - Cadre proposé par le livre. De l'hypothèse au fait accompli : une propagation possible, non un processus obligatoire.

La figure ne dit pas qu'il faut arrêter toute production dès qu'une information manque. Elle montre autre chose : l'endroit où une équipe peut reprendre de la liberté. Avant que l'interprétation ne devienne un contrat, il reste possible de demander ce qui est établi, ce qui est seulement plausible, et quel choix est en train d'être produit.

Cette pause est souvent courte. Elle ne consiste pas à instruire toutes les questions imaginables. Elle vise l'incertitude qui conditionne une décision difficile à annuler.

> **Repère de lecture** Une exigence indique ce qui doit être préservé. Un fait établi indique ce que les éléments disponibles permettent de tenir pour vrai. Décider de préserver une propriété ne prouve pas encore que la solution la respecte.

Ici, l'exigence peut être claire : chaque référence active doit continuer à désigner le même client dans les systèmes concernés. Mais cette exigence ne répond pas à la question : *les formes historiques représentent-elles aujourd'hui la même identité ?* La confondre avec une réponse, c'est transformer la volonté de bien faire en preuve de conformité.

La propreté du code ne tranche pas le sens

La difficulté du scénario ne vient pas d'une fonction mal écrite. Elle vient précisément du contraire : la fonction est séduisante parce qu'elle rassemble plusieurs cas sous une même règle et parce qu'elle retire du code dupliqué. C'est un bénéfice réel. Nous avons besoin de ce type de simplification pour maintenir un système lisible.

Mais une simplification a toujours un domaine de validité. Elle affirme que des différences auparavant conservées ne sont plus pertinentes pour le comportement concerné. Dans beaucoup de cas, cette affirmation est juste. Dans d'autres, elle réduit une distinction qui avait un sens dans un contrat, une règle métier ou une exception dont nous ne disposons plus de la mémoire.

Les signaux techniques restent indispensables, mais chacun répond à une question limitée.

Signal disponible	Ce qu'il rend crédible	Ce qu'il ne décide pas
La fonction est plus courte et utilisée partout.	La transformation est plus homogène qu'avant.	Que le format supprimé ne porte aucune signification.
Le code compile et les	Les comportements déjà	Que les tests couvrent les

Signal disponible	Ce qu'il rend crédible	Ce qu'il ne décide pas
tests connus passent.	décrits par ces tests restent cohérents.	identités historiques ou les contrats externes.
Aucun incident récent n'a été lié au préfixe.	Aucun problème n'a été observé dans le périmètre connu.	Que l'absence de signal soit une preuve d'équivalence.
Un collègue se souvient que le préfixe était « ancien ».	Il existe une piste utile pour l'enquête.	Que la règle a été abandonnée, ni qui peut l'abandonner aujourd'hui.

Ce tableau ne dévalorise pas les tests ni l'expérience. Il les remet à leur juste place. Une suite de tests vérifie les propriétés qu'elle exprime. Si aucun test ne distingue `P-0042` de `42`, son succès ne dit rien de cette distinction. De même, l'absence d'incident peut signifier que la situation est rare, qu'elle est restée confinée, ou qu'elle n'a pas encore rencontré le partenaire qui rendrait la collision visible.

La qualité de l'implémentation et la qualité de la décision ne sont donc pas rivales. Une équipe mature veut les deux. Elle cherche une transformation claire, puis elle s'assure que la règle rendue claire est bien celle qu'elle a l'autorité et les éléments d'adopter.

Ce point devient plus important lorsque la production est assistée par une IA générative. Un agent IA peut très bien proposer la fonction la plus élégante à partir des informations qu'il reçoit. Il n'a pas besoin de mal raisonner pour manquer la distinction métier absente du dépôt. Sa fluidité peut même rendre le vide moins visible : il transforme des indices incomplets en une solution si cohérente que nous oublions de demander si la question avait été posée.

Le geste utile n'est pas de soupçonner chaque simplification. Il est de repérer celles qui changent la signification d'une donnée, la portée d'un engagement ou la réversibilité d'une décision. Dans ces cas, « les tests passent » reste une bonne nouvelle. Ce n'est pas encore le dernier mot.

Nommer la dette sans tout confondre

La dette de décision n'est pas un synonyme élégant de dette technique.

Le Software Engineering Institute définit la dette technique comme une approche de conception ou de construction avantageuse à court terme qui crée un contexte dans lequel le même travail coûtera davantage plus tard. Cette définition est utile ici : elle nous rappelle qu'un choix local peut renchérir le changement futur, y compris lorsque l'équipe avait de bonnes raisons d'avancer vite.[^sei]

Mais notre sujet est différent. La dette de décision regarde le choix lui-même : *a-t-il été reconnu ? Quels éléments le justifient ? Quelle information manque-t-il ? Qui peut assumer sa conséquence ?* Elle peut recouper la dette technique, sans s'y réduire.

Question	Dette technique	Dette de décision
Point d'attention	Une structure de conception ou d'implémentation qui renchérit le travail futur.	Un choix qui progresse sans être assez explicite, justifié ou réexaminable.
Exemple dans le scénario	Une transformation dispersée, difficile à modifier de façon cohérente.	Tenir pour équivalentes deux références dont le sens métier n'est pas établi.
Ce que le coût tardif peut révéler	Une architecture, une duplication ou une dépendance devenue coûteuse.	Une hypothèse, un compromis ou une autorité qui n'avait pas été mise au jour.
Ce que le chapitre ne dit pas	Que toute dette technique révèle une mauvaise décision.	Que tout report ou toute incertitude crée une dette.

Le chevauchement est réel. Une normalisation erronée peut créer plus tard de la duplication, des correctifs et des contrats difficiles à faire évoluer : il y aura alors aussi un problème technique. Inversement, une décision parfaitement explicite peut produire une dette technique parce qu'un compromis assumé était nécessaire à une date donnée.

Cette distinction évite deux erreurs. La première consiste à moraliser le passé : si le système est compliqué, quelqu'un aurait forcément mal décidé. La seconde consiste à réduire tous les problèmes futurs à la qualité du code. Il arrive qu'une base soit nette, testée et documentée, tout en portant une décision que personne n'est encore capable d'expliquer.

La dette de décision n'accuse donc pas. Elle demande : *quel choix allons-nous rendre plus coûteux à rouvrir en intégrant ceci ?*

Quatre manières de la laisser s'accumuler

Le mécanisme ne se présente pas toujours comme une grande décision. Il s'accumule souvent à travers de petits gestes raisonnables.

Mécanisme	**Dans le scénario**	**Coût qui peut apparaître plus tard**
Décision par défaut	La première normalisation qui passe les tests devient la règle commune.	Il faut défaire des dépendances qui n'avaient jamais été choisies comme politique d'identité.
Connaissance incomplète présentée comme suffisante	Un commentaire ancien est lu comme une curiosité plutôt que comme un indice à investiguer.	L'équipe doit reconstituer sous pression le sens d'un format historique.
Temps de contestation raccourci	Une proposition complète est disponible avant que la question métier soit formulée.	Le code paraît trop avancé pour être remis en discussion, alors que l'arbitrage n'a jamais eu lieu.
Coût révélé après propagation	Le partenaire reçoit une clé déjà normalisée et construit ses propres associations.	Une correction suppose des rapprochements, des exceptions, des migrations ou une coordination externe.

Ces mécanismes ne sont pas propres à l'IA. Ils existaient avec les scripts copiés-collés, les décisions prises dans un couloir et les délais de livraison. L'IA ajoute une propriété importante : elle rend très peu coûteuse la fabrication d'une réponse cohérente en apparence. Elle réduit potentiellement le temps entre « nous avons une interprétation » et « nous avons un changement qui semble prêt ».

Ce raccourcissement ne rend personne moins compétent. Il réduit simplement une friction qui, parfois, laissait le temps aux objections d'émerger. Lorsque plusieurs options arrivent en quelques minutes, l'équipe doit choisir délibérément où maintenir l'espace de contestation. Sans cela, le moment où l'on pouvait encore dire « nous ne savons pas » se déplace après l'intégration, lorsque revenir en arrière coûte davantage.

Il faut rester précis. Une livraison rapide peut être le bon choix. Une hypothèse peut être parfaitement acceptable si son coût d'erreur est faible et si l'équipe peut la corriger. Ce chapitre ne propose pas une règle de lenteur. Il rend visible ce qui change lorsqu'une question importante cesse d'être une question parce que le code a déjà répondu à sa place.

Le coût tardif ne ressemble pas toujours à une dette

Le mot « dette » peut faire imaginer un montant connu, une échéance et un responsable clairement identifiés. Dans le travail d'ingénierie, le coût d'une décision implicite apparaît plus souvent sous une autre forme. Il se disperse entre des heures de recherche, une réunion où chacun reconstruit une histoire différente, une exception ajoutée à un contrat, une correction plus risquée parce qu'un partenaire s'appuie déjà sur la règle, ou une livraison réduite faute de pouvoir expliquer ce que l'on modifie.

Dans ces moments, l'équipe peut croire qu'elle paie seulement la complexité ordinaire d'un système ancien. C'est parfois le cas. Mais elle peut aussi payer le fait qu'une décision est devenue réelle avant que sa raison, son périmètre et sa condition de réexamen ne soient conservés.

Ce diagnostic ne se résout pas en cherchant un coupable. Les personnes qui ont intégré la première règle avaient peut-être peu de temps, peu d'accès à l'historique ou une contrainte réelle de continuité de service. Nommer la dette de décision sert plutôt à changer la question posée au présent : *quelle partie de ce que nous sommes en train de figer mériterait encore une décision reconnaissable demain ?*

Cette formulation garde une vertu pratique. *Elle déplace l'attention de la perfection du passé vers la réversibilité du prochain choix*. Le coût sera peut-être accepté. L'incertitude sera peut-être tolérée. Mais l'équipe pourra dire ce qu'elle accepte et ne traitera pas le simple passage du temps comme une validation.

Donner un statut à ce que nous savons

Nous avons tendance à parler de la connaissance d'une équipe comme d'un bloc : « on sait », « on ne sait pas », « il faut vérifier ». Dans une décision technique, ces phrases sont trop grossières. Elles masquent la différence entre une exigence, une observation, un calcul et une supposition.

Reprenons le cas de la référence client.

Élément	**Statut**	**Pourquoi ce statut compte**	**Ce qu'il permet maintenant**
Chaque référence active doit conserver la bonne identité dans les systèmes concernés.	**Exigence confirmée**	C'est une propriété à protéger, pas une preuve déjà obtenue.	Rejeter toute option qui rendrait cette préservation impossible.
Des formats différents coexistent ; les services ne les traitent pas tous de la même manière.	**Observé**	Cela vient des données et du code examinés.	Justifier l'investigation et la nécessité de clarifier le contrat.
`P-0042` et `42` deviennent tous	**Déduit**	Le calcul est	Voir qu'une collision de

Élément	Statut	Pourquoi ce statut compte	Ce qu'il permet maintenant
deux 42 après la transformation.		reproductible.	représentation est possible.
Préfixe et zéros initiaux n'ont plus de signification métier.	**Supposé**	Aucun élément disponible ne le confirme suffisamment.	Formuler l'hypothèse au lieu de l'intégrer comme règle.
Les contextes où les deux formes distinguent encore des identités ; l'autorité qui peut abandonner cette distinction.	**À confirmer**	La question est identifiée, mais la validation manque.	Demander la recherche ou l'arbitrage qui conditionne la suite.
La raison précise d'un ancien choix dont aucune trace n'a encore été retrouvée.	**Inconnu**	L'information elle-même manque.	Éviter de lui attribuer une explication inventée.

Ces catégories ne sont pas des cases administratives. Une même information peut circuler entre elles. Une observation, répétée et corroborée, peut contribuer à établir un fait. Une déduction correcte peut rester insuffisante pour décider, si elle ne répond pas à la question métier qui compte. Une hypothèse peut devenir une règle choisie, mais il faut alors dire qui l'a choisie, dans quel périmètre et avec quel risque accepté.

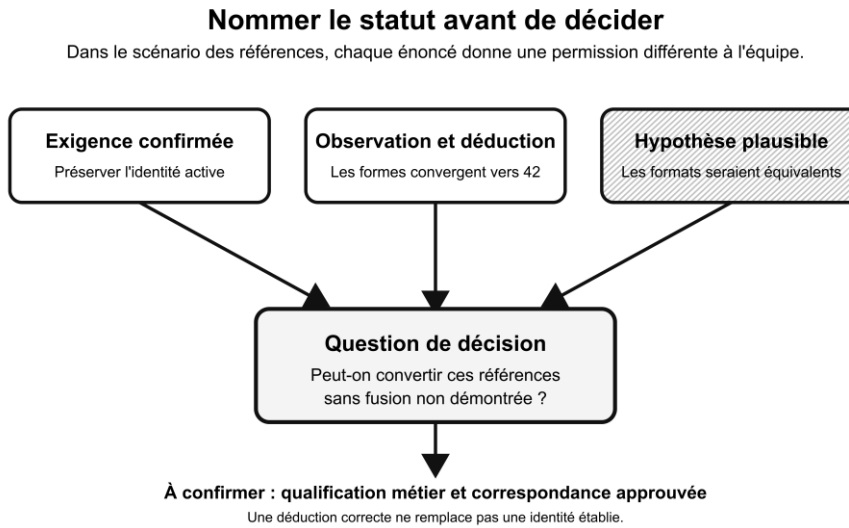


Figure 3.3 - Scénario construit. Distinguer les statuts rend visible la connaissance qui manque avant de convertir une collision de représentation en correspondance d'identité.

Cette grammaire n'a pas pour but de tout ralentir. Elle empêche surtout deux glissements.

Le premier est de traiter une déduction comme une équivalence métier. Le calcul prouve le résultat de la fonction ; il ne prouve pas l'identité des personnes, des comptes ou des contrats derrière les données.

Le second est de présenter une exigence comme si elle décrivait déjà l'état du système. « Nous devons préserver l'identité » est une décision claire. « La normalisation préserve l'identité » est une affirmation qui appelle une démonstration. Les chapitres suivants préciseront la forme de cette démonstration. Pour l'instant, il suffit de ne pas fusionner les deux phrases.

Une question peut rester ouverte sans rester vague

Reconnaître une incertitude ne veut pas dire inscrire « à vérifier » dans un ticket et espérer qu'elle se résolve un jour. Une question ouverte devient utile lorsqu'elle rend l'action suivante plus précise.

Dans notre scénario, « les préfixes sont-ils importants ? » est un début, mais pas encore une question de décision. L'équipe peut la rendre plus concrète : existe-t-il des références actives qui se rejoignent après normalisation ? dans quels systèmes le préfixe est-il encore lu ? quel rôle métier peut attester que la distinction peut être abandonnée ? que fera-t-on des références dont l'historique demeure ambigu ?

Ces questions ne forment pas un protocole à appliquer à chaque changement. Elles tirent leur valeur du risque particulier du scénario : une même valeur partagée avec un partenaire peut rendre plus difficile la correction d'une erreur d'identité. Dans une autre situation, la bonne question et le bon niveau d'enquête seront différents.

L'essentiel est de transformer une inquiétude diffuse en condition de décision. À ce stade, l'équipe n'a toujours pas besoin de savoir quelle fonction écrire. Elle doit seulement éviter de confondre deux phrases : « nous pouvons produire cette normalisation » et « nous pouvons décider de l'utiliser comme clé d'identité ». La première relève de la capacité de construire ; la seconde du sens, des preuves disponibles et du mandat.

Cette distinction nous protège aussi d'un perfectionnisme paralysant. Nous n'avons pas à obtenir une histoire complète de toutes les données avant de travailler. Nous devons savoir quelle inconnue ferait changer la décision présente. Si aucune réponse réaliste ne peut être trouvée à temps, le choix devient visible : limiter le périmètre, conserver la valeur d'origine, différer l'échange, ou demander une décision explicite sur le risque. Le chapitre 9 mettra plus loin en scène une situation de ce type. Ici, nous cherchons seulement à préserver la possibilité d'un arbitrage ultérieur.

Reporter sans enfouir

Que faire lorsque la réponse manque réellement ? Le mauvais réflexe serait de croire qu'il n'existe que deux options : tout intégrer ou tout arrêter.

Une équipe peut reporter une décision. Ce report n'est pas une fuite s'il rend visibles quatre choses : ce qui manque, le risque assumé pendant

l'attente, la condition qui déclenchera la reprise et l'autorité qui pourra choisir lorsque l'information sera disponible.

Dans notre cas, l'équipe pourrait décider de ne pas remplacer les références actives dans l'échange partenaire tant que l'équivalence n'est pas établie. Elle peut conserver la demande d'intégration, rechercher les traces métier, identifier les populations affectées et définir ce qui se passera si la connaissance historique reste introuvable.

Cette décision a un coût. Le partenaire attend peut-être ; une partie de l'intégration reste incomplète ; l'équipe maintient temporairement deux représentations. Ce coût n'est pas un échec de la méthode. Il est le prix visible d'une information qui n'existe pas encore. Le rendre explicite permet de le comparer au coût, beaucoup moins visible, d'une collision propagée.

Report explicite	**Choix enfoui**
Le manque de connaissance est nommé.	L'hypothèse est absorbée par l'implémentation.
Une condition de reprise est connue.	On découvre la question lors d'un changement ou d'un incident.
Le périmètre provisoire et son coût sont assumés.	Le périmètre réel apparaît après propagation.
Une autorité devra trancher à partir d'éléments identifiés.	La règle semble avoir été décidée parce qu'elle existe déjà dans le code.

Reporter ne garantit pas une décision facile ni un résultat satisfaisant. L'information peut rester absente. La contrainte de calendrier peut devenir plus forte. L'équipe pourra devoir choisir une alternative moins riche ou accepter un risque dans un mandat explicite. Mais elle ne se laissera pas raconter que la décision a été prise par la machine, par le test vert ou par le simple fait que le code a été fusionné.

Retrouver une connaissance, ne pas l'inventer

La tentation est grande de demander à un agent IA de combler ce vide. Il peut aider utilement : retrouver les points où une référence est lue, cartographier les transformations, repérer les contrats et préparer des questions pour les personnes qui connaissent l'historique.

Il ne peut pas transformer un commentaire ambigu, des formats hétérogènes et une documentation silencieuse en fait établi. Une réponse rédigée avec assurance peut être un excellent point de départ pour l'enquête ; elle ne remplace ni l'indice manquant ni l'autorité capable de redéfinir une identité métier.

Le rôle de l'équipe n'est donc pas de refuser toute assistance. *Il est de distinguer l'aide à l'exploration de l'autorité de conclure*. Cette frontière vaut aussi pour les humains : le souvenir d'un collègue est précieux, mais il reste une source à situer et, si l'enjeu le demande, à corroborer.

Ce que nous choisissons avant de le savoir

Une hypothèse n'est pas un défaut. Dans un système vivant, toute équipe avance avec des informations partielles. Le danger apparaît lorsqu'elle ne voit plus qu'une hypothèse conditionne une décision, ou lorsqu'elle laisse le coût d'une remise en question grossir sans mandat pour l'assumer.

Une décision laissée implicite revient alors sous une forme moins favorable : une exception ajoutée dans l'urgence, une discussion où chacun reconstruit une histoire différente ou une règle découverte après qu'un partenaire s'y est déjà adapté. Dans le cas des références, fusionner les valeurs et traiter les collisions plus tard ne résout pas l'incertitude sur l'identité. Cela décide seulement que son coût sera porté par une future migration, par le support ou par la personne qui découvrira qu'une distinction a été effacée.

Rendre le choix explicite conserve trois possibilités : savoir ce qui a été décidé, rouvrir cette décision quand une preuve nouvelle apparaît et identifier qui peut accepter le risque provisoire. L'équipe peut ainsi

préserver la collision, consigner la question, définir la condition de reprise et choisir un périmètre qui ne fixe pas encore l'équivalence.

> **Dans une pratique SDD** Une hypothèse, une question ouverte ou une décision à confirmer peut rester dans `spec.md`. Un passage `clarify` la rend visible avant que `plan.md` et `tasks.md` n'en dérivent du travail. Elle conserve ainsi son statut et sa condition de reprise.

La dette de décision commence donc moins par une erreur de code que par une différence non déclarée entre ce que l'équipe sait, ce qu'elle suppose et ce qu'elle a choisi d'engager malgré cette différence.

Le prochain chapitre partira de là. Avant de choisir une solution ou même de préciser ses limites, nous devons savoir quel résultat nous cherchons réellement à rendre possible.

Source et portée

Sources et portée

sei. Software Engineering Institute, Carnegie Mellon University, *Managing Technical Debt in Complex Software Systems*, 2016, vérifié le 19 septembre 2026. Cette source borne la définition de la dette technique ; elle ne définit pas la dette de décision et ne valide pas empiriquement le cadre proposé dans ce chapitre. Le registre complet, y compris les limites de citation, est disponible dans `06_Matière`.

Chapitre 4 - Commencer par l'intention

Une équipe d'exploitation demande un tableau de bord des importations en échec. La demande paraît suffisamment précise pour être prise en charge immédiatement. Un agent IA peut proposer les cartes de statut, les graphiques, l'endpoint qui agrège les résultats et même le bouton pour relancer l'importation. Quelques échanges suffisent à obtenir une première interface crédible.

Rien de tout cela n'est absurde. Le total des échecs existe, les lots posent un problème réel et l'équipe a besoin d'avancer. Pourtant, une question demeure : *qu'allons-nous rendre possible grâce à ce tableau de bord ?*

Cette question n'est pas un raffinement de vocabulaire. Elle sépare une solution disponible d'une direction choisie. Quand produire une solution devient rapide, cette séparation compte davantage : la première réponse convaincante peut prendre la place de la question que l'équipe devait encore poser.

Une demande précise qui ne dit pas encore où aller

« Ajouter un tableau de bord des importations en échec » est une demande concrète. Elle suggère déjà des écrans, des données à afficher et une liste de travaux. Elle peut organiser un sprint. Mais elle ne dit pas pourquoi l'opérateur consulte cet écran, quelle décision il doit prendre, ni ce qui doit changer après son intervention.

Nous confondons facilement cette précision avec une direction. Parce qu'un objet technique est nommé, nous pouvons découper le travail, estimer une charge et demander une implémentation. La production démarre alors avec une impression de clarté. Cette clarté porte sur la forme de la réponse, non nécessairement sur le résultat recherché. Le phénomène n'a pas attendu l'arrivée des harnais, on connait certains informaticiens qui se jettent sur leur clavier alors qu'on a à peine terminé d'expliquer le besoin. Cette précipitation n'est pas à confondre avec de l'efficacité ou de la diligence. Ici l'IA est cet informaticien trop zélé, sauf qu'elle travaille cent fois plus vite que lui et produit à cette

même vitesse des réponses solides qui pourront passer pour des décisions claires alors qu'il n'en est rien.

Le risque n'est pas de construire un mauvais tableau de bord au sens esthétique ou technique. L'équipe peut livrer un écran rapide, lisible et correctement intégré. Le risque est de construire exactement ce qui a été demandé et de découvrir qu'il ne permet pas à la personne concernée d'agir mieux.

Dans un travail moins assisté, le temps requis pour explorer une solution laissait parfois émerger une objection : « De quelles informations aurons-nous besoin ? » ou « Quelle décision cet écran doit-il faciliter ? » Cette friction n'était pas une méthode. Elle pouvait tout aussi bien retarder inutilement une amélioration simple. Le processus créait cette lenteur et le processus pouvait souvent en tirer parti grâce au temps de respiration qu'il laissait. Mais lorsque l'IA produit plusieurs variantes qui arrivent dans la même journée, l'absence de cette plage de réflexion, de rythme qui autorise l'interrogation, la question devient moins visible. Une proposition complète donne l'impression que le besoin a été compris parce qu'elle donne déjà prise à la discussion.

Le chapitre précédent a montré comment une hypothèse peut devenir coûteuse à rouvrir une fois transformée en code. Ici, le mécanisme commence plus tôt. La demande de solution contient déjà un choix sur ce que l'équipe juge utile de produire. Si ce choix n'est pas reconnu, l'agent IA et l'équipe peuvent l'optimiser sans jamais l'avoir véritablement discuté.

Le lot qui échoue à moitié

Le scénario de ce chapitre est construit. Une plateforme reçoit chaque nuit des fichiers de commandes. Un lot peut s'interrompre après avoir créé certaines commandes et avant d'avoir traité les suivantes. Le matin, un opérateur voit seulement que l'importation a échoué.

La demande arrive sous une forme familière : il faut un tableau de bord des échecs, avec le nom du fichier, l'heure, le nombre de lignes et un

bouton « relancer ». L'équipe peut en produire une version presque aussitôt. Elle dispose déjà des journaux et connaît les tables où sont conservés les statuts. L'agent IA aide à parcourir les traces, à proposer des requêtes et à dessiner l'interface.

Le premier écran répond à la demande. Il ne répond pas encore à la situation.

Lors d'un échange, l'opérateur formule ce qui lui manque : « Le total des échecs est déjà visible. Ce que je dois savoir, c'est si je peux relancer ce lot sans appliquer deux fois les lignes déjà passées. » Parfois, il devra relancer. Parfois, il devra traiter les lignes restantes manuellement. Parfois, il devra mettre le lot en attente et demander une correction à l'organisation qui a fourni le fichier.

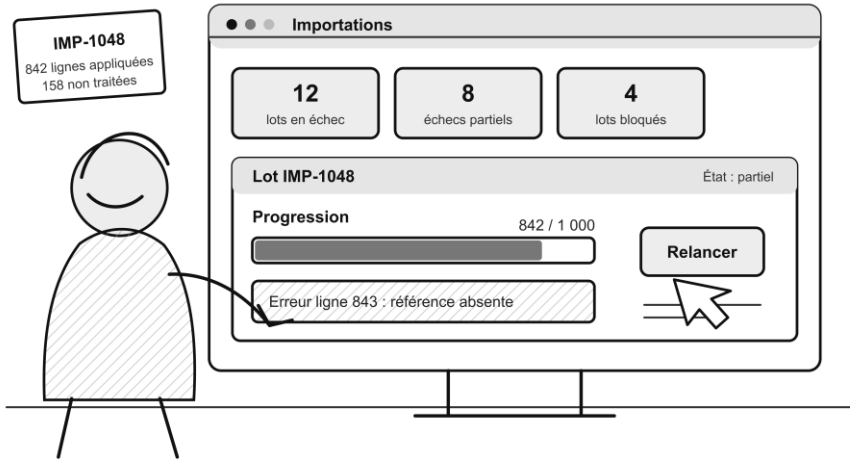


Figure 4.1 - Scénario construit. Le tableau de bord montre un lot partiellement traité sans établir si sa relance rejouera les lignes déjà appliquées.

Le tableau de bord n'est donc ni rejeté ni sacralisé. Il devient une réponse possible parmi d'autres. Une classification des situations, une vue sur l'avancement réellement atteint, une alerte accompagnée d'une action guidée ou une procédure de mise en attente pourraient mieux servir le même besoin. Une combinaison de ces éléments pourrait être nécessaire. À ce stade, l'équipe n'a pas décidé laquelle.

Elle peut toutefois formuler une intention : permettre à un opérateur d'identifier assez vite l'action appropriée face à une importation en échec, afin de rétablir le flux sans aggraver le problème.

Cette phrase ne décrit ni écran, ni API, ni schéma de données. Elle rend visible la personne concernée, le jugement qu'elle doit exercer et le changement recherché. Elle donne aussi une manière d'évaluer une proposition. Une interface qui embellit le décompte des erreurs mais laisse l'opérateur incapable de choisir son action est insuffisante, même si elle correspond au ticket. Une solution plus modeste qui éclaire cette action peut au contraire être pertinente.

L'intention n'affirme pas que l'opérateur peut relancer sans risque. Elle ne dit pas encore quelles données doivent être conservées, quels droits sont nécessaires ou comment démontrer qu'une action est sûre. Ce sont des questions indispensables, mais elles n'ont pas la même fonction. Les traiter ici comme si elles étaient déjà résolues donnerait une fausse netteté au scénario.

Cette limite est exigeante en pratique. Une équipe qui a déjà les journaux et les statuts peut être tentée de commencer par ce qui est immédiatement observable : le nombre d'erreurs, leur répartition, la durée d'un lot. Ces données peuvent devenir importantes. Elles ne déterminent pas seules la décision qui mérite d'être soutenue. Un compteur indique qu'un lot s'est interrompu ; il ne dit pas si l'opérateur cherche d'abord à protéger les commandes déjà passées, à raccourcir le délai de reprise ou à éviter une escalade inutile.

L'inverse est tout aussi vrai. Une intention ne donne pas le droit d'ignorer ce que le système permet réellement de voir. Si l'état d'avancement est introuvable, si les erreurs ne sont pas classifiables ou si personne ne sait identifier le propriétaire du fichier, l'équipe ne peut pas promettre une capacité qu'elle n'est pas en mesure de fournir. Elle pourra proposer une première étape plus modeste : rendre visible l'incertitude, conserver davantage de traces ou organiser le passage à une personne compétente. L'intention aide alors à reconnaître que cette étape est un moyen provisoire, et non la fin de la question.

Une direction avant une solution

Dans ce livre, l'intention désigne le résultat recherché, le problème à résoudre et la valeur attendue. Elle relie une situation présente à un changement que quelqu'un doit pouvoir constater ou utiliser. Elle ne remplace pas le travail de conception ; elle lui donne une direction.

Cette distinction n'est pas une découverte propre à l'IA. Le *NASA Systems Engineering Handbook* rappelle qu'une exigence devrait exprimer ce qui est nécessaire, plutôt que la manière de le fournir, et propose de demander « pourquoi » pour remonter vers le besoin réel.[^nasa] Le Service Manual britannique sépare de son côté les besoins utilisateurs - ce qu'une personne cherche à accomplir - des fonctionnalités plus spécifiques qui peuvent y répondre.[^govuk]

Ces textes ne constituent pas une validation empirique de la discipline proposée dans ce livre. Ils viennent de contextes différents : ingénierie système pour l'un, conception de services publics pour l'autre. Ils ne disent rien de la qualité d'un agent IA, ni du scénario des imports. Ils soutiennent seulement une distinction utile : un besoin ou un résultat peut rester stable tandis que plusieurs manières de le satisfaire restent ouvertes.

L'intention sert précisément à conserver cette ouverture assez longtemps pour que l'équipe puisse juger. Elle permet de comparer des réponses plausibles à autre chose qu'à leur élégance, leur nouveauté ou leur vitesse de production.

Dans le scénario, le résultat recherché n'est pas « disposer d'un tableau de bord ». Il est de restaurer un flux de commandes sans laisser l'opérateur agir à l'aveugle. Cette direction permet de poser des questions plus utiles à une proposition : rend-elle perceptible la situation du lot ? Aide-t-elle à choisir entre relancer, traiter, attendre ou signaler ? Réduit-elle une incertitude qui bloque réellement l'action ?

Elle ne promet pas qu'une équipe trouvera immédiatement la bonne réponse. Une intention peut être mal comprise, contestée ou limitée par ce que l'organisation est réellement capable de faire. Son rôle est

plus modeste et plus concret : *rendre cette discussion possible avant qu'une forme technique ne soit devenue difficile à remettre en question*.

Une direction explicite réduit aussi une erreur de communication fréquente. Dans une conversation de livraison, deux personnes peuvent approuver le même tableau de bord tout en espérant des effets différents. L'une cherche à diminuer le temps de diagnostic ; l'autre veut rendre la relance autonome ; une troisième souhaite surtout produire un historique pour les responsables de compte. Toutes peuvent trouver la demande raisonnable. Elles ne choisiront pourtant pas les mêmes informations, ni les mêmes priorités, ni les mêmes compromis. Formuler l'intention n'élimine pas ce désaccord. Elle le rend visible à un moment où la réponse n'a pas encore absorbé une interprétation.

Le mot « intention » ne signifie pas non plus « intention individuelle ». Elle peut être portée par une équipe, un produit ou une organisation. Ce qui importe est que quelqu'un puisse en discuter la formulation et en assumer la direction. Une phrase inscrite dans une documentation, sans personne pour dire pourquoi elle compte encore, offre peu de protection contre la dérive qu'elle était censée prévenir.

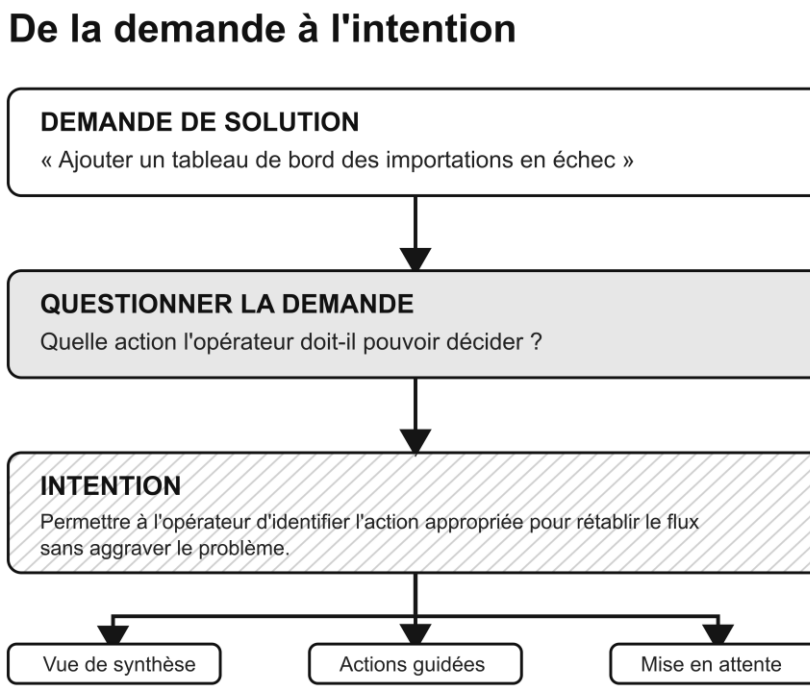


Figure 4.2 - Scénario construit. La demande de solution est interrogée pour faire apparaître l'intention.

Ce que l'intention n'est pas

Un intitulé de ticket peut être utile. Il est court, identifiable et suffisamment concret pour organiser un travail. « Ajouter une recherche », « moderniser le module » ou « intégrer un agent IA » peuvent signaler un chantier réel. Ils ne disent pas encore quel changement la réalisation doit produire pour une personne, une activité ou une organisation.

Dans notre cas, « tableau de bord des imports » reste un bon titre de discussion. Le problème apparaît quand il devient la seule formulation disponible. L'équipe peut alors répondre à la demande sans savoir si elle aide l'opérateur à décider ou si elle ajoute simplement une surface de consultation.

L'intention n'est pas non plus une solution déguisée. « Permettre à l'opérateur de relancer les lots en un clic » paraît orienté vers l'action, mais enferme déjà le résultat dans une option. Elle suppose que relancer est le bon geste, que l'opérateur doit l'exécuter directement et que la vitesse du clic est le principal enjeu. Ces hypothèses sont peut-être justes. Elles doivent pouvoir être discutées comme telles.

Enfin, une intention n'est pas une promesse vague. « Simplifier les imports », « améliorer l'expérience opérateur » ou « rendre le système plus robuste » peuvent exprimer une ambition honorable. Elles ne permettent pas encore d'écarter une solution ou d'en préférer une autre. Une direction devient utile lorsqu'elle relie une personne, une situation, une décision ou une action, et le changement attendu.

Formulation	**Ce qu'elle apporte**	**Ce qui manque encore**
Ajouter un tableau de bord des imports en échec.	Une solution envisageable.	La décision et le résultat recherchés.
Relancer les lots en un clic.	Une option d'action.	La preuve que relancer est l'action appropriée.
Améliorer le suivi des imports.	Une ambition générale.	Une personne, un jugement et un effet observable.
Permettre à l'opérateur d'identifier l'action appropriée pour rétablir le flux sans aggraver le problème.	Une direction de décision.	Les limites, les critères et les preuves nécessaires.

La dernière formulation n'est pas supérieure parce qu'elle est plus longue. Elle est plus utile parce qu'elle permet de refuser certaines réponses sans imposer encore une seule manière de réaliser les autres.

Cette distinction explique aussi pourquoi une intention n'est pas une user story rédigée avec plus de soin. Une user story peut relier une fonctionnalité à une personne et à une valeur ; elle peut être une excellente manière de préparer un travail. Mais elle reste un artefact parmi d'autres, avec un niveau de détail et un cycle de vie qui

dépendent de l'équipe. Le livre ne propose pas de format imposé. Il demande seulement qu'avant d'accepter une solution, l'équipe sache dire quel changement de situation lui donne sa pertinence.

De même, une intention n'est pas une formulation marketing. Dire « donner confiance aux opérateurs » peut éclairer une ambition, mais ne permet pas encore de comprendre ce qu'ils doivent pouvoir faire lorsque le lot s'arrête. La valeur attendue devient exploitable lorsqu'elle rencontre une situation et un choix. Dans ce chapitre, cette précision suffit ; les chapitres suivants exigeront que l'équipe rende certaines propriétés observables et démontrables.

Quatre questions pour reconnaître une intention nette

Il serait tentant de transformer cette idée en formulaire obligatoire. Ce serait déplacer le problème : une équipe remplirait des rubriques sans mieux comprendre la décision à prendre. Les quatre questions suivantes sont plutôt des repères de lecture. Elles aident à reconnaître qu'une demande est encore trop vague ou trop proche de sa solution.

Pour qui cherchons-nous à améliorer la situation ? Ici, la réponse est l'opérateur responsable de la reprise. Nommer ce rôle ne signifie pas que ses préférences décident de tout. Cela rappelle que le travail doit aider une personne exerçant un rôle précis, non une abstraction appelée « utilisateur ».

Quelle décision, action ou compréhension doit-elle obtenir ? L'opérateur n'a pas seulement besoin de voir un échec. Il doit savoir s'il convient de relancer, de traiter autrement, d'attendre ou d'escalader. Cette précision déplace la discussion de l'information affichée vers l'action qu'elle doit rendre possible.

Quel problème ou quelle friction cherchons-nous à faire disparaître ? Le problème n'est pas nécessairement l'absence d'un écran. C'est l'incertitude qui laisse un lot interrompu sans voie de rétablissement compréhensible. Une solution peut réduire cette incertitude sans produire un tableau de bord complet.

Quel changement nous montrera que nous avons progressé ? Cette dernière question ne demande pas encore une métrique ni un critère de recette. Elle invite à rendre la valeur visible : l'opérateur sait plus souvent quelle action engager et le flux peut être rétabli sans que la situation soit empirée. Le chapitre 6 transformera plus loin certaines attentes en propriétés vérifiables. Ici, nous identifions seulement le changement qui mérite cet effort.

Ces questions peuvent révéler qu'une intention reste inconnue. Toutes les équipes n'ont pas un accès direct aux personnes qui utilisent le système ; l'historique peut être dispersé et la demande peut venir d'une contrainte de calendrier réelle. Dans ce cas, l'honnêteté consiste à nommer ce qui est supposé plutôt qu'à fabriquer une histoire utilisateur rassurante. Une hypothèse sur le besoin peut être assez solide pour commencer une exploration limitée. Elle ne devient pas un fait parce qu'elle a été inscrite dans un prompt ou dans un ticket.

Il est utile de distinguer ce que l'équipe sait de ce qu'elle espère. Dans le scénario, elle observe que des lots échouent et que l'opérateur doit intervenir. Elle peut déduire que l'état intermédiaire mérite d'être rendu plus lisible. Elle suppose peut-être que l'action la plus fréquente sera la relance. Cette dernière proposition doit rester une hypothèse tant que les situations réelles n'ont pas été examinées. Le chapitre 3 a donné des mots pour ce type de différence ; l'intention les mobilise sans les faire disparaître.

Cette prudence ne conduit pas à suspendre toute production jusqu'à ce qu'une enquête parfaite soit terminée. L'équipe peut par exemple explorer les journaux, interviewer un opérateur disponible, ou construire une vue temporaire qui n'exécute encore aucune action. Elle doit simplement garder visible ce que cette exploration cherche à apprendre. Sinon, un prototype présenté comme une manière de comprendre devient facilement une solution de fait, avec les attentes et les dépendances qui l'accompagnent.

Lorsque la demande vient d'une urgence

Toutes les demandes ne naissent pas d'un travail posé de compréhension. Un incident peut interrompre le service ; un partenaire peut attendre une réponse ; une règle externe peut imposer un changement. Dans ces moments, demander de prendre le temps de « mieux comprendre le besoin » peut sonner comme un luxe détaché de la réalité.

L'intention garde pourtant une utilité, peut-être même plus nette. Elle aide à distinguer ce qui doit être restauré maintenant de ce qui pourra être amélioré ensuite. Une équipe ne dira pas : « Nous allons repenser l'expérience complète des imports. » Elle pourra dire : « Nous devons permettre aux opérateurs de traiter les lots bloqués aujourd'hui sans faire passer deux fois une commande. » La formulation est provisoire, mais elle clarifie la priorité et empêche une réponse urgente de se présenter comme une transformation achevée.

Cette distinction protège également contre deux réactions opposées. La première consiste à élargir sans limite le périmètre d'une correction parce que toutes les imperfections rencontrées semblent liées. La seconde consiste à livrer un contournement qui soulage immédiatement la pression, puis à oublier ce qu'il ne résout pas. Une intention explicite permet de conserver le bénéfice recherché et la limite assumée. Elle donne une phrase à laquelle revenir lorsque le contexte redevient moins pressant.

Dans le scénario, une vue minimale pourrait permettre à l'opérateur d'identifier les lots qui nécessitent une intervention humaine. Cette amélioration ne résout pas à elle seule le problème des relances. Elle peut néanmoins servir l'intention du moment si l'action sûre consiste justement à ne pas relancer. L'équipe peut accepter le coût d'un traitement manuel temporaire, à condition de ne pas faire passer cette limite pour une capacité définitive du système.

L'intention ne transforme donc pas toute urgence en atelier de cadrage. Elle rend le choix de vitesse lisible : quelle situation voulons-nous améliorer maintenant, pour qui, et qu'acceptons-nous de laisser ouvert ? Les contraintes diront ensuite ce qui ne peut pas être

compromis, même sous pression. Les preuves diront plus loin ce qui permet d'élargir une action. Mais sans direction formulée, l'urgence se contente souvent de sélectionner la solution qui était la plus facile à produire.

Confier la production à une IA sans céder l'arbitrage

Une IA peut aider à clarifier une intention. Elle peut résumer des journaux, rapprocher des tickets, repérer les étapes d'un parcours, formuler des questions ou proposer plusieurs interprétations de la demande. Ces apports sont précieux, surtout lorsque les informations sont dispersées.

Elle peut aussi produire une réponse cohérente à une direction insuffisamment discutée. Elle n'a pas besoin de se tromper pour cela : si « construire un tableau de bord » est le seul objectif fourni, elle peut très bien optimiser cet objectif. Le danger n'est donc pas une machine qui choisirait mystérieusement à la place de l'équipe. Il est plus ordinaire : nous acceptons comme direction ce qui n'était qu'une première forme de réponse.

Choisir la valeur qui mérite d'être protégée, la personne dont le jugement doit être soutenu ou le délai auquel l'organisation consent reste un arbitrage de produit et d'organisation. La production peut être largement confiée à un agent IA. Cette direction ne l'est pas par défaut.

Ce point ne donne pas à une seule personne le monopole du besoin. Dans une situation réelle, l'opérateur, le responsable de produit, l'équipe d'exploitation et les personnes qui portent les conséquences d'une interruption peuvent percevoir des enjeux différents. L'intention sert justement de support à leur discussion. Elle ne remplace ni le mandat de décider ni l'enquête nécessaire pour comprendre.

La bonne question à adresser à un agent IA n'est donc pas seulement « construis-moi ce tableau de bord ». Elle peut être : « Quelles interprétations de cette demande vois-tu ? Quelles décisions chacune rend-elle possible ? Quelles informations manquent pour choisir ? » L'agent peut contribuer à explorer l'espace, à comparer des options et à

rendre les suppositions visibles. Il ne reçoit pas par cette formulation le pouvoir de décider quelle valeur l'organisation doit privilégier.

Cette manière de l'utiliser a un autre avantage : elle rend sa production plus facile à évaluer. Si trois propositions correspondent à trois intentions légèrement différentes, le débat ne porte plus seulement sur la qualité de leur code ou de leur interface. L'équipe peut dire quelle direction chacune sert, laquelle n'est pas encore assumée et quelle question doit être posée avant d'engager davantage de travail. La vitesse de génération devient alors une capacité d'exploration, non une raison d'accepter la première interprétation.

On notera ici aussi que les IA récentes ne sont pas uniquement des machines à produire du code, elles peuvent jouer le rôle *sparring partner*, elles peuvent devenir des interviewers, des forces de proposition, jouer l'avocat du diable pour trouver les failles d'une idée tout autant que produire des schémas, des synthèses, et mille autres choses. Le livre présente principalement leur capacité à fabriquer du code à grande vitesse et ses implications, mais il ne faut surtout pas penser qu'il limite l'utilisation des IA à cette seule et mécanique fonction, elles sont capables de bien plus et se passer de leur aide dans tous les domaines indiqués fait passer à côté de leur plein potentiel.

Une direction qui reste révisable

Présenter l'intention en premier ne décrit pas une séquence rigide. Une contrainte découverte plus tard peut conduire l'équipe à la préciser. Une preuve peut montrer que l'action imaginée ne produit pas le résultat attendu. Une information métier nouvelle peut imposer de revoir le problème lui-même.

Dans le scénario, découvrir qu'une relance duplique certaines commandes pourrait obliger à modifier l'ambition immédiate : permettre non plus de rétablir chaque lot automatiquement, mais de décider lesquelles des actions disponibles sont sûres dans le périmètre actuel. Ce n'est pas un échec de l'intention initiale. C'est la raison pour

laquelle elle doit rester explicite : nous pouvons voir ce qui change, pourquoi, et ce que nous cessons provisoirement de promettre.

Il existe une différence importante entre réviser une intention et l'abandonner silencieusement. Une équipe peut découvrir que le bénéfice attendu ne justifie pas le coût, que le besoin concerne finalement un autre rôle ou qu'une solution non logicielle répond mieux à la situation. Parce que la direction avait été formulée, elle peut voir ce qui change au lieu de mesurer la réussite au seul fait d'avoir livré du code.

Une intention nette ne donne donc pas immédiatement la bonne solution. Elle permet de confronter un tableau de bord, une action guidée ou une mise en attente au résultat recherché, puis d'en réviser la formulation lorsqu'une contrainte ou une preuve nouvelle le justifie.

> **Dans une pratique SDD** `constitution.md` peut porter des principes durables, tandis que `spec.md` donne une forme à l'intention d'une fonctionnalité : problème, objectif et résultat attendu. Ces repères permettent de comparer plusieurs réponses avant de détailler leur implémentation.

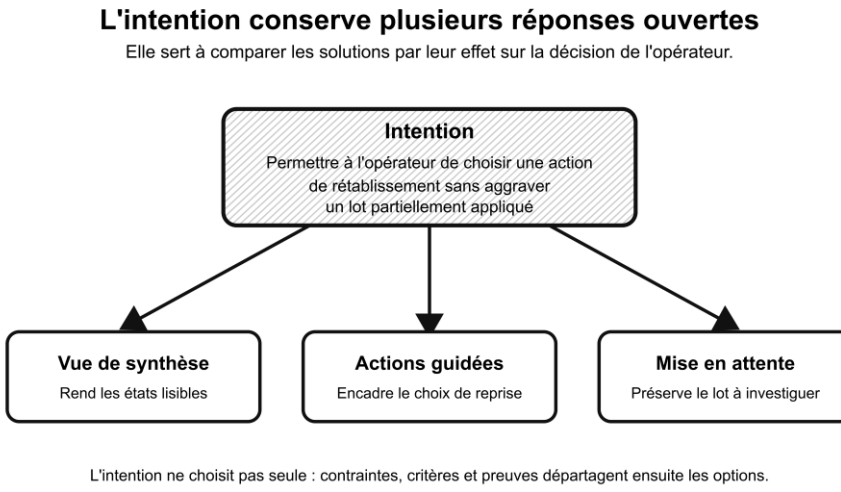


Figure 4.3 - Scénario construit. L'intention aide à comparer une vue, une action guidée ou une mise en attente par leur capacité à soutenir la décision de rétablissement, sans choisir à la place des contraintes et des preuves.

Pour confier du travail à un agent IA sans perdre le contrôle, il faut donc commencer non par ce que nous voulons faire produire, mais par le changement que nous cherchons à obtenir. Cette direction ne délimite pas encore l'acceptable. Le chapitre suivant s'occupe des contraintes qu'une solution utile n'a pas le droit de franchir.

Sources et portée

Sources et portée

nasa. NASA, *Systems Engineering Handbook - Appendix C: How to Write a Good Requirement*, consulté le 19 septembre 2026. La source soutient la distinction entre besoin et moyen d'implémentation en ingénierie système ; elle ne valide pas le scénario ni le cadre de délégation du livre.

govuk. Government Digital Service, *Learning about users and their needs*, consulté le 19 septembre 2026. La source sépare les besoins d'une fonctionnalité et recommande de les étayer par la recherche ; elle ne prescrit pas la grammaire conceptuelle de ce livre ni une méthode de product discovery.

Chapitre 5 - Rendre les contraintes visibles

Une intention donne une direction. Elle ne dit pas encore ce qu'une solution n'a pas le droit de compromettre.

Dans le chapitre précédent, l'équipe cherchait à aider une personne à choisir l'action de rétablissement pertinente après l'échec partiel d'un import. Cette direction empêchait déjà de confondre un tableau de bord avec le résultat recherché. Elle ne suffisait pourtant pas à accepter n'importe quel tableau, n'importe quel bouton de relance de l'importation ni n'importe quelle automatisation.

Une solution peut servir l'intention et rester inacceptable. Elle peut être utile à première vue, fonctionner dans une démonstration et franchir malgré tout une limite que l'équipe n'avait pas formulée. C'est le rôle des contraintes : rendre visibles les conditions dans lesquelles une réponse peut réellement être considérée comme bonne.

Quelles limites doivent être connues avant que plusieurs solutions plausibles puissent être départagées ?

Une intention utile, une solution trop large

Le scénario de ce chapitre est une construction. Une équipe doit aider les personnes d'astreinte pendant les incidents de production. Après plusieurs échanges, elle a clarifié son intention : permettre à l'astreinte d'identifier plus vite l'action de rétablissement pertinente, sans naviguer entre plusieurs outils au moment où l'attention est la plus rare.

Un agent IA aide à produire une console convaincante. Elle rassemble dans une même vue les journaux, les traces, les configurations récentes et les informations de déploiement. Une recherche retrouve rapidement une erreur. Les tests d'affichage passent. La démonstration est fluide : au lieu d'ouvrir cinq écrans, la personne d'astreinte voit une histoire d'incident dans une seule interface.

La solution n'est pas absurde. Elle répond même à une part importante de l'intention. Son défaut ne se trouve pas dans un bouton manquant ni dans une erreur visible pendant la démonstration. Il apparaît dans ce qu'elle rend possible par défaut.

Les journaux bruts sont copiés sans distinction dans un index de recherche, puis affichés dans le navigateur. Or ils peuvent contenir des identifiants techniques, des informations fournies par un client ou des secrets mal masqués. Personne ne dit encore que c'est le cas pour chaque source. C'est précisément le point : certains journaux n'avaient jamais été conçus pour une consultation large et recherchable.

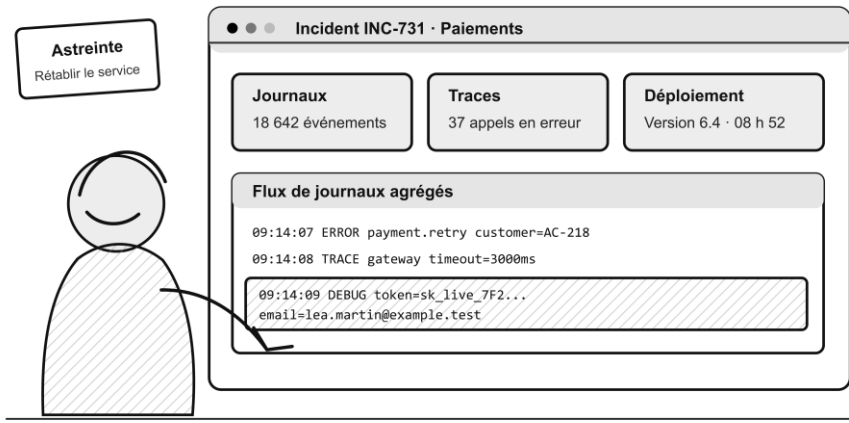


Figure 5.1 - Scénario construit. La console réunit les informations utiles à l'astreinte, mais expose aussi des données qui n'étaient pas destinées à cette consultation.

La même console porte une autre limite. Pendant un incident, le volume de journaux augmente souvent. L'index qui rend la démonstration si riche peut devenir lent ou indisponible au moment précis où l'astreinte en dépend. La confidentialité et la disponibilité ne sont pas deux formulations du même problème. L'une concerne ce qui peut être exposé et à qui. L'autre concerne la capacité de l'outil à rester utile sous charge.

La première proposition doit donc être refusée tardivement, alors que son utilité fonctionnelle est reconnue. Ce refus ne signifie pas que l'équipe rejette l'intention ou que l'agent IA a échoué à produire du code. Il révèle qu'une direction utile ne dessine pas encore l'espace des solutions acceptables.

Rendre les limites discutables

Une limite implicite ne disparaît pas parce qu'elle n'est écrite nulle part. Elle est simplement plus susceptible d'être tranchée par la première implémentation qui semble fonctionner.

Dans le scénario, l'équipe met à plat les questions que la console avait contournées : quelles catégories de journaux peuvent entrer dans l'index ? Lesquelles doivent être exclues ou masquées avant toute indexation ? Qui peut consulter les détails pendant l'incident et après lui ? Quelle trace doit conserver une consultation sensible ? Combien de temps les données peuvent-elles rester disponibles ? Et quel sous-ensemble d'informations doit rester accessible lorsque la charge augmente ?

Cette liste n'est pas un référentiel de sécurité. Elle n'est utile que parce qu'elle répond à une situation précise. Dans une autre console, d'autres limites pourraient compter davantage. Le geste important consiste à passer de « la console rassemble ce qui aide » à « la console ne peut rassembler que ce qui reste acceptable dans cet usage ».

Certaines limites sont déjà établies. L'équipe sait, par exemple, qu'un secret ne doit pas être rendu consultable par défaut. D'autres sont observées ou supposées. Un mainteneur signale qu'une source de journaux reçoit parfois des données fournies par les clients, mais le contenu exact reste à confirmer. Cette information doit guider la prudence sans acquérir silencieusement le statut d'un fait complet.

Cette distinction modifie la décision immédiate. L'équipe ne conclut pas que toute source de journaux est interdite. Elle distingue celles dont le contenu et la finalité sont connus de celles qui demandent une enquête. Les premières peuvent entrer dans un périmètre contrôlé. Les secondes restent hors de l'index jusqu'à confirmation. Cette attente n'est pas l'aveu que le projet est bloqué. C'est une manière de ne pas transformer une absence de connaissance en accès durable.

Le même raisonnement vaut pour la conservation. Dire « nous garderons les informations pendant un temps raisonnable » ne délimite rien. Dire combien de temps une catégorie donnée reste disponible,

dans quel but et sous quel accès ne transforme pas l'équipe en service de conformité. Cela rend une décision discutable avant qu'elle ne soit enfouie dans un paramètre par défaut. La contrainte devient visible parce que sa portée, son statut et son effet sur les options peuvent être examinés.

Une contrainte exprime ce qui doit être respecté. Le statut de connaissance exprime ce que l'équipe sait de cette contrainte, de sa portée et des données qu'elle concerne. Confondre les deux créerait une bureaucratie aussi trompeuse que l'absence de règle : une hypothèse présentée comme une exigence ferme peut bloquer inutilement une décision ; une exigence rapportée comme une simple préférence peut être écartée par commodité.

Une contrainte est une condition d'acceptabilité

L'intention répond à une question de direction : quel résultat cherchons-nous à rendre possible ? Une contrainte répond à une question différente : quelles limites toute solution acceptable doit-elle respecter ?

« Permettre à l'astreinte de choisir une action de rétablissement » est une intention. « Des informations non destinées à une large consultation ne doivent pas devenir visibles par défaut » est une contrainte. La première oriente l'évaluation des solutions. La seconde en écarte certaines, même lorsqu'elles paraissent plus riches ou plus rapides à produire.

Une contrainte ne prescrit pas nécessairement une solution. Elle ne dit pas ici quel moteur de recherche employer, quelle base utiliser ni quelle interface dessiner. Elle établit une limite à respecter. Plusieurs conceptions peuvent y répondre : exclure certaines sources, masquer des champs avant l'indexation, restreindre les accès détaillés, différer une source dont le contenu n'est pas confirmé ou proposer un mode dégradé.

Les contraintes portent souvent la mémoire de compromis que le code ne peut pas deviner seul. Facilité d'accès contre confidentialité, richesse d'information contre disponibilité, vitesse de livraison contre

coût d'exploitation, adaptation locale contre cohérence d'un contrat. Les rendre visibles ne supprime pas ces tensions. Elles empêchent seulement qu'elles soient décidées sans débat par l'option la plus immédiatement disponible.

Exigence, arbitrage autorisé, préférence

Toutes les limites ne possèdent pas le même statut. Une exigence impérative est une condition à laquelle l'équipe ne peut pas déroger dans son mandat. Si l'interdiction d'indexer des secrets est confirmée, la faible fréquence d'usage de la console ne la transforme pas en préférence.

Un arbitrage autorisé laisse une décision à une personne ou une équipe habilitée. L'astreinte peut peut-être accepter que certaines informations moins détaillées ne soient disponibles qu'après un délai, si ce choix préserve la stabilité de la console et si son mandat couvre cet effet. Une préférence de conception peut, elle, être abandonnée lorsqu'une autre réponse respecte mieux l'intention et les exigences.

Statut	Exemple dans la console	Ce qu'il autorise
Exigence impérative	Ne pas rendre consultables des secrets.	Chercher une solution conforme, pas une dérogation pratique.
Arbitrage autorisé	Réduire la richesse affichée sous charge.	Comparer des compromis dans le mandat défini.
Préférence	Conserver une vue unique pour toutes les sources.	Abandonner l'option si elle gêne une réponse plus acceptable.

Accepter un risque résiduel ne signifie pas déclarer satisfaite une exigence non démontrée. Cela signifie accepter une incertitude permise dans un périmètre explicite. Cette distinction protège l'équipe contre une formule commode : « Nous savons qu'il existe un risque, mais nous avançons quand même. » Avancer peut être légitime. Encore faut-il savoir quelle limite reste respectée, quelle incertitude est admise et qui a le droit de porter cette décision.

Le vocabulaire aide aussi à éviter une négociation involontaire. Une préférence comme « tout voir depuis une seule page » peut être abandonnée pour préserver la disponibilité. Un arbitrage autorisé peut conduire à différer une source dans un premier périmètre. Une exigence impérative ne peut pas être compensée par la richesse de la démonstration. Si ces trois statuts ne sont pas distingués, la solution la plus persuasive tend à les traiter de la même manière : comme des inconvénients à contourner.

Il est utile d'examiner ce qui se passe lorsque ces statuts restent mélangés. Un développeur peut traiter une information sensible comme un champ à masquer parce qu'il pense que l'enjeu est seulement visuel. Une personne d'exploitation peut considérer que la console doit rester disponible parce qu'elle y voit une attente non négociable. Une autre peut accepter un périmètre réduit parce qu'elle sait que l'astreinte conserve un outil de secours. Toutes trois ont des raisons plausibles. Sans vocabulaire commun, elles discutent des solutions sans savoir si elles parlent d'une obligation, d'un compromis autorisé ou d'un souhait.

Rendre le statut explicite n'élimine pas le désaccord. Il permet de le placer au bon niveau. Si une exigence est contestée, l'équipe sait qu'elle doit chercher son autorité ou sa portée réelle. Si un arbitrage est possible, elle sait qui doit choisir et quelle conséquence doit être assumée. Si une préférence bloque une solution plus sûre, elle peut être abandonnée sans présenter ce changement comme une dérogation. Cette précision est particulièrement importante lorsque l'IA rend rapide la production d'une option qui paraît immédiatement séduisante.

Les contraintes se rencontrent parfois

La console d'incident ne porte pas une seule contrainte à la fois. Réduire les données indexées peut améliorer la confidentialité et diminuer la charge de recherche. Mais cela peut aussi priver l'astreinte d'un indice utile pour comprendre un incident. Limiter les accès détaillés protège une information sensible, mais peut ralentir une personne qui doit agir dans l'urgence. Conserver moins longtemps

certaines traces réduit une exposition durable, mais peut compliquer une enquête qui survient plus tard.

Il ne s'agit pas de résoudre ces tensions par une formule. L'équipe doit rendre visible la conséquence de chaque option. Dans le scénario, elle choisit de préserver l'action de rétablissement plutôt que la promesse d'une exploration exhaustive. Elle filtre les sources incertaines, rend disponibles les informations nécessaires dans un périmètre maîtrisé et conserve un mode dégradé lorsque la charge augmente. Le bénéfice abandonné est nommé : certaines recherches devront encore passer par un autre outil ou attendre une clarification.

Cette manière de procéder évite deux faux raccourcis. Le premier affirme que la confidentialité exige de ne rien rendre visible. Le second affirme que l'urgence permet de rendre tout visible. Entre les deux, une équipe peut définir quelles informations restent nécessaires à la décision, lesquelles demandent une protection supplémentaire et lesquelles ne doivent pas entrer dans le périmètre tant que leur statut n'est pas établi.

Réduire la solution, conserver la valeur

L'équipe ne renonce pas à la console. Elle la réduit.

Les champs interdits sont exclus avant l'entrée dans l'index. Les données sensibles dont la consultation demeure permise sont masquées avant l'indexation et l'accès détaillé est limité. Masquer seulement l'affichage dans le navigateur ne suffirait pas : une information déjà indexée reste exposée à d'autres usages de recherche ou d'administration.

Les sources dont le contenu est incertain sont différées. Cette décision a un coût. La première démonstration était plus spectaculaire parce qu'elle promettait une vision exhaustive. L'équipe accepte de perdre cette richesse immédiate pour ne pas créer un accès qu'elle ne saurait justifier ni reprendre facilement.

La disponibilité reçoit une réponse distincte. La console conserve un ensemble d'informations plus petit lorsque les journaux affluent. Elle

reste utile pour orienter l'action de rétablissement, même si elle ne peut pas offrir toute l'exploration prévue dans une situation calme. L'équipe ne traite donc pas la dégradation comme un échec absolu. Elle choisit ce qui doit rester fiable quand les conditions deviennent difficiles.

Cette réduction n'est pas un ajout de contrôle après la production. Elle définit ce que signifie réussir la production. Une console qui expose trop ou qui s'effondre sous charge peut être techniquement achevée et fonctionnellement séduisante ; elle ne répond pas encore correctement au besoin qui l'avait motivée.

Risque et réversibilité

Le risque aide à proportionner l'attention sans fournir une formule de calcul. Une incohérence visuelle facilement corrigible, une information sensible rendue consultable, une perte de données ou une règle métier erronée n'ont pas les mêmes conséquences. La gravité compte, mais elle ne suffit pas. Il faut aussi considérer la capacité à détecter l'erreur, le nombre de personnes affectées, le délai avant sa découverte et les engagements que la solution crée pendant ce temps.

La réversibilité pose une autre question : si nous découvrons que cette décision est mauvaise, pouvons-nous réellement revenir en arrière, rapidement et sans créer un nouveau problème ?

Dans le cas de la console, l'indexation rend cette question concrète. Retirer le bouton de recherche ou restaurer une version précédente ne supprime pas une donnée qui a déjà été consultée, copiée ou utilisée pour orienter une action. Un rollback peut arrêter la poursuite d'un effet. Il ne rétablit pas nécessairement la situation antérieure. Cette différence explique pourquoi l'équipe exige un filtrage avant l'indexation plutôt qu'un simple masquage tardif dans l'interface.

La disponibilité possède elle aussi sa forme de réversibilité. Une console qui tombe pendant un incident peut parfois être relancée, mais la décision de la faire dépendre d'un index saturable a déjà déplacé une partie du risque vers le moment où l'astreinte a le moins de capacité pour la contourner. Prévoir un mode dégradé ne promet pas l'absence

de panne. Cela préserve une voie de retour pratique vers l'information essentielle.

Le risque devient plus clair lorsque l'équipe décrit le trajet d'une erreur. Un champ sensible peut apparaître dans un journal, être copié dans un index, être retrouvé par une recherche, être consulté par une personne qui n'avait pas besoin de le voir et devenir difficile à retirer de cette chaîne. Chaque étape ajoute un effet possible. Le problème n'est pas seulement la présence initiale de l'information ; c'est la multiplication des endroits où elle peut être recherchée, recopiée ou interprétée.

Ce trajet aide également à distinguer prévention et réparation. Masquer une valeur avant indexation réduit les possibilités d'exposition. Constater après coup qu'une valeur était sensible peut encore conduire à retirer une source, à corriger une règle ou à examiner les accès, mais ne ramène pas l'équipe à la situation où l'index n'avait jamais contenu l'information. La différence entre ces gestes explique pourquoi les contraintes doivent être examinées avant que l'architecture ou le flux de données ne fige une décision.

La même logique existe pour la disponibilité. Un test qui démontre l'affichage normal de la console ne dit pas ce qui arrive lorsque les journaux affluent. Une équipe peut décider qu'un sous-ensemble de sources doit rester interrogeable, qu'un délai de réponse doit être signalé ou qu'un autre outil doit prendre le relais. Ces décisions ne donnent pas une garantie de disponibilité générale. Elles définissent ce qui doit rester possible pour que l'intention conserve son sens dans la situation où elle compte le plus.

Les contraintes préparent une décision, elles ne la terminent pas

Une fois les limites rendues visibles, l'équipe peut savoir que l'indexation brute est inacceptable et qu'un mode dégradé est nécessaire. Elle ne sait pas encore, par ce seul fait, comment vérifier que le filtrage fonctionne, que les accès sont réellement limités ou que le mode dégradé conserve les informations utiles. Les contraintes

réduisent l'espace des réponses. Elles ne constituent pas la preuve qu'une réponse y demeure.

Cette frontière évite de transformer le chapitre en liste de contrôles. Dire que la non-duplication est une contrainte dans le cas des imports ne suffit pas à autoriser la relance. Dire que les champs sensibles doivent être exclus ne suffit pas à démontrer qu'ils le sont. Le chapitre suivant devra transformer les limites importantes en propriétés observables, avec des cas qui permettent de départager des solutions plausibles.

L'équipe n'a pas besoin d'écrire toutes ces propriétés avant d'explorer. Elle doit éviter que l'exploration devienne une livraison avant que les limites qui comptent aient reçu une formulation contrôlable. C'est cette différence qui permet de conserver la vitesse de production sans laisser un agent IA choisir silencieusement les arbitrages d'acceptabilité.

Cette clarification améliore aussi la demande adressée à un agent IA. Plutôt que de lui demander une console qui rassemble « tout ce qui peut aider », l'équipe peut préciser les catégories admissibles, les données à exclure, les conditions d'accès et le comportement attendu sous charge. La production résultante peut encore être incomplète ou demander une revue. Mais elle expose moins de décisions que personne n'avait choisi de confier à cet agent.

Les contraintes ne sont donc pas une couche ajoutée à une intention déjà achevée. Elles donnent à cette intention sa forme praticable. Aider l'astreinte n'autorise pas n'importe quel accès, n'importe quelle conservation ni n'importe quelle dépendance technique. La valeur recherchée demeure, mais elle doit être livrée dans les limites que l'équipe peut expliquer et assumer.

Lorsqu'une limite paraît seulement gêner la solution préférée, l'équipe peut enfin demander quelle conséquence elle accepterait si cette limite cessait de jouer son rôle. L'impact et la réversibilité pratique indiquent alors le niveau de clarification, de spécification et de preuve nécessaire ; le chapitre 7 précisera comment placer cette preuve avant la propagation.

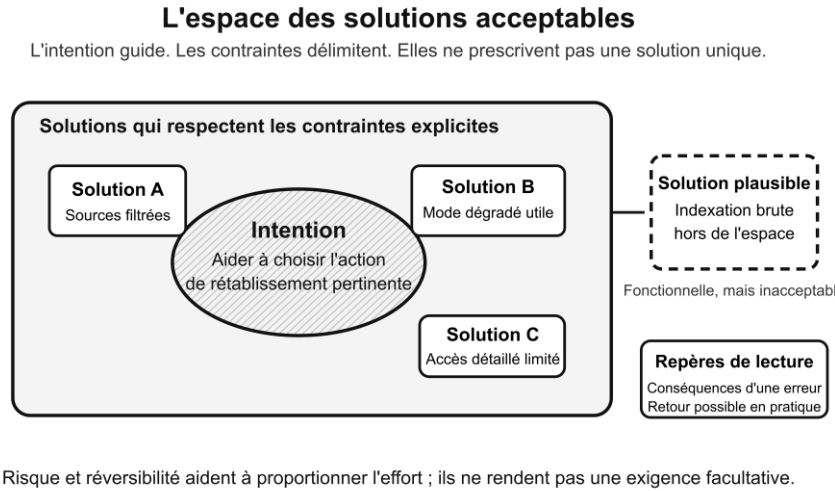


Figure 5.2 - Cadre proposé par le livre. L'intention oriente les solutions ; les contraintes rendent certaines d'entre elles inacceptables.

Les limites ne sont pas un refus de la valeur

Une équipe peut ressentir la contrainte comme une perte : moins de sources dans la console, moins d'accès immédiat, moins de richesse dans la démonstration. Ce ressenti est juste. Un arbitrage abandonne parfois un bénéfice visible. Mais il permet aussi de conserver la valeur qui avait justifié le changement : une aide réellement utilisable pendant l'incident, plutôt qu'une interface exhaustive dont l'accès ou la disponibilité ne peuvent être assumés.

Cette distinction évite deux excès. Le premier consiste à appeler « prudence » le refus vague de toute évolution. Le second consiste à appeler « livraison » une fonctionnalité dont les conséquences ont été laissées aux personnes qui l'exploiteront. Les contraintes ne demandent ni immobilité ni héroïsme. Elles demandent de dire quel coût est accepté et quelle limite ne l'est pas.

Dans la console, différer une source incertaine est une décision productive : elle permet de livrer un premier périmètre qui aide l'astreinte, tout en évitant de fabriquer un accès qu'il faudrait retirer

après une exposition. Le périmètre réduit reste une réponse. Il ne devient pas une promesse que l'équipe ne pourrait pas tenir.

Une solution réduite peut même améliorer la compréhension du produit. Lorsque la console ne prétend plus agréger indistinctement tous les éléments, l'équipe doit dire ce que la personne d'astreinte doit réellement pouvoir voir pour choisir une action. Les informations qui restent deviennent plus faciles à justifier. Les sources différées deviennent des questions de produit et d'exploitation plutôt que des données ajoutées par défaut parce qu'elles étaient disponibles.

La réduction crée aussi une condition de reprise. Une source exclue aujourd'hui n'est pas oubliée. L'équipe peut la réexaminer lorsque son contenu est caractérisé, lorsque le filtrage avant indexation existe ou lorsqu'un mandat d'accès plus précis a été défini. Cette condition évite que le « pas maintenant » soit soit un refus sans retour, soit une promesse silencieuse de tout réintégrer plus tard.

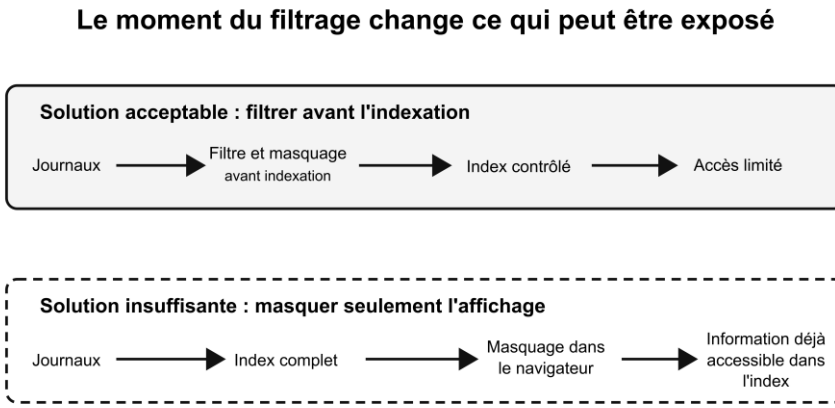


Figure 5.3 - Scénario construit. Le filtrage intervient avant l'indexation lorsque l'index lui-même crée une possibilité d'accès.

Une reprise : la relance des imports

Revenons brièvement aux imports de commandes. L'intention était d'aider l'opérateur à choisir une action de rétablissement. Un bouton de relance paraît servir cette direction. Pourtant, si un lot a déjà appliqué

certaines lignes, une relance aveugle peut créer des commandes en double.

La non-duplication est ici une limite confirmée. Le bouton peut rester absent, ou ne devenir disponible que dans des conditions que l'équipe n'a pas encore démontrées. Cette retenue n'empêche pas d'aider l'opérateur. Elle évite de lui donner une action rapide dont la conséquence serait plus coûteuse que l'incident initial.

Le chapitre ne décide pas encore comment établir cette non-duplication. Il ne produit pas les tests ni les preuves nécessaires. Il rend seulement visible pourquoi une solution conforme à l'intention ne peut pas être acceptée sans cette limite.

> **Dans une pratique SDD** `constitution.md` exprime des principes durables et `spec.md` les limites propres à la fonctionnalité : règles d'acceptabilité, frontières de périmètre ou exigences impératives. Écrites avant `plan.md`, elles peuvent encore écarter une conception qui les franchit.

Ce qui doit être vrai

Une intention sans contrainte peut produire très vite une solution convaincante, mais incapable de respecter ce qui compte réellement. Les contraintes réduisent le champ des réponses. Elles ne sont pas des freins ajoutés à la fin : elles définissent l'espace dans lequel une réponse devient acceptable.

Elles ne suffisent pas encore à accepter une solution. Dire qu'une console doit préserver la confidentialité ou rester utile sous charge ne permet pas, à lui seul, de constater qu'elle le fait. Il faut maintenant exprimer ce qui devra être vrai, ce qui reste à confirmer et ce qu'une preuve devra pouvoir contredire.

Le chapitre suivant transforme ces limites en propriétés observables. C'est là que la spécification vérifiable reliera l'intention, les contraintes et la décision d'accepter une solution.

Chapitre 6 - Spécifier ce qui doit être vrai

Les contraintes ont réduit l'espace des solutions acceptables. Elles ne disent pas encore comment constater qu'une solution y appartient.

Dire qu'une console doit préserver la confidentialité ou qu'un changement ne doit pas compromettre un engagement commercial donne une limite nécessaire. Une implémentation peut néanmoins sembler cohérente, être accompagnée de tests et manquer exactement le comportement que cette limite devait protéger. Il faut donc pouvoir formuler une question plus précise : qu'est-ce qui doit être vrai pour accepter ce résultat ?

Cette question ne demande pas de décrire tout le code à produire. Elle demande de rendre observables les propriétés qui départagent une solution plausible d'une solution acceptable.

Une demande assez précise pour coder, trop vague pour décider

Le scénario est construit. Une équipe doit adapter le calcul d'une option de livraison urgente. Une livraison standard coûte fictivement 100 € ; l'option urgente ajoute 20 % du prix standard et porte donc le total de cet exemple à 120 €. La règle est le pourcentage, non un supplément fixe de 20 €. L'intention est simple : proposer le bon montant sans promettre un délai raccourci qui ne pourra pas être tenu. Les contraintes sont également connues dans leurs grandes lignes : ne pas appliquer l'option hors des conditions prévues, ne pas annoncer un prix définitif quand une information manque et permettre au responsable qui présente le prix de l'expliquer.

Un agent IA, souvent fondé sur un LLM, produit rapidement une modification lisible, des tests unitaires et une explication. La demande initiale disait seulement : « appliquer l'option de livraison urgente ». Plusieurs exemples simples donnent le montant attendu. Pourtant, deux implémentations raisonnables peuvent prendre des décisions opposées à la frontière de l'heure limite, si le créneau urgent n'est pas confirmé ou si la préparation a déjà commencé.

Pour la catégorie d'envoi standard de cet exemple, l'équipe fixe les règles suivantes : la livraison de base vaut 100 €, l'option urgente ajoute 20 % de ce prix, la demande doit être reçue strictement avant 16 h, un créneau urgent doit être confirmé et la préparation ne doit pas avoir commencé. L'heure retenue est celle enregistrée par le serveur, dans le fuseau d'exploitation du service. Les montants sont fictifs et ne décrivent aucune politique commerciale réelle.

Deux formes de code peuvent alors sembler naturelles. L'une accepte une demande « avant 16 h » ; l'autre l'accepte « jusqu'à 16 h incluse ». Elles donnent le même résultat à 15 h 59 et à 16 h 01. Elles divergent exactement là où le langage vague cachait une décision.

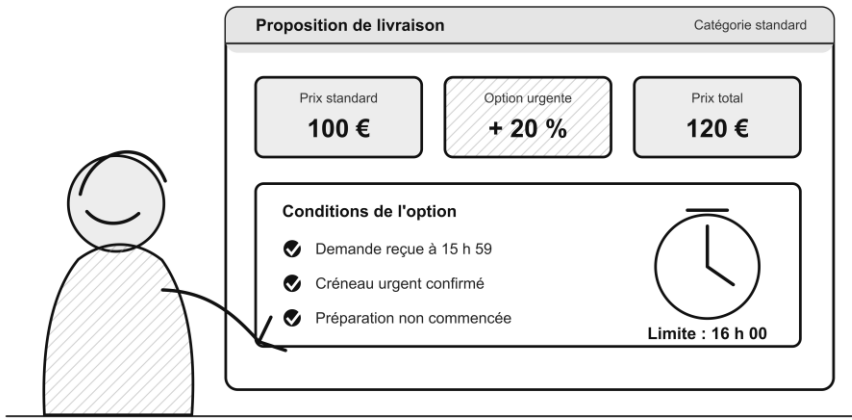


Figure 6.1 - Scénario construit. Le montant de l'option urgente dépend du prix, de l'heure reçue, du créneau confirmé et de l'état de préparation.

Situation construite	Ce qui doit être vrai
Demande à 15 h 59, créneau confirmé, préparation non commencée	Le prix total est de 120 € ; la base de 100 € et les 20 % de l'option, soit 20 €, sont visibles.
Demande à 16 h exactement	L'option urgente est indisponible ; aucun supplément n'est appliqué.
Demande à 16 h 01	L'option urgente est indisponible.

Situation construite	Ce qui doit être vrai
Créneau inconnu à 15 h 59	Le statut reste « à confirmer » ; aucun prix total avec l'option n'est proposé.
Créneau refusé	L'option urgente est indisponible.
Préparation déjà commencée	L'option n'est pas ajoutée rétroactivement au montant engagé.

Cette table ne donne pas une recette d'implémentation. Elle rend possible une discussion honnête sur le résultat. Une équipe peut choisir une règle différente pour un autre produit, mais elle ne peut plus prétendre que les deux lectures de la même borne sont équivalentes.

Le responsable qui présente le prix doit également pouvoir expliquer ce qui a conduit au montant. Lorsque l'option urgente est acceptée, la base, le taux de 20 %, le montant calculé du supplément, l'heure de réception et le créneau confirmé font partie du résultat attendu. Cette exigence n'impose pas une interface particulière. Elle empêche qu'un calcul correct en apparence devienne inexplicable au moment où un client demande pourquoi l'option a été acceptée ou refusée.

La distinction entre une option urgente indisponible et une livraison standard encore possible reste volontairement ouverte. Le scénario ne décide pas qu'un refus de l'option doit produire une seconde proposition. Il fixe seulement ce que le système ne doit pas faire : annoncer un prix incluant l'option lorsque le créneau urgent n'est pas confirmé. Une spécification utile protège aussi ce qu'elle ne décide pas encore.

Les six cas se lisent comme une petite enquête sur le sens de la règle. Le premier confirme que l'option peut réellement être proposée. Les deux suivants montrent que la borne exclusive n'est pas décorative. Le quatrième fait apparaître qu'une information inconnue n'est pas une autorisation implicite. Le cinquième sépare l'absence de créneau d'un défaut de calcul. Le dernier protège un prix déjà engagé contre une modification rétroactive.

Cette progression évite de résumer le besoin par une formule numérique. « Ajouter 20 % » décrit un calcul. Elle ne décrit pas quand ce calcul a le droit d'exister, ce que le système doit expliquer, ni ce qui doit rester inchangé. Les cas frontières donnent à l'équipe des moyens de comparer des implémentations différentes sans leur demander d'adopter la même structure technique.

Ils permettent aussi d'éviter une confusion fréquente entre refus et erreur. À 16 h 01, l'option urgente indisponible n'est pas une panne de l'interface. C'est le résultat attendu de la règle. Lorsqu'aucun créneau urgent n'est disponible, le même résultat vient d'une autre condition. Conserver cette distinction aide le responsable à expliquer ce qui s'est passé et évite que l'équipe traite tous les refus comme des défauts techniques à contourner.

La préparation déjà commencée ajoute une limite temporelle différente. Même si un créneau urgent apparaissait ensuite, l'option ne modifie pas le montant déjà engagé. Cette propriété protège une relation commerciale et pas seulement un calcul. Elle rappelle qu'une spécification peut concerner un état du monde, un engagement ou une explication attendue, et non la seule valeur renvoyée par une fonction.

Qui doit pouvoir prédire le résultat ?

Une spécification utilisable ne s'adresse pas seulement à la personne qui implémente. Dans le scénario, le responsable qui présente le prix, la personne qui maintient le calcul et celle qui vérifie la modification doivent pouvoir prédire les résultats des cas importants sans lire le code. Ils n'ont pas besoin d'employer les mêmes mots techniques. Ils doivent pouvoir reconnaître le même engagement : avant 16 h ne signifie pas à 16 h, un créneau inconnu ne devient pas une promesse et une préparation commencée n'est pas recalculée.

Cette possibilité de prédiction change la qualité de la conversation. Si une personne lit la règle et s'attend à 120 € à 16 h, le désaccord apparaît avant la mise en production. Si elle découvre la différence seulement devant l'interface, le système a déjà choisi une

interprétation. Le critère observable ne remplace pas la discussion métier ; il lui donne un moment et un objet plus précis.

L'équipe peut également vérifier la cohérence d'une explication. Un prix total de 120 € doit faire apparaître la base de 100 €, le taux de 20 %, les 20 € qui en résultent, l'heure retenue et le créneau urgent confirmé. Cette exigence n'ajoute pas un écran pour le plaisir. Elle permet de relier le montant à la règle lorsque le client, le support ou un responsable demande pourquoi le résultat a été produit. Un système qui calcule correctement mais ne peut plus expliquer ce qu'il a calculé garde une fragilité importante.

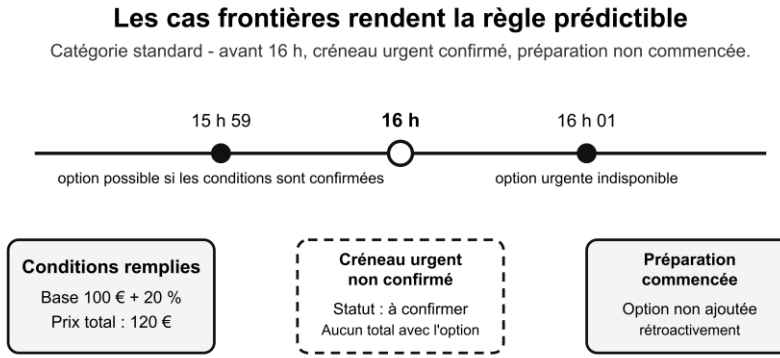


Figure 6.2 - Scénario construit. Les cas frontières distinguent une règle de résultat d'une simple formule de calcul.

Une spécification vérifiable

Une spécification vérifiable exprime ce qui doit être vrai pour accepter un résultat. Elle peut prendre la forme d'une règle, d'un exemple, d'un contre-exemple, d'un seuil, d'une propriété ou d'une condition d'arrêt. Sa forme importe moins que sa capacité à rendre une réponse observable.

Le mot « observable » ne demande pas que tout soit automatiquement mesuré. Un responsable peut devoir relire les éléments qui justifient un prix. Une personne d'exploitation peut confirmer un créneau urgent. Un cas de test peut comparer un résultat à une frontière. Ces moyens sont

différents, mais ils partagent une exigence : l'équipe doit pouvoir répondre autrement que par « la solution nous semble raisonnable ».

Cette exigence protège également les personnes qui reçoivent la production. Une proposition générée peut inclure une explication persuasive, des tests et un exemple. Si personne ne sait quel résultat elle devait préserver, la revue devient une appréciation de style ou de confiance. Une propriété annoncée donne au relecteur la possibilité de demander ce qui manque, sans exiger qu'il réécrive l'implémentation.

Elle sépare quatre objets qui se confondent facilement lorsque le code arrive vite :

Objet	**Question qu'il porte**
Intention	Quel résultat cherchons-nous à rendre possible ?
Contrainte	Qu'est-ce qu'une solution ne peut pas compromettre ?
Spécification	Qu'est-ce qui doit être vrai pour ce changement ?
Preuve	Qu'est-ce qui nous permettra de constater ou de contredire cette propriété ?

L'intention explique pourquoi le bon montant compte. Les contraintes excluent un engagement injustifié ou impossible à expliquer. La spécification rend concrètes ces limites dans les cas du changement étudié. La preuve, elle, reste à produire. Déclarer qu'une borne est exclusive ne démontre pas que le code l'applique correctement.

Cette séparation laisse de la liberté là où elle est utile. La règle « une livraison dont la préparation a commencé ne reçoit pas l'option rétroactivement » n'impose ni un langage, ni une structure de données, ni une suite particulière d'appels. Elle ne transforme pas la spécification en cahier de solution déguisé. Elle permet seulement d'écarter une implémentation qui modifierait un montant déjà engagé.

L'inconnue n'est pas un trou à camoufler

Une autre catégorie d'envoi, dite spéciale dans le scénario, ne possède pas encore d'heure limite confirmée. La règle de la catégorie standard ne lui est pas appliquée par défaut. Tant que le responsable d'exploitation n'a pas clarifié le seuil, le système ne propose pas l'option urgente pour cette catégorie.

Cette limite ouverte ne rend pas la spécification faible. Elle rend sa portée honnête. Une formulation apparemment complète qui inventerait une borne pour la catégorie spéciale produirait plus vite une réponse, mais ferait passer une connaissance manquante pour une décision établie.

Statut	**Place légitime dans la spécification**
Exigence confirmée	Ce qui doit être respecté.
Fait établi	Ce qui peut fonder une décision, sans devenir automatiquement une exigence.
Observation ou déduction	Ce qui éclaire la décision avec son statut conservé.
Supposition	Une hypothèse à nommer, non une règle à dissimuler.
À confirmer	Une condition ouverte qui limite ce qui peut être accepté.

Spécifier ne signifie donc pas tout prévoir. Cela signifie dire ce que nous savons exiger, ce que nous sommes capables de constater et ce que nous n'avons pas encore le droit de considérer comme réglé.

Cette clarté peut accélérer une exploration au lieu de la ralentir. L'équipe peut demander à un agent IA plusieurs variantes de calcul, comparer leur traitement des six cas et écarter immédiatement celles qui appliquent l'option après 16 h ou sans créneau confirmé. Elle lui confie la recherche de forme sans lui confier le choix de ce qui compte. Lorsque la catégorie spéciale revient dans la discussion, elle sait que ce n'est pas un défaut de l'implémentation : c'est une décision métier encore ouverte.

Une précision proportionnée

Une spécification ne devient pas meilleure parce qu'elle décrit chaque détail. Pour un changement local, facile à observer et à annuler, quelques propriétés claires peuvent suffire. Lorsque les effets se propagent dans des données, un contrat, un prix ou une relation avec un client, les cas frontières et les conditions d'arrêt demandent davantage d'attention.

La granularité peut aussi évoluer avec l'enquête. Au début, l'équipe peut ne formuler que le besoin d'empêcher un prix urgent injustifié. Les premières discussions révèlent que l'heure de réception doit être définie, que l'heure du poste client n'est pas une référence fiable et qu'un créneau urgent ne peut pas être remplacé par une estimation. Chaque précision ajoutée doit répondre à une ambiguïté qui risquerait autrement de départager les implémentations au hasard. Elle n'est pas une démonstration de rigueur pour elle-même.

Une bonne question de relecture est donc : quelle décision une variation de cette phrase pourrait-elle changer ? Si la réponse est « aucune », le détail peut probablement attendre. Si elle est « le montant proposé, l'engagement pris ou la possibilité de revenir en arrière », la propriété mérite d'être explicite. Ce test aide à produire une spécification proportionnée sans donner un formulaire universel.

Le risque et la réversibilité modulent donc la précision nécessaire. Ils ne rendent pas une exigence impérative facultative. Une équipe peut accepter qu'un détail de présentation reste à préciser ; elle ne peut pas appeler « incertitude acceptable » une condition commerciale qu'elle n'a pas le droit de violer.

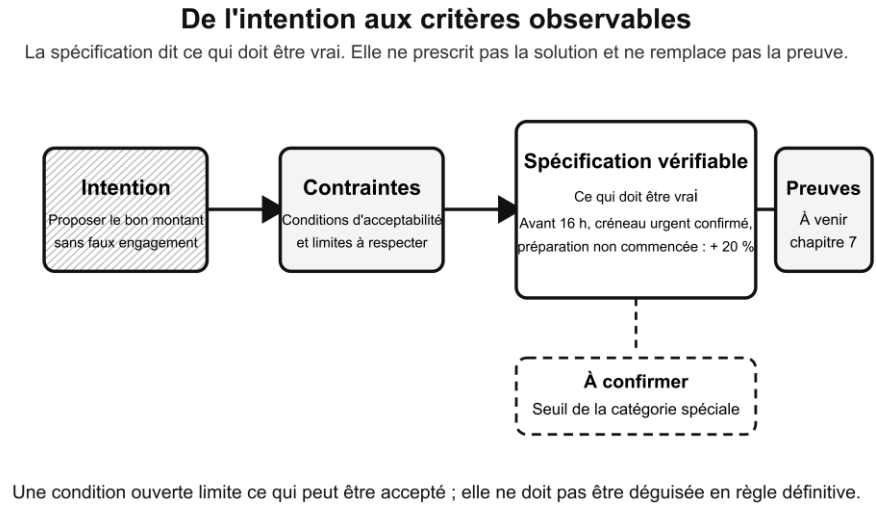


Figure 6.3 - Cadre proposé par le livre. L'intention et les contraintes rendent la spécification possible ; les preuves devront ensuite la confronter au résultat.

Les cas frontières ne sont pas des détails

Les cas les plus faciles créent souvent un accord trompeur. À 15 h 59, une règle inclusive et une règle exclusive donnent le même résultat. Elles divergent seulement à 16 h. Pourtant, c'est cette minute qui révèle ce que l'équipe a choisi de promettre. La frontière n'est pas un détail de programmation ajouté après la règle ; elle est l'endroit où la règle devient intelligible.

Les conditions de créneau et de préparation jouent le même rôle. Elles empêchent qu'un calcul exact en apparence propose une livraison urgente impossible ou modifie après coup un engagement déjà pris. Les rendre visibles aide autant le responsable qui explique le prix que la personne qui implémente le calcul. Tous deux peuvent alors discuter d'un résultat, non d'une intuition cachée dans une branche de code.

Une règle peut protéger une décision sans la prendre

La règle de tarification n'indique pas quel outil doit calculer le montant, ni quelle personne doit écrire le texte présenté au client. Elle fixe ce qui

doit rester vrai lorsque ces décisions techniques et opérationnelles sont prises. Cette séparation permet à l'équipe de modifier une interface, de déplacer un calcul ou de confier à un agent IA une variante d'implémentation sans rouvrir à chaque fois le sens de la promesse commerciale.

Elle ne dispense pas pour autant de choisir. Dire que l'heure du serveur fait foi est déjà une décision. Elle écarte l'heure affichée par le poste client parce que l'équipe a besoin d'une référence commune pour expliquer le prix. Dire que le créneau urgent doit être confirmé est une autre décision : elle refuse qu'une estimation soit transformée en engagement. Une spécification vérifiable ne masque pas ces arbitrages ; elle leur donne une forme que le résultat devra respecter.

Cette forme protège aussi contre les solutions qui semblent plus généreuses parce qu'elles remplissent les zones grises par défaut. Une implémentation pourrait appliquer l'option lorsqu'aucune information de créneau n'est reçue, afin d'éviter un écran d'attente. Elle proposerait ainsi davantage de livraisons urgentes. Mais elle ne répondrait plus à l'intention de ne pas prendre un engagement que l'équipe ne sait pas tenir. Le statut « à confirmer » n'est pas une absence de produit : c'est le résultat honnête d'une condition non établie.

Spécifier pour pouvoir comparer

Lorsqu'une équipe reçoit plusieurs propositions, la spécification devient un instrument de comparaison. Elle ne demande pas quelle variante est la plus élégante, la plus courte ou la plus persuasive. Elle demande comment chacune traite l'heure exacte, la préparation déjà commencée, le créneau manquant et l'explication du montant. Une variante peut alors être rejetée sans que son auteur soit accusé d'avoir « mal codé » : elle ne respecte simplement pas une propriété que l'équipe a choisi de rendre décisive.

Cette comparaison peut révéler que la spécification elle-même est insuffisante. Si deux propositions respectent tous les cas connus mais produisent des conséquences très différentes pour le client, l'équipe n'a pas nécessairement trouvé deux bonnes solutions. Elle a peut-être

découvert une intention ou une contrainte encore trop vague. Le retour vers les chapitres précédents est normal. Intention, contraintes et critères ne forment pas une chaîne à sens unique ; ils se précisent mutuellement.

La spécification ne promet donc pas de supprimer les désaccords. Elle les déplace vers l'objet qui doit être réellement discuté. C'est un progrès lorsque la production assistée rend trop facile de contourner cette discussion par une implémentation déjà prête.

Elle rend aussi les accords réutilisables. Une fois la borne, le créneau urgent et la préparation déjà commencée explicités, l'équipe ne doit pas recommencer la même enquête à chaque correctif d'interface, migration de service ou reprise de calcul. Elle peut réexaminer la règle lorsque le contexte change, mais elle ne dépend plus de la mémoire implicite de la personne qui avait compris la première version.

Cette réutilisation ne transforme pas le critère en vérité éternelle. Elle conserve justement la possibilité de dire pourquoi il existe, dans quel périmètre il s'applique et quelle information exigerait de le réviser. Une spécification est vivante lorsqu'elle maintient cette relation entre une décision, ses conditions et les résultats qu'elle permet de vérifier.

Une condition ouverte protège aussi la suite

L'équipe pourrait être tentée de compléter la règle de la catégorie spéciale avec une valeur provisoire, simplement pour rendre l'écran uniforme ou obtenir une suite de tests complète. Cette apparente complétude aurait un prix : elle ferait d'une hypothèse locale une règle commerciale qui pourrait être reprise par les clients, le support ou un autre système.

Conserver la condition ouverte donne une autre forme de progression. L'équipe peut préparer l'interface, isoler le calcul standard, rendre le statut « à confirmer » compréhensible et adresser la question au responsable d'exploitation. Elle ne propose pas l'option urgente à la catégorie spéciale tant que le seuil manque. Cette limite indique aussi ce qui devra être revu lorsque la réponse arrivera : la règle, les cas frontières et les preuves qui permettront de vérifier son application.

Une inconnue bien visible est donc plus utile qu'une précision inventée. Elle évite deux dérives opposées : bloquer toute production sous prétexte qu'un point reste ouvert, ou laisser une valeur par défaut prendre la place d'une décision. La première empêche l'exploration. La seconde crée une dette de décision. Entre elles, une spécification peut délimiter un périmètre réellement accepté et une question explicitement réservée.

Cette réserve devient particulièrement importante lorsqu'une variante générée paraît complète. Le code peut afficher un montant, traiter tous les chemins connus et produire des tests cohérents. Sa complétude apparente ne donne pas le statut de fait à une règle que l'équipe n'a pas confirmée. Une solution peut être prête à être explorée tout en restant non prête à être engagée.

Reprendre l'identité des références

Le nettoyage des références clients du chapitre 3 offre un contraste utile. L'exigence confirmée est de préserver l'identité. La connaissance manquante concerne l'équivalence de `P-0042` et `42`.

La propriété peut alors être formulée sans inventer cette connaissance : deux références dont l'équivalence métier n'est pas confirmée ne sont pas fusionnées ; une collision vers une clé commune reste visible et empêche l'acceptation de la conversion. Cette règle ne dit pas si les deux références appartiennent au même client. Elle empêche seulement le code de décider cette question à la place de l'équipe.

Le critère peut être vérifiable et pourtant mal choisi. Il peut aussi être respecté par une implémentation qui échoue ailleurs. C'est pourquoi spécifier n'épuise ni le jugement ni la validation. Mais sans propriété explicite, une solution qui paraît propre peut faire disparaître l'information même dont l'équipe avait besoin pour décider.

Le même raisonnement s'applique au langage employé. « Fusionner les doublons » semble simple tant que l'on ne demande pas ce qui fait un doublon. La propriété choisie ne prétend pas résoudre cette question ; elle rend impossible une fusion silencieuse lorsque l'équivalence n'est

pas confirmée. La spécification ne produit pas la connaissance manquante. Elle empêche que l'implémentation transforme cette absence de connaissance en fait accompli.

Préparer la contradiction utile

Une solution peut être lisible, cohérente et accompagnée de tests sans répondre encore à ce qui doit réellement être vrai. Les tests peuvent partager la même hypothèse erronée que le code. Une démonstration peut couvrir les cas faciles et éviter précisément la frontière qui compte.

La spécification ne demande pas une confiance plus grande dans la production. Elle donne à une preuve la possibilité de la contredire. La règle « strictement avant 16 h » permet de concevoir un cas qui révélera une interprétation inclusive. La confirmation du créneau rend visible ce qui doit empêcher une promesse de délai injustifiée. La limite ouverte de la catégorie spéciale interdit de prendre une décision définitive avant l'information nécessaire.

Cette capacité de contradiction modifie aussi la manière de demander une production assistée. Au lieu de solliciter seulement « une implémentation de l'option de livraison urgente avec ses tests », l'équipe peut fournir les propriétés et demander quelles hypothèses la proposition fait sur les cas limites. La réponse produite devient alors une matière à examiner : elle peut révéler une condition oubliée, une ambiguïté dans le vocabulaire ou une liberté technique qui mérite d'être conservée.

La spécification protège ainsi l'équipe d'une fausse alternative entre deux excès. Elle ne doit pas donner à l'agent une consigne si vague que la solution choisit les règles à la place des personnes responsables. Elle ne doit pas non plus décrire la solution au point de supprimer toute capacité de proposition et de comparaison. Elle rend visible le résultat auquel une solution devra répondre, puis laisse ouverte la recherche de sa forme.

Cette frontière est particulièrement utile quand plusieurs solutions paraissent fonctionner. Une variante peut calculer correctement 120 €

dans le cas favorable mais ignorer le créneau inconnu. Une autre peut gérer la borne de 16 h mais modifier le montant après le début de la préparation. Sans les propriétés explicites, chacune peut présenter des tests verts et une explication cohérente. Avec elles, l'équipe peut dire ce que le résultat ne doit pas sacrifier pour réussir un exemple simple.

> **Dans une pratique SDD** Dans Spec Kit, `specify` peut produire un `spec.md` d'exigences, de scénarios, de cas frontières et de questions ouvertes. `clarify` fait apparaître les ambiguïtés utiles avant que `plan` et `tasks` ne donnent une forme au travail dérivé.[^speckit] Ce parcours reste adaptable : l'équipe choisit ce qui compte et ce qui doit encore être confirmé.

La preuve du chapitre suivant ne remplacera pas cette clarification. Un test ou une simulation ne peut établir que ce qui a été demandé ou découvert à son occasion. En revanche, une propriété formulée avec assez de précision permet de construire une preuve qui ne se contente pas d'accompagner le code. Elle peut réellement le mettre à l'épreuve.

La question suivante n'est donc plus seulement « qu'avons-nous demandé au système de faire ? ». Elle devient : « quelle preuve, à quel moment, pourrait nous apprendre que ce que nous avons produit ne respecte pas encore cette règle ? »

Portée

Les montants, délais, catégories et résultats de ce chapitre sont des constructions pédagogiques. Ils illustrent la différence entre une solution proposée, une propriété à préserver et une condition ouverte. Ils ne démontrent ni une règle tarifaire générale ni l'efficacité universelle d'une pratique de spécification.

Sources et portée

speckit. GitHub, *Spec Kit - Spec-Driven Development*, consulté le 20 septembre 2026. La source décrit une boîte à outils, ses artefacts et des parcours indépendants ; elle ne valide ni le scénario tarifaire ni le cadre de décision proposé par ce livre.

Chapitre 7 - Valider avant que l'erreur ne se propage

Un code compilé, des tests verts et une démonstration locale peuvent donner l'impression qu'un changement est terminé. Ils établissent parfois des propriétés importantes. Ils n'établissent pas nécessairement ce qui se produira là où les conséquences du changement apparaîtront.

La validation n'est donc pas le dernier contrôle appliqué à une solution déjà finie. Elle place une preuve au moment où une erreur peut encore modifier une décision, avant de se propager dans des données, des intégrations ou des habitudes de travail.

Un événement correct pour son producteur

Le scénario est construit. Une équipe enrichit l'événement `RequestAccepted` d'un champ facultatif, `serviceLevel`. Un nouveau partenaire a besoin de cette information pour traiter une catégorie de demandes qui lui échappait jusque-là.

Un agent IA identifie les producteurs et consommateurs connus, adapte le schéma, modifie la sérialisation et génère des tests. Le build est vert. Le producteur émet bien la nouvelle valeur dans son environnement. Cette preuve est réelle, mais elle est locale : elle montre ce que le producteur fait, non ce que tous les lecteurs pourront comprendre.

Un consommateur ancien valide strictement les champs qu'il accepte et rejette tout champ supplémentaire. Un consommateur nouveau, inversement, doit pouvoir recevoir d'anciens événements qui ne comportent pas `serviceLevel` sans inventer un niveau de service. Deux questions distinctes apparaissent : les lecteurs existants peuvent-ils continuer à traiter le nouveau format ? Et le nouveau lecteur sait-il conserver l'absence de l'information comme une inconnue ?

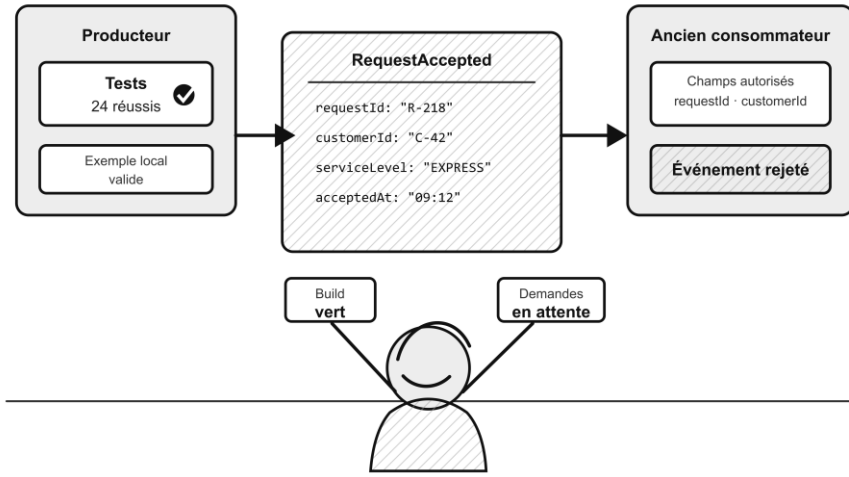
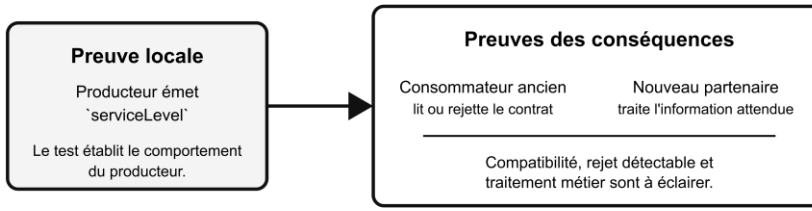


Figure 7.1 - Scénario construit. Le producteur émet un événement valide localement que l'ancien consommateur rejette.

Le défaut ne tient pas au principe d'ajouter un champ. Il tient à une règle de lecture qui n'apparaît pas dans le test du producteur. Un rejet non observé pourrait laisser des demandes sans traitement pendant plusieurs heures. La documentation de certains registres de schémas rappelle d'ailleurs que la compatibilité dépend du lecteur, de la version et du format ; elle ne peut pas être déduite du seul changement de l'émetteur.[^confluent]

Le producteur peut être correct sans que le flux soit validé



Un test vert ne vaut que pour la propriété qu'il a réellement exercée.

Figure 7.2 - Scénario construit. La preuve éclaire une propriété là où les conséquences du changement apparaissent.

Deux décisions, deux moments de preuve

La première décision est d'accepter ou non le contrat avant de le diffuser. L'équipe confronte la nouvelle version à l'ancien consommateur et reproduit le rejet du champ supplémentaire. Elle confronte aussi le nouveau consommateur à un ancien événement incomplet. La preuve contredit l'idée que le changement peut être publié tel quel sur le flux existant.

Dans ce cas, l'équipe choisit un flux de version distincte pour le nouveau partenaire et conserve le contrat ancien. Ce choix crée un coût de coexistence. Il ne recommande pas de versionner chaque ajout de champ. Il répond à une compatibilité précise, étudiée avec les versions concernées. Si le routage ou le comportement attendu ne peuvent pas être établis, le contrat reste à reprendre.

Cette alternative illustre un point important : une preuve contradictoire ne force pas toujours l'abandon du résultat recherché. Elle peut conduire à modifier la forme de la propagation. L'équipe conserve le bénéfice de la nouvelle information pour le partenaire qui en a besoin, mais elle refuse de faire porter le coût de cette évolution aux consommateurs dont la lecture n'a pas été confirmée. Réduire le périmètre est ici une décision de livraison, non une capitulation.

La deuxième décision intervient après une diffusion bornée. Avant la première émission externe, l'équipe vérifie une simulation de rejet : l'alerte fonctionne, l'émission peut être arrêtée et les événements en attente sont conservés pour une reprise. Elle autorise ensuite une diffusion limitée au nouveau partenaire.

Les réceptions, rejets et traitements observés après cette émission sont des informations nouvelles. Elles peuvent justifier d'élargir, de réduire ou d'arrêter la diffusion. Elles ne prouvent pas rétroactivement que la première décision était sans risque. Cette distinction empêche de confondre une capacité de surveillance vérifiée avant émission avec le résultat réel d'une diffusion.

La décision d'élargir mérite d'être formulée séparément parce qu'elle engage un nouveau périmètre. Une première réception réussie chez le nouveau partenaire ne signifie pas encore que les événements inhabituels, les périodes de forte charge ou les reprises après interruption auront le même comportement. L'équipe n'a pas besoin d'attendre une certitude totale. Elle doit savoir quelle information manque encore, quelle conséquence deviendrait plus coûteuse avec un périmètre plus large et quel signal la ferait revenir sur son choix.

Dans le scénario, l'arrêt d'émission et la conservation des événements en attente ne sont pas des détails opérationnels ajoutés pour rassurer. Ils donnent une issue à une preuve défavorable. Sans eux, une alerte signalerait un rejet alors que l'équipe ne pourrait plus empêcher la perte ou la poursuite silencieuse du flux. Une preuve a davantage de valeur lorsque l'équipe peut encore agir à partir de ce qu'elle apprend.

Seuil de décision	Preuve utile	Ce qui reste ouvert
Accepter le contrat	Les lecteurs étudiés traitent le format prévu ou la compatibilité est volontairement isolée.	Les comportements non étudiés et les versions hors périmètre.
Élargir la diffusion	Les signaux de réception, rejet et traitement sont observables sur un périmètre borné.	Les conséquences d'un périmètre plus large.

Le tableau ne remplace pas le jugement. Il montre qu'une même évolution peut exiger plusieurs décisions parce qu'elle change de nature lorsqu'elle franchit un nouveau seuil. Accepter un contrat revient à dire que la forme du message est assez comprise pour un périmètre donné. Élargir une diffusion revient à accepter les conséquences d'une exposition plus grande. Réunir ces deux décisions sous le mot vague de « déploiement » rend plus difficile de savoir quelle preuve manque réellement.

Un gate n'est pas une formalité ni une étape universelle. C'est un seuil où une décision devient plus coûteuse à annuler. La question est simple : quelle propriété importante devons-nous éclairer avant de

franchir ce seuil, et que ferons-nous si la preuve contredit notre hypothèse ?

Une preuve peut contredire la production

Le chapitre précédent a fixé une règle construite : l'option de livraison urgente peut être proposée strictement avant 16 h. Une implémentation peut traduire cette règle par une borne inclusive. Si les tests ont été générés à partir de la même lecture, ils attendront eux aussi l'option à 16 h exactement. Le pipeline vert prouvera leur accord, pas leur conformité à la règle.

Le cas métier indépendant attend au contraire que l'option urgente soit indisponible à 16 h. Il peut faire échouer une solution convaincante. C'est ce qui lui donne sa valeur. Une seconde IA pourrait aider à produire ce test, mais elle ne garantit pas l'indépendance de l'hypothèse. La question importante est l'origine justifiée du résultat attendu et sa capacité à contredire ce qui a été produit.

Une preuve utile répond donc à une propriété précise. Un test de contrat peut éclairer une compatibilité. Une simulation peut montrer un comportement de flux. Une relecture peut examiner un compromis qu'aucun test ne capture seul. Une mesure peut signaler une dégradation. Une activité de contrôle sans propriété annoncée peut avoir une utilité, mais elle ne doit pas devenir une preuve vague de « qualité ».

La revue est particulièrement utile lorsqu'elle rend une hypothèse lisible. Dans le scénario, elle peut demander : quel consommateur a été réellement exercé ? Quelle version lit l'événement ? Que signifie l'absence de `serviceLevel` pour le nouveau partenaire ? Ces questions ne remplacent pas l'essai sur le lecteur strict. Elles évitent que le résultat de cet essai soit interprété comme une propriété plus large qu'il ne l'est.

De même, une alerte n'est pas une preuve que le flux est sain. Elle établit, si elle a été exercée, qu'un type de rejet peut être rendu visible et conduire à un arrêt. Cette capacité change la décision de diffuser parce qu'elle réduit le temps de détection. Elle ne garantit ni que tous

les rejets seront observés, ni que l'équipe saura immédiatement leur donner le bon sens. La précision du vocabulaire protège ici la précision de la décision.

La preuve peut aussi révéler une inconnue. Elle peut montrer qu'un comportement n'est pas observable, qu'une contrainte doit être reformulée ou qu'une information manque encore. Dans ce cas, elle a évité que l'équipe prenne une décision sur une confiance injustifiée.

Une preuve peut aussi confirmer quelque chose de moins ambitieux que l'équipe ne l'espérait. Un test de contrat peut démontrer que le consommateur strict rejette le champ supplémentaire. Il ne démontre pas la meilleure architecture de coexistence. Une simulation de rejet peut démontrer que l'arrêt est déclenché. Elle ne démontre pas que tous les incidents futurs seront correctement diagnostiqués. Cette modestie est une force : elle empêche de faire porter à un résultat local une conclusion générale qu'il ne contient pas.

Une relance autorisée par ce qu'elle démontre

Le cas des imports rend cette logique concrète. L'intention était d'aider l'opérateur à rétablir une situation. La contrainte était de ne pas réappliquer des lignes déjà traitées. Sur un lot construit où deux lignes ont été appliquées et une troisième a échoué, l'équipe vérifie qu'une relance conserve les deux premières et traite la troisième une seule fois.

Sans cette preuve sur l'état partiel, afficher un bouton de relance ne suffit pas à autoriser son usage. Avec elle, dans le périmètre étudié, la relance peut devenir acceptable. Le test ne couvre pas tout système d'import. Il établit la propriété qui rend cette décision possible.

L'équipe doit aussi prévoir ce qu'elle fera si le résultat contredit l'attente. Si une ligne déjà appliquée est rejouée, le bon effet de la preuve n'est pas de produire un rapport de test supplémentaire. C'est d'empêcher l'ouverture de la relance, de conserver l'état utile à l'enquête et de reprendre la solution avant qu'elle ne soit proposée à l'opérateur. Une preuve acquiert sa portée décisionnelle parce qu'elle est reliée à une action possible.

Une observation n'est pas une permission rétroactive

Après une diffusion bornée, l'équipe peut constater que les premiers événements ont été reçus et traités. Cette observation est précieuse. Elle ne transforme pas l'émission initiale en décision sans incertitude. Le périmètre limité avait précisément pour fonction de contenir ce que l'équipe ne savait pas encore.

Cette différence compte lorsque les signaux sont incomplets. Une absence d'alerte peut signifier qu'aucun rejet n'a eu lieu, mais aussi qu'un rejet n'est pas observable. Une réception technique peut montrer que l'événement est arrivé, sans démontrer que le partenaire a pris la bonne décision métier. L'équipe doit donc nommer ce que chaque signal rend crédible, et ce qu'il laisse encore ouvert, avant d'élargir la diffusion.

Une observation obtenue après propagation aide à apprendre. Une preuve obtenue avant propagation aide à décider. Les deux sont nécessaires, mais elles ne jouent pas le même rôle.

Proportionner la preuve

La proportion ne se compte pas en nombre de tests, de revues ou de signatures. Elle dépend des conséquences d'une erreur, de sa détectabilité, de l'étendue de sa propagation, de la réversibilité pratique du changement et de ce qui reste incertain.

Une modification locale, observable et facile à annuler n'appelle pas le même effort qu'un changement distribué dont le défaut peut rester invisible chez plusieurs partenaires. Cette proportion ne rend jamais une exigence impérative facultative. Elle aide à rapprocher la bonne preuve du point où l'erreur deviendrait coûteuse.

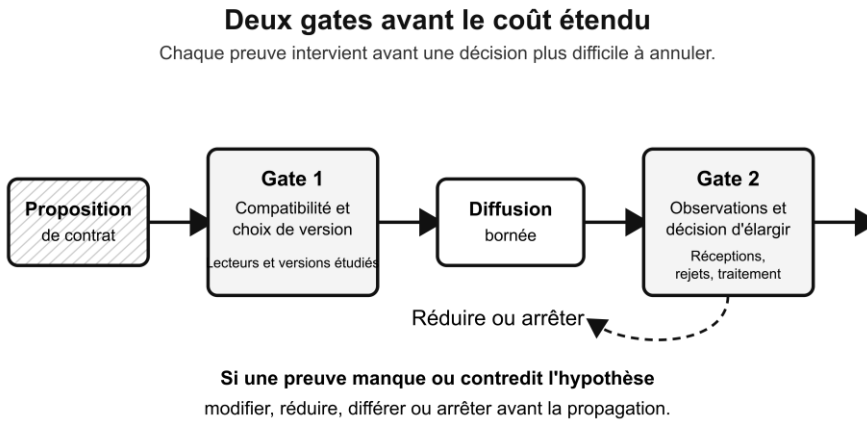


Figure 7.3 - Cadre proposé par le livre. Les preuves éclairent deux décisions distinctes : accepter un contrat, puis élargir une diffusion.

Ce qu'un gate ne promet pas

Un gate ne promet pas qu'une décision est devenue infaillible. Il ne transforme pas non plus une incertitude en problème résolu parce qu'un document porte le mot « validé ». Dans le cas de l'événement, l'équipe peut démontrer la lecture de consommateurs précis, sur des versions précises, avec des exemples représentatifs. Elle ne démontre pas les usages qu'elle n'a pas identifiés, les évolutions futures des partenaires ni les effets qui dépendront d'un autre contrat.

Cette limite ne retire pas sa valeur à la preuve. Elle définit ce que la décision peut honnêtement engager. Si l'équipe sait que le consommateur strict a rejeté le nouveau champ, elle a appris quelque chose qui doit modifier le contrat ou son périmètre. Si elle sait que le nouveau consommateur maintient l'absence de `serviceLevel` comme inconnue, elle évite de faire passer une information manquante pour une catégorie par défaut. Ces résultats sont déjà suffisants pour empêcher une propagation erronée dans le cas étudié.

La tentation inverse consiste à demander une preuve de tout avant toute action. Elle peut transformer la prudence en immobilité et masquer le fait qu'une partie du risque est faible, observable ou réversible. Une diffusion bornée est précisément une manière de

poursuivre sans prétendre avoir établi davantage que ce qui est établi. Elle ne remplace pas le premier gate ; elle donne à l'équipe une occasion contrôlée d'apprendre avant de créer un engagement plus étendu.

Lorsque la preuve oblige à changer de question

Une preuve contradictoire ne mène pas toujours à la même correction. Le rejet du consommateur ancien peut indiquer qu'il faut isoler les versions. Il peut aussi révéler que le nouveau champ ne devait pas voyager dans cet événement, qu'un consommateur a un besoin qui n'a pas été compris ou que la propriété cherchée doit être formulée autrement. La bonne réaction n'est pas de contourner le test pour retrouver un pipeline vert. C'est de demander quelle décision l'échec vient de rendre visible.

Le même mécanisme apparaît dans la règle tarifaire. Lorsque le cas de 16 h contredit le code et les tests, l'équipe ne découvre pas seulement une erreur de comparaison. Elle découvre que les exemples qui avaient guidé la génération ne portaient pas la borne décisive. Corriger l'opérateur de comparaison est nécessaire. Conserver le cas frontière comme propriété explicite est ce qui réduit la probabilité de rejouer la même confusion sous une autre forme.

Une validation utile garde donc la trace de ce qui a été réfuté et de la décision qui a changé. Cette trace n'a pas besoin de devenir une lourde procédure. Elle peut tenir dans un scénario, une revue ou une note de décision. Elle évite que le résultat d'une preuve disparaisse dès que le correctif a été fusionné.

Continuer, réduire ou différer

Une preuve défavorable n'ordonne pas toujours l'arrêt complet. Elle oblige d'abord à identifier la partie de la décision qu'elle contredit. Dans le scénario, le rejet du consommateur ancien ne prouve pas que le nouveau partenaire n'a pas besoin de `serviceLevel`. Il montre que l'équipe ne peut pas diffuser ce champ sur le contrat existant comme si tous les lecteurs avaient la même tolérance.

Plusieurs réponses restent alors ouvertes. L'équipe peut modifier le contrat et rechercher une forme lisible par les consommateurs existants. Elle peut isoler le nouveau besoin dans un flux distinct. Elle peut conserver l'information à l'intérieur de son système, réduire le périmètre de la première livraison ou différer l'évolution jusqu'à ce qu'une connaissance supplémentaire soit disponible. Ces réponses n'ont pas le même coût, ni le même délai, ni la même capacité à préserver le besoin du partenaire. La preuve ne choisit pas entre elles. Elle rend impossible de continuer comme si aucune différence de lecture n'avait été trouvée.

Cette nuance compte parce qu'elle évite une caricature de la validation. Valider n'est pas chercher un motif pour bloquer une production. C'est rechercher suffisamment tôt les informations qui permettent de choisir une forme de progression assumable. Une équipe qui réduit un périmètre après un test contradictoire peut conserver davantage de valeur qu'une équipe qui livre tout puis découvre le rejet après une propagation large.

La décision de différer demande la même honnêteté. Elle peut être justifiée lorsqu'un contrat manque, lorsqu'un consommateur n'est pas identifiable ou lorsqu'une exigence ne peut pas être respectée dans le mandat disponible. Elle ne devient pas une réponse satisfaisante parce qu'elle évite un risque. Il faut déclarer ce qui est différé, la condition qui permettrait de reprendre et ce qui arrive au besoin entre-temps. Sinon, l'incertitude est simplement déplacée hors du changement en cours.

La réversibilité règle le moment, pas l'exigence

Une erreur détectée dans un environnement local est généralement plus facile à corriger qu'une erreur publiée dans un contrat consommé par plusieurs systèmes. Ce constat n'implique pas que tout changement distribué doit être traité comme irréversible. Il invite à demander ce qui pourra réellement être défait si la preuve survient tard.

Dans l'événement `RequestAccepted`, arrêter une émission future peut empêcher de produire d'autres messages incompatibles. Cela ne corrige pas les messages déjà rejetés, ni les décisions métier que des

partenaires auraient prises à partir d'une interprétation erronée. Conserver les événements en attente crée une possibilité de reprise, mais ne garantit pas que chaque conséquence externe sera annulable. La réversibilité est ici un ensemble de capacités concrètes : arrêter, retrouver, expliquer, corriger et, lorsque c'est possible, rejouer sans créer une seconde erreur.

Cette analyse aide à placer les preuves. Lorsque le retour arrière est facile et que l'effet est observable, l'équipe peut apprendre plus tôt par une diffusion limitée. Lorsque l'information se propage vers des lecteurs inconnus ou engage une donnée difficile à corriger, elle a de bonnes raisons de demander davantage avant la première émission. Dans les deux cas, une exigence impérative garde son sens. Le caractère réversible d'un changement ne rend pas acceptable ce qui ne l'est pas.

Le coût d'une preuve tardive n'est pas seulement le temps de correction. C'est aussi le temps de reconstituer l'état du système au moment du défaut, d'expliquer un comportement aux partenaires, de distinguer les événements à reprendre de ceux qui ont été traités et de décider si une réparation peut créer une seconde incohérence. Ces tâches sont parfois nécessaires. Les reconnaître à l'avance aide l'équipe à choisir la preuve qui peut les éviter, sans lui demander d'anticiper chaque incident possible.

Cette perspective rend également la revue plus concrète. Un relecteur ne cherche pas seulement une erreur dans la proposition. Il peut demander quelle conséquence deviendrait difficile à réparer une fois l'événement reçu, quel lecteur a été réellement exercé et comment l'équipe saura qu'un arrêt de diffusion a conservé les éléments utiles à une reprise. Ces questions lient le changement de code à la décision de propagation, là où une simple lecture de différence resterait insuffisante.

La preuve appelle une interprétation

Un résultat ne porte pas son propre mandat. Le test qui montre le rejet du consommateur ancien peut être clair et pourtant laisser ouvertes

plusieurs questions : le flux distinct crée-t-il un coût acceptable ? Le nouveau partenaire peut-il attendre ? Une compatibilité est-elle impérative pour ce contrat ou un périmètre réduit est-il permis ? Ces questions dépassent la seule exécution de la preuve.

La personne qui a produit le changement peut expliquer la proposition. Une personne connaissant le consommateur peut interpréter ce que le rejet signifie. Une autre peut porter la conséquence métier d'un délai ou d'un périmètre réduit. La validation rend leur décision plus informée ; elle ne rend pas leur responsabilité superflue. C'est pourquoi la dernière étape de ce chapitre ne consiste pas à ajouter une signature automatique après chaque test vert.

Le chapitre suivant examinera ces fonctions sans les figer dans un organigramme. Pour l'instant, retenons que l'équipe doit pouvoir distinguer quatre gestes : produire une preuve, comprendre sa portée, décider de la suite et accepter ou escalader ce qui reste incertain. Les confondre revient à faire dire à un outil de validation plus qu'il ne peut réellement établir.

Avant la diffusion, après la diffusion

Avant la diffusion, la preuve cherche à protéger une décision qui peut encore être modifiée sans exposer largement le système. Après la diffusion, l'observation cherche à apprendre de ce qui s'est réellement produit et à contenir ce qui reste réversible. Les deux temps demandent une attention différente.

L'équipe peut, avant l'émission, vérifier que l'alerte sera déclenchée par un rejet représentatif. Après l'émission, elle peut observer que des événements ont été reçus et traités. Entre ces deux affirmations, il existe une différence essentielle : la première porte sur la capacité à voir un problème ; la seconde porte sur un comportement effectivement survenu. Les confondre peut conduire à étendre un flux parce que l'outillage paraît prêt, alors que le traitement métier n'a pas encore été observé.

Cette séparation allège également la revue. Au lieu de demander si « tout est validé », les personnes concernées peuvent poser des

questions plus utilisables : qu'avons-nous établi avant d'émettre ? Qu'allons-nous observer pendant le périmètre borné ? Quel résultat suspendrait l'élargissement ? Ces questions préparent le chapitre suivant, car elles révèlent qu'une preuve ne dit pas d'elle-même qui peut accepter l'incertitude restante.

La vitesse de production peut faire croire que le changement est achevé lorsque l'exemple local fonctionne. Ce qui compte est pourtant de savoir ce qui doit rester vrai là où ses conséquences apparaîtront.

> **Dans une pratique SDD** `spec.md` donne aux tests, revues et démonstrations des critères à exercer. `checklist` explicite les contrôles attendus et `analyze` cherche les écarts entre `spec.md`, `plan.md` et `tasks.md` avant que la diffusion n'en élargisse les conséquences.

Valider avant la propagation, c'est donner à une preuve la possibilité de modifier encore la décision. Une preuve ne décide pas seule. Il reste à rendre visible qui peut l'interpréter, accepter un risque résiduel ou demander une escalade.

Sources et portée

Sources et portée

confluent. Confluent, *Schema Evolution and Compatibility for Schema Registry on Confluent Platform*, 9 septembre 2026, vérifié le 19 septembre 2026. La documentation décrit des mécanismes dépendant des formats, lecteurs et versions ; elle n'impose ni la solution ni les gates du scénario construit.

Chapitre 8 - Organiser les responsabilités

Une preuve peut arrêter une mauvaise idée. Elle ne peut pas, à elle seule, accepter ce qui reste incertain. Le chapitre précédent a montré comment placer une preuve avant que le coût d'une erreur ne s'étende. Il reste une question moins confortable, parce qu'elle ne se résout ni par un test supplémentaire ni par une automatisation du pipeline : *qui est en droit de donner un sens à cette preuve et d'accepter ce qu'elle ne couvre pas ?*

Cette question devient plus fréquente lorsque la production de code s'accélère. Une proposition paraît cohérente, les scénarios automatisés passent, la revue technique ne trouve pas de défaut évident. Il est alors tentant de déduire que le changement est prêt. Mais « prêt » rassemble souvent plusieurs décisions différentes : le mécanisme fonctionne-t-il dans les cas étudiés ? Son comportement est-il acceptable pour les personnes qui le subiront ? Qui peut autoriser son périmètre initial ? Qui doit être averti lorsque la réponse manque ?

Ces questions ne demandent pas une bureaucratie plus dense. Elles demandent que l'autorité cesse d'être implicite. Confier la production à un agent IA, à une équipe plateforme ou à un développeur expérimenté ne délègue pas automatiquement le jugement sur les conséquences du résultat. Une organisation peut rester rapide si elle sait quelles décisions sont locales, quelles connaissances doivent être réunies et à quel moment une décision doit changer de périmètre.

Le scénario de ce chapitre est construit. Il ne propose pas une politique de notification à copier. Il sert à rendre visible un point souvent masqué par la formule « le code a été validé ».

Une relance techniquement réussie peut rester une décision incomplète

Une équipe plateforme maintient un composant partagé qui envoie des notifications pour plusieurs produits. Les destinataires reçoivent par exemple un message de suivi, une confirmation de paiement ou la modification d'un rendez-vous. L'équipe observe des échecs

intermittents. Dans certains cas, le délai d'attente est dépassé alors que le fournisseur a peut-être déjà remis le message. Le composant connaît un résultat incertain : il ne peut pas affirmer avec certitude que le destinataire n'a rien reçu.

Un agent IA aide à proposer une évolution. Il conserve davantage de contexte, classe les échecs et relance automatiquement certains envois. Les tests de simulation deviennent plus satisfaisants : lorsqu'une première tentative échoue de façon nette, une tentative ultérieure délivre le message dans plusieurs situations étudiées. La revue confirme que l'état est conservé et que les tentatives ne dépassent pas la limite définie par le composant.

Une conclusion paraît disponible : il faut activer la relance. Pourtant, cette phrase mélange le fonctionnement d'un mécanisme et le sens du message pour son destinataire. Si un accusé de réception a été perdu, une relance peut produire un doublon. Pour une notification qui rappelle qu'un dossier attend une réponse, ce doublon peut être simplement gênant. Pour une confirmation de paiement, il peut amener une personne à croire qu'une nouvelle opération a eu lieu. Pour un rendez-vous, un message en retard peut déclencher une action devenue inappropriée.

Le paiement lui-même n'est pas rejoué dans ce scénario. Le risque ne porte pas sur l'exécution financière, mais sur l'information reçue et sur les décisions qu'elle provoque. Cette distinction est importante. Elle empêche de dramatiser un mécanisme de notification tout en conservant la réalité du compromis : absence de message, retard du message ou répétition du message ne se valent pas pour tous les produits.

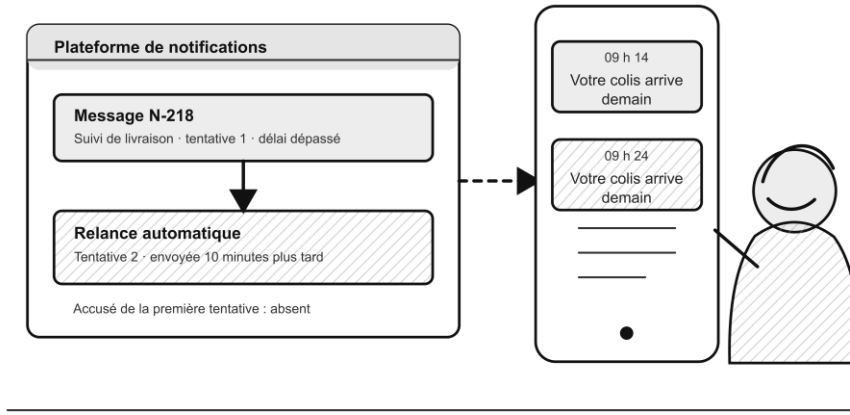


Figure 8.1 - Scénario construit. Une relance automatique peut délivrer le message attendu et produire un doublon visible par son destinataire.

La documentation du *Retry pattern* décrit pourquoi un système distribué peut rencontrer des résultats incertains et pourquoi une relance doit tenir compte de l'idempotence, des délais et des effets de bord.[^retry] Elle ne dit pas que deux tentatives pendant dix minutes sont un bon réglage. Ces nombres appartiennent au scénario construit. Ils donnent une forme précise à une décision dont l'organisation doit accepter le coût.

Affirmation	**Ce que la preuve technique peut établir**	**Ce qu'elle ne peut pas décider**
La relance est exécutée	Les tentatives respectent le nombre et la fenêtre prévus.	Le coût d'un doublon pour un produit.
L'accusé peut être perdu	La simulation produit un état où le message a pu être reçu.	La tolérance métier à cette ambiguïté.
Les échecs simulés diminuent	Le mécanisme récupère certains cas étudiés.	La classe de notifications où l'activer.
Le journal contient une trace	L'équipe peut retrouver une tentative.	La personne qui accepte le risque résiduel.

Le tableau ne diminue pas la valeur des tests. Il leur rend une portée utile. Les tests répondent à la question posée sur le composant. Une décision utile commence précisément lorsque nous refusons de leur faire répondre à une autre question par défaut.

Une politique fictive, avec un coût réel dans le scénario

L'équipe ne choisit pas une politique uniforme. Pour les notifications de suivi, les responsables produit concernés indiquent qu'un doublon limité est préférable à une absence silencieuse. Ils acceptent, dans le périmètre construit ici, au plus deux nouvelles tentatives durant les dix minutes qui suivent l'événement. Si cette fenêtre est dépassée, la notification est marquée non délivrée et signalée pour traitement.

Cette politique conserve un bénéfice : certains messages qui auraient été perdus sont réémis sans qu'un opérateur doive intervenir à chaque fois. Elle conserve aussi un coût. Une personne peut voir deux notifications de suivi. Des notifications restent non relancées après la fenêtre. Un traitement doit exister pour comprendre les échecs persistants. L'équipe ne prétend pas avoir supprimé l'incertitude, elle a choisi une manière bornée de la traiter.

Les confirmations de paiement et les modifications de rendez-vous restent exclues. Pour ces classes sensibles, il faudrait pouvoir établir des conditions supplémentaires : déduplication adaptée au produit, ordre pertinent des messages, expiration et conséquence d'un retard. Le canal existant et son traitement actuel des échecs sont donc conservés. Cette exclusion n'est pas une punition de l'équipe plateforme. C'est le prix accepté pour ne pas faire passer une connaissance manquante pour une tolérance acquise.

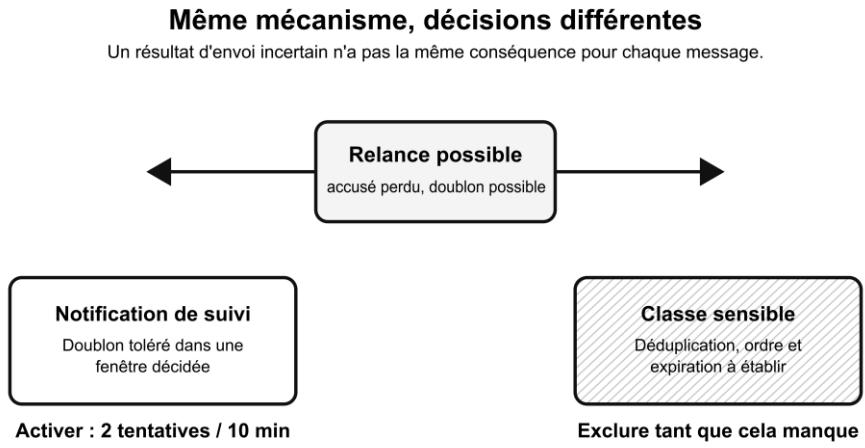


Figure 8.2 - Scénario construit. Une même capacité de relance peut être activée pour une classe où son compromis est accepté et rester exclue d'une classe où les conditions ne sont pas établies.

Une politique commune devient parfois désirable parce qu'elle simplifie la maintenance. Cette simplicité mérite d'être recherchée, mais elle ne transforme pas des conséquences différentes en conséquences identiques. Une plateforme peut définir le mécanisme commun, ses limites et ses traces. Elle ne possède pas, par ce seul fait, le mandat d'accepter le retard ou la répétition pour tous les produits qui utilisent le mécanisme.

Quelle information ferait changer la décision d'activer la relance pour les messages de suivi ? Une simulation qui montrerait un doublon persistant au-delà de la fenêtre, une information produit qui révélerait une conséquence non tolérable, ou une contrainte de conservation qui interdirait la trace prévue pourraient suffire. La réponse n'est pas « un test vert de plus ». Elle dépend de la propriété qui doit rester vraie et de la personne capable d'en évaluer la portée.

Quatre fonctions autour d'un même choix

Pour rendre cette décision praticable, il est utile de distinguer quatre fonctions. Ce ne sont pas quatre rôles fixes, ni quatre cases d'un organigramme. Dans une petite équipe, une même personne peut

exercer plusieurs de ces fonctions. Dans une organisation plus vaste, elles peuvent être réparties entre plusieurs personnes. La distinction sert à éviter qu'une contribution indispensable disparaisse sous l'étiquette vague de « validation ».

Formuler la décision

Avant de chercher une approbation, il faut nommer le choix réel. Dans le scénario, la décision n'est pas simplement « ajouter des tentatives ». Elle porte sur le compromis entre absence, retard et doublon d'une notification, pour des classes déterminées. Cette formulation rend visible que la solution technique cache une conséquence produit.

Nommer la décision a un effet immédiat sur la revue. Au lieu de demander seulement si l'algorithme de relance est correct, les participants peuvent demander si la classe concernée tolère le résultat incertain, si le délai proposé change la valeur du message et si l'exclusion d'une classe sensible est maintenue. La même différence de code reçoit alors les questions que son impact exige réellement.

Apporter la connaissance nécessaire

Une décision ne peut pas être plus solide que les connaissances dont elle dépend. La plateforme connaît les limites du fournisseur, le format de la trace et le comportement de l'état de relance. Les responsables produit connaissent les conséquences d'un message répété, tardif ou absent. Une personne d'exploitation peut connaître la capacité à traiter les échecs marqués. Chacun apporte une partie de la situation, sans que cette contribution soit nécessairement une approbation.

L'absence de connaissance n'est pas un défaut moral. Elle devient dangereuse lorsqu'elle est masquée par une réponse technique qui semble complète. Dire « nous ne savons pas encore si un doublon de ce message est acceptable » est une information exploitable. Cela peut conduire à exclure une classe, à réduire un périmètre ou à organiser une recherche. Prétendre que le mécanisme est générique pour éviter cette conversation crée une décision implicite.

Interpréter la preuve

Une preuve n'est pas auto-interprétable. Un test peut montrer qu'un accusé perdu entraîne une seconde émission. Pour comprendre ce que ce résultat impose, il faut savoir quelle propriété était recherchée. Le test réfute peut-être l'idée que le composant garantit une livraison exactement une fois. Il ne dit pas, sans interprétation, si le compromis est acceptable pour la notification de suivi ou interdit pour une confirmation.

Interpréter ne consiste pas à rendre la preuve subjective. Cela consiste à relier un résultat à la question précise qui justifiait son existence. La plateforme peut expliquer ce que la simulation couvre et ce qu'elle ne couvre pas. Le produit peut traduire le doublon en conséquence métier. La revue peut constater qu'une condition nécessaire manque. Chaque geste est plus clair lorsque la preuve garde une portée déclarée.

Accepter ou escalader

Enfin, une personne ou une autorité doit pouvoir accepter le risque qui demeure dans son mandat, ou reconnaître que la décision le dépasse. Accepter ne signifie pas dire que le risque a disparu. Cela signifie que, avec les connaissances et preuves disponibles, le compromis est assumé pour un périmètre explicite.

L'escalade intervient lorsque ce mandat manque, lorsqu'une exigence impérative n'est pas démontrée, lorsque plusieurs autorités compétentes divergent ou lorsque l'effet franchit un périmètre plus large que celui de la décision locale. Elle ne mesure ni la qualité d'un développeur ni la maturité d'une équipe. Elle empêche une décision de devenir irréversible parce que personne n'a osé dire qu'elle dépassait son autorité.

Fonction	Question rendue visible dans le scénario	Exemple de contribution
Formuler	Quel compromis choisissons-nous réellement ?	Distinguer absence, retard et doublon.
Connaître	Que savons-nous des conséquences par	Signaler qu'un rendez-vous tardif peut être

Fonction	Question rendue visible dans le scénario	Exemple de contribution
	produit ?	trompeur.
Interpréter	Que prouve exactement le rejeu d'un accusé perdu ?	Constater qu'il établit un doublon possible, pas sa tolérance.
Accepter ou escalader	Qui peut limiter l'activation ou la faire remonter ?	Autoriser le suivi, maintenir les classes sensibles exclues.

Ces fonctions ne retirent pas la responsabilité à la personne qui produit le changement. Elle reste responsable d'expliquer ce qu'elle propose, ses limites et les preuves disponibles. Elles empêchent seulement de lui attribuer, silencieusement, une autorité produit ou de service qu'elle ne possède peut-être pas.

Des mandats lisibles sans hiérarchie universelle

Dans le scénario, le mandat plateforme porte sur le mécanisme partagé : nombre maximal de tentatives, conservation de l'état, signalement des échecs et capacités de simulation. Il permet de dire ce que le composant sait faire, de demander des preuves sur son fonctionnement et d'arrêter une activation techniquement dangereuse.

Le mandat produit porte sur les conséquences d'une classe de notifications. Il permet de dire que le doublon de suivi est acceptable dans une fenêtre donnée, ou au contraire qu'un message ne doit pas être relancé tant que certaines conditions ne sont pas établies. Ce mandat n'autorise pas à modifier seul le comportement commun de tous les produits.

Une autorité de service désignée intervient si le périmètre commun est contesté. Elle peut arbitrer entre les produits et la plateforme dans les limites qui lui sont confiées. Elle ne peut pas déclarer satisfaite une exigence impérative non démontrée. Elle ne peut pas non plus remplacer par une signature la connaissance que les responsables produit doivent fournir sur leurs propres conséquences.

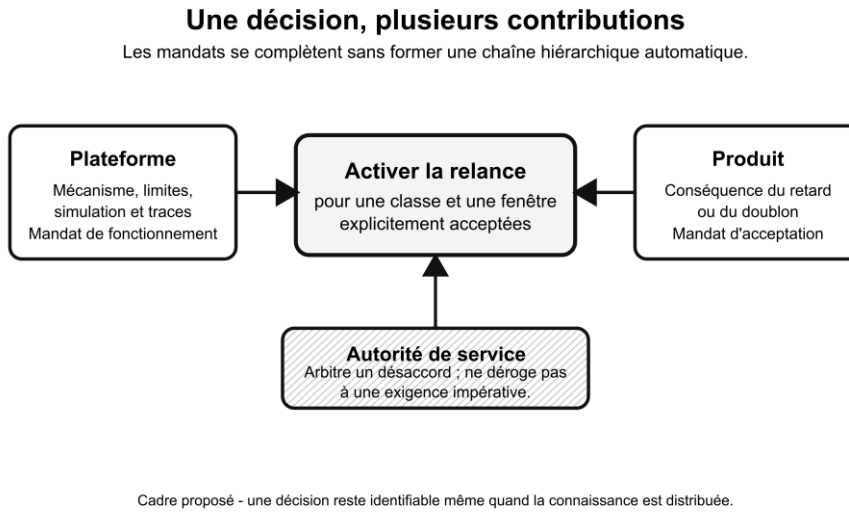


Figure 8.3 - Cadre proposé par le livre. Les contributions convergent vers une décision identifiable ; elles ne dessinent pas une chaîne hiérarchique obligatoire.

Ce dessin n'est pas un RACI réduit. Un RACI attribue utilement certaines activités, mais il peut donner l'impression qu'une lettre suffit à résoudre un désaccord sur le risque. Ici, la question est plus étroite et plus concrète : qui peut apporter la connaissance, interpréter une preuve et accepter l'activation dans ce périmètre ? La réponse dépend de l'organisation, du produit et du mandat existant.

La clarté n'exige pas que chaque décision passe par un comité. Pour une classe de suivi connue, avec une tolérance déjà explicite et un mécanisme déjà éprouvé, l'équipe peut décider rapidement. Pour une classe sensible, la même vitesse serait une fuite en avant si les conditions manquent. Organiser les responsabilités consiste justement à préserver cette différence au lieu de traiter toute modification comme une exception ou toute modification comme une routine.

Ce que la responsabilité n'est pas

La responsabilité utile n'est pas une signature administrative. Ajouter un approbateur à la fin d'un flux sans lui présenter la décision, les connaissances et les preuves ne crée qu'une étape supplémentaire. Cette personne peut être tenue de répondre d'un choix qu'elle n'a pas

eu les moyens de comprendre. L'équipe peut alors croire qu'elle a sécurisé la décision alors qu'elle a seulement déplacé son opacité.

Elle n'est pas non plus une délégation vers un agent IA. Un agent IA peut rassembler les échecs, proposer un tableau de conséquences, écrire des scénarios ou signaler une incohérence. Ces capacités sont précieuses. Elles ne lui donnent pas la possibilité d'assumer une perte de confiance client, une contrainte contractuelle ou une décision d'allocation de ressources. L'agent peut rendre le jugement mieux informé ; il ne porte pas les conséquences à la place de l'organisation.

Enfin, elle n'est pas la centralisation de toutes les décisions. Une équipe qui doit faire approuver chaque préférence locale par une direction lointaine perd sa capacité de répondre. À l'inverse, une équipe qui traite comme local tout changement parce qu'il tient dans une petite différence de code expose des conséquences qui ne sont plus locales. Le bon périmètre ne se déduit ni du volume de fichiers modifiés ni du fait qu'une IA a participé à leur production.

Une décision peut rester locale lorsque ses conséquences, ses contraintes et sa réversibilité sont comprises dans le mandat de l'équipe. Elle doit devenir plus visible lorsque plusieurs produits sont concernés, lorsqu'une information manque, lorsqu'un retour arrière est difficile ou lorsqu'une exigence importante ne peut pas être établie. Ces signaux ne composent pas un algorithme. Ils aident à reconnaître que la question a changé de nature.

Faire de l'escalade un geste possible

Une escalade praticable demande plus qu'une adresse de messagerie ou une règle générale. Il faut savoir ce qui est escaladé. Dans le scénario, ce pourrait être la demande d'ajouter les confirmations de paiement à la relance alors que les preuves de déduplication ne sont pas disponibles. La question formulée est alors : pouvons-nous accepter un doublon ou un retard pour cette classe, et qui peut accepter cette conséquence ?

Il faut aussi laisser une option sûre pendant l'attente. La classe contestée reste exclue, le comportement existant est maintenu et les

échecs sont visibles. Sans cette possibilité, l'équipe peut se sentir contrainte d'activer trop tôt pour tenir une date ou de bloquer l'ensemble du mécanisme alors qu'un périmètre réduit demeure utile.

L'escalade devient également nécessaire lorsqu'une exigence impérative est contredite. Si une règle impose que certains messages ne soient jamais remis après une date donnée, le fait qu'une autorité souhaite tout de même les relancer ne crée pas une dérogation. L'équipe doit d'abord établir si une exception existe réellement, qui peut l'autoriser et comment elle sera démontrée. Proportionner l'effort ne permet pas de négocier ce qui ne peut pas être négocié.

Une trace légère peut suffire : la classe concernée, la décision à prendre, la connaissance manquante, la preuve disponible, l'autorité sollicitée et la condition de reprise. Cette trace n'est pas une fin administrative. Elle évite que, quelques semaines plus tard, une activation soit interprétée comme la preuve qu'un compromis a été accepté partout.

Situation	**Réponse qui conserve la décision ouverte**	**Réponse qui la masque**
Le suivi tolère un doublon borné	Activer cette classe avec la fenêtre décidée.	Affirmer que toutes les notifications tolèrent la relance.
Une classe sensible manque de preuves	La conserver hors périmètre et noter la condition de reprise.	La traiter comme identique pour simplifier le déploiement.
Deux mandats divergent	Saisir l'autorité de service et conserver l'exclusion.	Choisir le dernier avis reçu sans décision identifiable.
Une exigence impérative est en doute	Vérifier le mandat et la preuve avant toute activation.	Présenter une exception comme un réglage technique.

La décision répartie ne dissout donc pas la responsabilité. Plusieurs personnes peuvent contribuer à une même compréhension, mais l'acceptation doit conserver une autorité identifiable. C'est ce qui permet de répondre à une question ultérieure : pourquoi cette classe a-t-elle été activée, sur quelles preuves, avec quelles limites et par qui ?

Après l'activation, la responsabilité continue

Le moment de l'activation n'est pas une frontière après laquelle la décision disparaît. Si la plateforme observe un volume inattendu de doublons, si un produit modifie la nature de ses messages ou si le fournisseur change son comportement, le compromis doit être réexaminé. La responsabilité inclut la capacité de reconnaître que les conditions d'acceptation ne sont plus les mêmes.

Cela n'oblige pas à rouvrir toute politique à chaque événement. Il faut distinguer l'observation qui confirme que le périmètre reste celui qui était attendu de l'information qui le transforme. Une notification de suivi réellement reçue deux fois dans la fenêtre assumée n'invalide pas nécessairement la décision. Une répétition après expiration, une confusion produit récurrente ou l'ajout d'une classe nouvelle peuvent, eux, exiger une nouvelle interprétation.

Cette continuité donne un sens plus précis à la traçabilité. Une trace n'est pas seulement utile pour diagnostiquer un défaut. Elle relie un comportement observé à la décision qui l'avait autorisé. Sans ce lien, l'organisation conserve des journaux mais perd la possibilité de réexaminer les hypothèses et mandats qui avaient rendu le comportement acceptable.

La responsabilité ne promet pas l'absence d'erreur. Une décision peut être raisonnable au regard des informations disponibles et devoir être corrigée ensuite. Ce que l'explicitation conserve, c'est la possibilité de comprendre ce qui a été accepté, de réduire le périmètre, de réparer une conséquence et d'apprendre sans réécrire l'histoire comme si le choix avait été évident.

Décider au bon endroit plutôt que plus haut

Il existe une réponse commode à l'incertitude : faire remonter toute décision. Elle paraît rassurante parce qu'elle ajoute une autorité visible. Elle produit pourtant un autre problème lorsque l'autorité sollicitée ne connaît ni le produit, ni la contrainte, ni la preuve en cause. La décision est alors centralisée sans devenir mieux informée. L'équipe attend une

réponse qui ne peut être que générale, puis interprète cette généralité à la place de la personne qui aurait dû porter le compromis.

Le mouvement inverse est tout aussi fragile. Une équipe plateforme peut avoir raison de vouloir éviter une exception par produit. Si elle active la relance pour toutes les classes au nom de cette simplicité, elle convertit une préférence d'exploitation en acceptation métier généralisée. La simplification est réelle ; elle ne suffit pas à justifier la conséquence. Le bon emplacement de la décision n'est donc ni nécessairement le plus proche du code ni nécessairement le plus haut dans l'organisation. C'est celui où se rencontrent le mandat et la connaissance des effets.

Dans une petite structure, cette rencontre peut avoir lieu dans une conversation courte. La même personne peut concevoir le mécanisme, connaître le produit et accepter le compromis. Cela ne supprime pas les fonctions décrites plus haut ; cela les rend simplement plus concentrées. La décision gagne à être formulée parce qu'elle pourra être relue plus tard, ou parce qu'un changement de périmètre pourra la rendre insuffisante.

Dans une structure plus distribuée, la même décision peut demander plusieurs échanges. La plateforme ne doit pas attendre une formule parfaite avant de présenter ses limites. Le produit ne doit pas attendre que tous les détails d'implémentation soient clos avant de dire que la duplication est inacceptable. L'autorité de service ne doit pas devenir le seul endroit où une contradiction est découverte. Chacun peut contribuer tôt, à condition que la contribution soit reliée à une décision identifiable plutôt qu'à une approbation vague.

Cette pratique évite aussi le faux confort de la délégation ascendante. Dire « la direction a validé » ne répond pas à la question de savoir quelle classe a été activée, pour quelle durée, avec quel comportement de reprise et quelles conditions de réexamen. Une autorité peut valider une orientation sans remplacer les décisions plus locales qui en donnent la forme. À l'inverse, une décision locale peut déclencher une escalade sans perdre toute autonomie : l'équipe peut maintenir un périmètre sûr, continuer les preuves utiles et demander l'arbitrage seulement sur le point qu'elle ne peut pas accepter.

La forme d'une acceptation utile

Une acceptation n'a pas besoin de devenir un document long. Elle doit toutefois pouvoir répondre à quelques questions concrètes. Quel comportement est autorisé ? Pour quelle classe, quel périmètre et quelle durée ? Quel risque reste connu ? Quelle preuve a permis de comprendre ce risque ? Quel événement obligerait à réexaminer la décision ? Ces éléments forment une mémoire de l'arbitrage, pas une procédure supplémentaire.

Pour les notifications de suivi, une note concise pourrait dire que le produit accepte deux nouvelles tentatives dans les dix minutes, que le doublon est un risque connu, que les échecs persistants sont signalés et que tout ajout d'une classe sensible déclenche un nouvel examen. Cette note ne prouve pas la qualité du mécanisme. Elle évite que son activation soit ensuite lue comme une autorisation illimitée.

La même précision protège l'équipe contre un changement de vocabulaire. Une personne peut appeler « rappel » une notification qui est devenue, en pratique, une confirmation d'action. Une autre peut étendre une fenêtre parce qu'elle cherche à réduire un indicateur d'échec. Si le mandat initial est seulement implicite, ces changements paraissent mineurs. Lorsqu'il décrit la classe, la conséquence et le seuil d'acceptation, ils redeviennent ce qu'ils sont : de nouvelles décisions.

Une acceptation utile n'est donc ni une permission générale ni une approbation éternelle. Elle est un engagement à portée déterminée. Elle ne ferme pas l'apprentissage ; elle dit qui doit être sollicité lorsque ce qui a été accepté cesse de décrire le comportement réel. Cette limite rend la responsabilité compatible avec une évolution continue au lieu de la réserver aux grandes décisions rares.

> **Dans une pratique SDD** `converge` peut signaler un écart entre `spec.md`, `plan.md`, `tasks.md` et le code. Le mandat reste un objet distinct : l'acceptation indique le comportement concerné, son périmètre, les preuves consultées et la condition d'escalade.

Les preuves du chapitre précédent répondent à « qu'avons-nous établi avant que l'erreur ne se propage ? ». Ce chapitre ajoute « qui peut interpréter cette réponse et accepter ce qu'elle laisse ouvert ? ». La réponse ne prend pas la forme d'un organigramme universel. Elle prend la forme de mandats lisibles, de connaissances mobilisées et d'une escalade possible avant qu'une production rapide ne se transforme en engagement silencieux.

Le chapitre suivant mettra cette grammaire sous tension dans quatre équipes. Elles n'aboutiront pas à la même décision. Leur point commun sera plus utile : chacune devra rendre visibles l'intention, les limites, la connaissance, les preuves et l'autorité qui accepte ou refuse de poursuivre.

Sources et portée

Sources et portée

retry. Microsoft, *Retry pattern*, consulté le 19 septembre 2026. La source documente les raisons et précautions générales des tentatives répétées dans les systèmes distribués. Le délai, le nombre de tentatives, les classes de notifications et les mandats de ce chapitre sont des éléments du scénario construit.

Interlude pratique - Un dossier SDD minimal

Les chapitres précédents ont nommé les objets qu'une équipe doit garder visibles : intention, contraintes, connaissance, spécification, preuves et responsabilités. Les encarts ont également donné des noms à certains artefacts employés dans des pratiques SDD : `constitution.md`, `spec.md`, `plan.md` et `tasks.md`. Il reste à les regarder ensemble.

Cet interlude ne propose ni un modèle de fichiers à copier ni une procédure de déploiement. Il ne montre aucune commande. Son unique objectif est plus simple : donner une forme concrète à un dossier SDD afin que le lecteur voie ce que chaque document peut contenir, ce qu'il reçoit du précédent et ce qu'il ne peut pas décider seul.

Le cas est construit. Il reprend le scénario tarifaire du chapitre 6. Une livraison standard coûte fictivement 100 €. Avant le début de sa préparation, le client peut demander une **option de livraison urgente** facturée 20 % du prix standard. Dans cet exemple seulement, les 20 % de 100 € représentent 20 € et portent le prix total à 120 €. La règle porte donc sur un pourcentage, pas sur un supplément fixe. Cette option promet un délai de livraison raccourci ; elle ne désigne ni le devis complet ni un service d'urgence immédiat. Elle n'est proposée que si la demande est reçue strictement avant 16 h, si un créneau de livraison urgente est confirmé et si la préparation n'a pas commencé.

La règle est connue pour la catégorie d'envoi dite standard. Une seconde catégorie d'envoi, dite spéciale dans cet exemple, existe mais sa règle d'éligibilité n'a pas encore été confirmée. Ces noms servent uniquement à distinguer un périmètre décidé d'un périmètre encore ouvert. Le dossier ne doit pas faire disparaître cette absence de connaissance ; c'est précisément ce qu'il doit rendre visible.

Dans une boîte à outils comme Spec Kit, une constitution peut guider le projet, puis une spécification, un plan et des tâches sont produits pour une fonctionnalité. Des étapes de clarification, de contrôle et de convergence servent à rapprocher ces artefacts et ce qui est

effectivement réalisé.[^speckit] Le déroulé présenté ici s'en inspire sans prétendre reproduire les modèles ni les conventions de tous les outils.

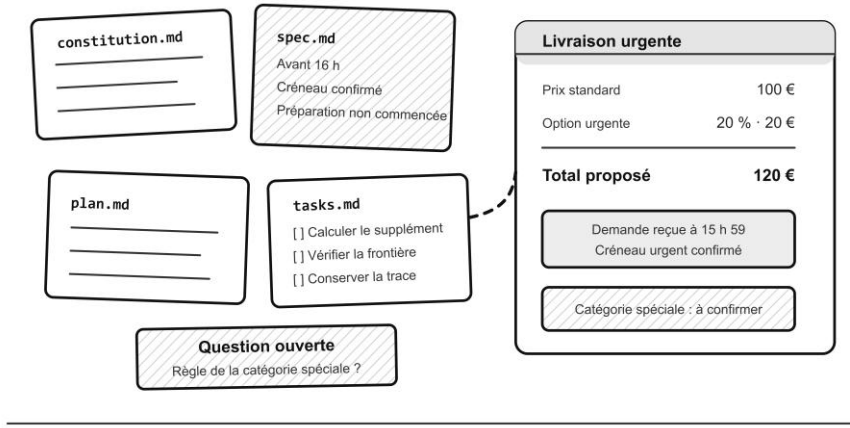


Figure I.1 - Cadre pédagogique. Le dossier SDD relie les artefacts de travail au cas tarifaire sans faire disparaître la règle encore inconnue.

Le cas avant les documents

Une responsable d'exploitation formule une demande apparemment simple : « Proposer l'option de livraison urgente aux clients qui la demandent avant la limite. » Un agent IA peut immédiatement produire une fonction, un écran et quelques tests à partir de cette demande. Mais le dossier ne commence pas avec cette production. Il commence en cherchant ce que l'équipe doit pouvoir décider sans créer de faux engagement.

L'intention retenue est la suivante : permettre à un client éligible de connaître le prix total d'une livraison urgente, sans promettre ce délai raccourci lorsqu'aucun créneau urgent n'est confirmé ou lorsque la préparation est déjà engagée. Cette phrase ne contient pas encore une solution. Elle permet de comprendre pourquoi un créneau inconnu et une demande à 16 h exactement ne doivent pas être traités comme des détails d'interface.

Le dossier qui suit est volontairement court. Un système réel contiendrait sans doute plus de règles, de rôles, de données et de traces. Une équipe plus petite pourrait fusionner certains éléments dans un seul document. Une autre pourrait employer d'autres noms. Ce qui compte ici est la relation entre les éléments, pas leur volume.

Les extraits sont volontairement montrés sous la forme que possède un fichier Markdown avant son affichage par un lecteur. Les # placés devant les titres, les listes, les identifiants et les liens visibles ne sont donc pas des erreurs typographiques : ce sont les caractères qu'un développeur rencontre dans le fichier `.md`.

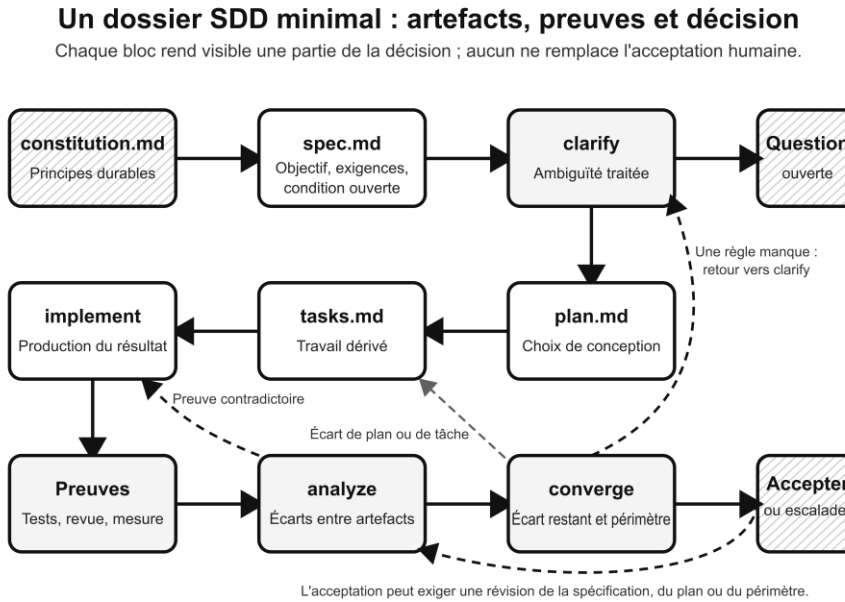


Figure I.2 - Cadre pédagogique. Le dossier ne suit pas une ligne irréversible : une question ouverte, une preuve contradictoire ou une acceptation de périmètre peut ramener l'équipe vers l'artefact qui doit être révisé.

1. `constitution.md` - les principes qui survivent à la fonctionnalité

La constitution ne décrit pas l'option de livraison urgente. Elle fixe les principes qui devraient gouverner plusieurs fonctionnalités du même

projet. C'est le bon endroit pour ce qui ne doit pas être redécidé à chaque changement, tout en restant révisable par l'autorité qui possède le projet.

```
# Constitution du service de livraison

## Socle technique

- Langage : C#.
- Interface : WinUI 3.
- Cible : Windows 11 ; Windows SDK 10.0.26100.0.
- Interface utilisateur : textes en anglais.
- Les commentaires de code et la documentation XML sont rédigés en
anglais.

## Principes

### Principe 1 - Engagement vérifiable

Un prix affiché au client doit correspondre à des conditions dont le
système peut établir le statut au moment de la décision.

### Principe 2 - Inconnu explicite

Une information inconnue ne reçoit pas une valeur par défaut qui crée un
engagement métier.

### Principe 3 - Exception traçable

Toute exception à une règle d'éligibilité indique son périmètre, son
autorité
d'acceptation et sa condition de réexamen.
```

Le socle technique donne au dossier un ancrage délibérément concret. Il évite qu'un plan suppose sans le dire le langage, la plateforme d'interface ou les conventions de documentation. Ces choix n'appartiennent pas à l'essence du SDD : une autre équipe peut retenir une autre cible. Dans ce projet, ils deviennent des invariants visibles auxquels le plan et les tâches doivent se conformer.

Ces principes ne calculent aucun prix. Ils ne disent pas si le seuil sera 16 h ou 17 h, ni quelle catégorie sera admise. Ils donnent une frontière à la discussion qui suit. Si `spec.md` proposait un prix définitif sans créneau urgent confirmé, le conflit ne serait pas une préférence entre deux

implémentations. Il contredirait le principe selon lequel un engagement doit être vérifiable.

La constitution ne remplace pas non plus le jugement humain. Quelqu'un doit toujours décider qu'un engagement vérifiable est bien le principe approprié pour ce service. Quelqu'un doit pouvoir le modifier si le métier change. La valeur du document est de rendre cette décision durable visible, pas de la rendre automatique.

2. `spec.md` - ce qui doit être vrai pour cette option

La spécification transforme la direction et les principes en attentes propres à la fonctionnalité. Elle répond d'abord à « quoi et pourquoi ? », avant la manière d'écrire la règle ou d'organiser les composants. Les exemples qui suivent ne constituent pas des exigences universelles de tarification. Ils expriment le contrat du scénario construit.

```
# Option de livraison urgente

## Objectif

Permettre au client de connaître un prix total fiable lorsque les
conditions
confirmées permettent réellement de proposer la livraison urgente.

## Exigences

### FR-01 - Livraison urgente éligible

Pour une demande de catégorie standard reçue strictement avant 16 h, avec
créneau urgent confirmé et préparation non commencée, afficher un prix
total
de 120 € : base de 100 € et supplément de 20 % visible, soit 20 €.

### FR-02 - Borne horaire

À 16 h exactement ou après 16 h, ne pas proposer l'option urgente pour
la catégorie standard.

### FR-03 - Créneau urgent inconnu

Lorsque le créneau urgent n'est pas confirmé, afficher le statut
« à confirmer » ; ne pas afficher de prix total incluant l'option.
```

```
### FR-04 - Préparation commencée

Lorsque la préparation a commencé, ne pas appliquer l'option
rétroactivement.

## Conditions ouvertes

### CO-01 - Catégorie spéciale

La règle applicable à la catégorie spéciale doit être confirmée par le
responsable d'exploitation avant toute proposition de livraison urgente.
```

Le contenu donne immédiatement des prises à la discussion. La borne est-elle vraiment stricte ? Le statut « à confirmer » est-il compréhensible pour le client ? L'option s'applique-t-elle à tous les envois de catégorie standard ? Ces questions ne sont pas des objections au SDD. Elles sont la matière que le SDD empêche de se cacher dans un `if` ou un test écrit après coup.

La condition ouverte est aussi importante que les exigences. Sans elle, une implémentation pourrait appliquer par défaut la règle standard à la catégorie spéciale. Le prix calculé serait peut-être cohérent dans le code. Il produirait néanmoins un engagement sans connaissance suffisante. `spec.md` ne résout pas l'incertitude ; il lui interdit de disparaître dans la solution.

Élément de la spécification	**Ce qu'il rend vérifiable**	**Ce qu'il laisse encore à décider**
FR-01	Les conditions et le montant total avec l'option urgente.	Le choix de conception du calcul.
FR-02	Le cas frontière de 16 h.	L'horloge de référence à clarifier avant que le plan en dérive le choix technique.
FR-03	L'absence de prix définitif si le créneau urgent n'est pas confirmé.	La manière de recueillir la confirmation.
CO-01	L'existence d'une limite de connaissance.	La règle de la catégorie spéciale et son autorité.

3. `clarify` - traiter une ambiguïté sans l'inventer

Une étape de clarification ne garantit pas que toute information sera trouvée. Elle rend l'ambiguïté assez précise pour qu'une personne puisse y répondre ou qu'une décision puisse être explicitement différée. Dans ce cas, deux questions apparaissent avant le plan.

```
# Clarifications de l'option de livraison urgente

## Q-01 - Horloge de référence

**Question :** quelle heure détermine qu'une demande est reçue avant
16 h ?

**Réponse :** l'heure enregistrée par le serveur, dans le fuseau
d'exploitation du service à la date de la demande.

## Q-02 - Catégorie spéciale

**Question :** la catégorie spéciale peut-elle recevoir l'option urgente
dans les mêmes conditions que la catégorie standard ?

**État :** à confirmer. Aucun prix total incluant l'option n'est affiché
pour
cette catégorie avant la décision du responsable d'exploitation.
```

La première réponse ajoute une précision indispensable au contrat. Sans elle, deux utilisateurs pourraient contester le même résultat en invoquant des horloges ou fuseaux différents. La seconde ne reçoit pas de réponse artificielle. Elle devient une condition de périmètre.

Cette différence est centrale. Clarifier n'est pas remplir les blancs avec la réponse la plus pratique pour l'implémentation. C'est séparer ce qui peut être défini maintenant de ce qui exige une connaissance, un mandat ou un arbitrage ultérieur. Une clarification qui aboutit à « à confirmer » peut être un très bon résultat si elle évite un engagement incorrect.

4. `plan.md` - choisir une forme sans réécrire le besoin

Le plan décrit comment l'équipe compte répondre à la spécification. Il peut indiquer un emplacement de décision, des données à consulter, des responsabilités techniques et des limites d'implémentation. Il doit rester dérivé de `spec.md` et de la constitution. Une solution séduisante qui ne respecte pas FR-03 ou CO-01 ne devient pas acceptable parce qu'elle est bien planifiée.

```
# Plan de l'option de livraison urgente

## Décisions de conception

### P-01 - Éligibilité

Le service de tarification C# évalue la catégorie et l'heure de
réception.
Il lit ces données sur le serveur, puis évalue créneau et préparation.
Il produit ensuite un résultat tarifaire destiné à l'interface WinUI 3.

### P-02 - Résultat incomplet

Lorsque le créneau urgent n'est pas confirmé ou que la catégorie est
spéciale
sans règle confirmée, le service renvoie un état « à confirmer » sans
prix
total incluant l'option urgente.

### P-03 - Traçabilité

Le résultat conserve les conditions qui ont fondé la proposition ou le
refus,
afin qu'une revue distingue une demande tardive d'un créneau non
confirmé.

## Hors périmètre

Le plan ne choisit pas la règle de la catégorie spéciale et ne modifie
pas le
montant d'une préparation déjà commencée.
```

Le plan fait apparaître quelques choix que `spec.md` n'avait pas à fixer : l'heure provient du serveur, les conditions sont évaluées ensemble, le résultat porte une trace compréhensible. Ces décisions sont techniques ou opérationnelles, mais elles restent contraintes par la spécification.

Le plan ne possède pas le pouvoir d'étendre le périmètre. Il ne peut pas décider que la catégorie spéciale utilise temporairement la règle standard pour « simplifier » le service. Il ne peut pas déclarer que l'affichage de 120 € est suffisamment clair pour compenser un créneau urgent non confirmé. Ces décisions reviendraient à modifier l'intention ou les exigences, et devraient donc remonter vers l'artefact qui les porte et vers le mandat approprié.

5. `tasks.md` - dériver du travail traçable, pas une liste indépendante

Les tâches traduisent le plan en travail réalisable. Elles sont utiles lorsque le lecteur peut remonter d'une tâche vers la décision et l'exigence qui la justifient. Une liste de tâches isolée peut donner une impression de progrès tout en perdant le sens du résultat attendu.

```
# Tâches de l'option de livraison urgente

- [ ] T-01 - Implémenter en C# les quatre conditions d'éligibilité de P-
01
  pour la catégorie standard. Répond à FR-01, FR-02 et FR-04.
- [ ] T-02 - Afficher dans l'interface WinUI 3 le statut « à confirmer »
  sans prix total incluant l'option. Répond à FR-03 et P-02.
- [ ] T-03 - Conserver les conditions ayant conduit à la proposition, au
refus
  ou au statut à confirmer. Répond à P-03.
- [ ] T-04 - Écrire les scénarios de preuve pour 15 h 59, 16 h, 16 h 01,
  créneau urgent inconnu et préparation commencée. Exerce FR-01 à FR-04.
- [ ] T-05 - Consigner la décision du responsable d'exploitation pour CO-
01
  ou conserver la catégorie spéciale hors périmètre.
```

T-05 est particulièrement révélatrice. Elle n'est pas une tâche de programmation qui peut être « terminée » par la production de code. Elle rend visible qu'un travail de décision reste nécessaire. Selon l'organisation, cette tâche peut être confiée à une personne, produire une note de décision ou aboutir à la confirmation que la catégorie spéciale demeure exclue. Le fichier de tâches aide à ne pas perdre cette dépendance ; il ne donne pas au développeur l'autorité de la supprimer.

6. `checklist` et `analyze` - chercher les écarts avant le code

Une checklist offre une manière simple de regarder si les contrôles importants ont été prévus. Une analyse de cohérence compare les artefacts afin de faire apparaître les exigences non couvertes, les tâches sans raison claire ou les choix de plan qui contredisent les limites. Ces gestes ne remplacent pas une preuve du comportement. Ils préparent une contradiction plus utile en empêchant que le code soit évalué contre un dossier déjà incohérent.

```
# Checklist de cohérence

- [x] Chaque exigence FR-01 à FR-04 possède au moins une tâche reliée.
- [x] Le plan explicite l'horloge de référence de la borne de 16 h.
- [x] Un créneau non confirmé ne produit pas de prix total avec l'option.
- [ ] CO-01 possède une décision d'exploitation confirmée.
# Résultat de `analyze`

CO-01 est toujours ouverte. Aucun plan ni aucune tâche d'implémentation
ne
doit attribuer l'option urgente à la catégorie spéciale. T-05 reste
visible jusqu'à une décision d'exploitation ou la confirmation de
l'exclusion.
```

Ce résultat est volontairement incomplet. L'analyse ne signale pas une défaillance du dossier parce qu'il contient une question ouverte. Elle signale une contradiction seulement si le plan ou les tâches font comme si cette question était résolue. C'est une nuance qui évite de transformer le SDD en quête de documents sans inconnues.

7. `implement` - produire contre un contrat qui peut répondre

L'implémentation intervient maintenant avec des limites mieux exposées. Elle peut rester créative sur la structure du service, la présentation du prix ou le mode de stockage de la trace. Elle ne peut pas traiter les règles de la spécification comme des suggestions lorsque le cas favorable est plus facile à coder.

La production obtenue dans ce scénario doit pouvoir être exercée au moins par les cas suivants :

Demande	Résultat attendu	Ce que le résultat protège
Standard, 15 h 59, créneau confirmé, préparation non commencée	Prix total de 120 € avec l'option urgente.	L'intention de proposer un prix fiable lorsque la livraison urgente est possible.
Standard, 16 h exactement	Option urgente indisponible.	La borne strictement avant 16 h.
Standard, créneau inconnu à 15 h 59	Statut « à confirmer », sans prix total définitif.	L'interdiction de promettre un délai non confirmé.
Standard, préparation commencée	Aucun ajout rétroactif de l'option.	L'absence de réécriture d'un montant déjà engagé.
Catégorie spéciale	Aucune proposition définitive de l'option avant CO-01.	La visibilité de la connaissance manquante.

Le tableau ressemble à un ensemble de tests, mais sa fonction est plus large. Il relie le comportement attendu à une intention et à une limite. Si un test échoue à 16 h exactement, l'équipe ne découvre pas seulement un opérateur de comparaison erroné. Elle découvre que l'implémentation ne respecte pas la borne à laquelle le métier avait donné une signification.

De même, un test vert sur le cas de 15 h 59 ne prouve pas que la catégorie spéciale peut recevoir l'option urgente. C'est la valeur de CO-01 : elle maintient une zone que le code ne peut pas annexer par défaut. L'implémentation donne une réponse aux conditions connues. Elle doit rester silencieuse, ou explicitement prudente, lorsque la règle n'est pas établie.

8. converge - regarder ce qui reste entre le dossier et le résultat

Converger ne signifie pas déclarer que tout travail imaginable est terminé. Dans un flux SDD, c'est le moment de comparer le résultat

produit avec les artefacts qui le justifient et d'identifier les écarts restants. Un écart peut conduire à modifier le code, à ajouter une tâche, à revoir le plan ou à reconnaître que la spécification elle-même doit changer.

```
# Résultat de `converge`

FR-01 à FR-04 sont couverts pour la catégorie standard par les scénarios
prévus. P-03 conserve la trace des conditions de décision. CO-01 demeure
volontairement ouverte : aucune option urgente définitive n'est
disponible
pour la catégorie spéciale. Ce n'est pas un défaut de l'implémentation
tant
que le périmètre accepté l'exclut explicitement.
```

Le mot important est « périmètre ». Une convergence sincère ne réécrit pas le dossier pour faire disparaître le manque. Elle peut conclure que le produit est cohérent avec une livraison réduite. Elle peut aussi découvrir que le responsable produit considère la catégorie spéciale indispensable à la valeur de la fonctionnalité. Dans ce cas, l'équipe ne devrait pas fermer la question avec un indicateur vert. Elle devrait revenir à CO-01, rechercher la règle et redéfinir l'acceptation.

`converge` ne porte donc pas la décision finale. Il rend plus difficile de prétendre que le code a satisfait une partie du besoin qui était explicitement laissée ouverte. Il garde possible le retour vers `spec.md`, `plan.md` ou `tasks.md` lorsque le résultat révèle une différence réelle.

9. L'acceptation reste humaine et déterminée

Le dossier est maintenant plus complet : il contient des principes, des exigences, une clarification, des choix, des tâches, des contrôles et des comportements exercés. Rien de cela ne dit encore qui accepte de mettre l'option en service.

Dans le scénario, le responsable d'exploitation accepte la livraison pour la catégorie standard, à condition que les résultats prévus soient observables et que la catégorie spéciale reste exclue. Il ne valide pas « le SDD » ni une collection de fichiers. Il accepte un périmètre précis : un prix total de 120 € lorsqu'un créneau urgent est confirmé, un refus après la borne ou après le début de la préparation, un statut prudent

quand le créneau manque, et aucune proposition définitive pour la catégorie spéciale tant que CO-01 n'est pas résolue.

Cette acceptation peut être brève, mais elle doit pouvoir être relue :

```
# Décision d'acceptation


L'option de livraison urgente de la catégorie standard est autorisée dans le
périmètre de FR-01 à FR-04. La catégorie spéciale demeure exclue jusqu'à une
décision d'exploitation documentée. Toute modification du seuil, du supplément
ou de la règle de créneau urgent déclenche une révision de la spécification et
du mandat d'acceptation.
```

Le texte ne remplace pas une autorité réelle. Il montre précisément pourquoi la décision reste nécessaire. Les artefacts ont préparé une conversation plus courte et plus défendable. Ils n'ont pas accepté le risque à la place de la personne qui en porte les conséquences.

Ce que ce dossier rend enfin visible

Le dossier n'est pas intéressant parce qu'il contient quatre fichiers et quelques étapes nommées. Il est intéressant parce qu'il maintient des liens que la vitesse de production peut rompre :

Si l'équipe regarde seulement…	**Elle peut perdre…**	**Le dossier SDD rend visible…**
La demande initiale	La différence entre une option demandée et un créneau urgent réellement disponible.	L'objectif et les principes qui orientent la fonctionnalité.
Le code de calcul	La borne, le créneau et la préparation déjà commencée.	Les exigences et cas frontières de `spec.md`.
La liste de tâches	La raison de la tâche et la question qui reste ouverte.	Le lien entre `tasks.md`, le plan et CO-01.
Les tests verts	La partie du besoin qui demeure hors périmètre.	La convergence avec un périmètre explicitement

Si l'équipe regarde seulement…	Elle peut perdre…	Le dossier SDD rend visible…
		réduit.
L'indicateur de livraison	La personne qui accepte le compromis.	Le mandat et la décision d'acceptation.

Le lecteur n'a pas besoin d'adopter ces noms, ni de produire autant d'artefacts, pour retenir la leçon. Il doit pouvoir reconnaître qu'une constitution porte des principes, qu'une spécification porte le résultat attendu et les inconnues, qu'un plan choisit une forme, que des tâches dérivent du plan, et que les contrôles ne remplacent ni la preuve ni l'acceptation.

Le chapitre suivant présentera quatre équipes qui ne remplissent pas le même dossier et ne prennent pas la même décision. L'interlude donne une base concrète pour comprendre cette diversité : le SDD n'est pas une cérémonie de fichiers. C'est une manière de conserver le chemin qui relie une intention à une décision assumée.

Source et portée

Sources et portée

speckit. GitHub, *Spec Kit - Spec-Driven Development*, consulté le 20 septembre 2026. La source décrit les artefacts, le flux central et des parcours indépendants de la boîte à outils. Les extraits, le scénario tarifaire, les décisions et les formulations de cet interlude sont des constructions pédagogiques du livre, non des modèles officiels ni une recommandation d'usage universelle.

Chapitre 9 - Quatre scénarios d'équipe

Les chapitres précédents ont séparé des objets que la vitesse de production tend à mêler : l'intention, les contraintes, ce que nous savons, ce qui doit être vrai, les preuves et les mandats. Cette séparation peut sembler nette tant qu'elle reste théorique. Dans une équipe, les objets n'arrivent jamais dans cet ordre. Une date est déjà annoncée, une proposition existe, une connaissance manque, un test échoue ou un responsable doit répondre avant que tous les éléments soient réunis.

Les quatre situations de ce chapitre sont construites. Elles ne décrivent ni des clients réels ni une méthode de transformation à appliquer. Elles ne cherchent pas à prouver que la discipline proposée améliore toute organisation. Elles permettent une comparaison plus modeste et plus utile : *lorsque le code devient rapide à produire, quelle information ferait réellement changer la décision ?*

Chaque équipe conserve un gain. Chaque équipe accepte aussi un coût. La discipline ne produit donc pas quatre refus prudents, ni quatre feux verts rapides. Elle rend la décision lisible avant que la production ne lui donne l'apparence d'un fait acquis.

A - Produire plus vite que l'on peut valider

Une petite équipe produit un service de livraison. À l'approche d'une période chargée, elle veut permettre à un client indisponible de demander le déplacement de son créneau. Un agent IA propose rapidement les trois formes déjà rencontrées au chapitre 1 : déplacer automatiquement le créneau, ouvrir une demande asynchrone traitée par un planificateur, ou conserver le créneau initial tout en proposant une nouvelle réservation.

La première proposition paraît séduisante. Elle réduit le délai apparent et donne une impression d'automatisation complète. Pourtant, l'équipe ne dispose pas encore de preuves suffisantes sur les interactions entre réservation automatique, créneaux concurrents et annulations. Un code qui trouve un créneau libre dans un test local ne démontre pas

qu'il respecte les engagements pris par la tournée, ni qu'il gère correctement les conséquences d'une annulation simultanée.

L'intention est plus étroite que la solution proposée : permettre à un client indisponible de signaler le changement assez tôt pour qu'une nouvelle livraison puisse être organisée, sans annoncer un créneau qui n'est pas confirmé. Trois limites sont déjà connues. La demande doit être reçue avant le départ en tournée. La promesse initiale doit rester traçable. Une demande reçue ne modifie pas automatiquement le créneau existant avant l'acceptation du planificateur.

Le responsable produit réduit donc le périmètre de la livraison. Le parcours proposé au client enregistre une demande, confirme sa réception et rend son état visible. Il ne promet pas un nouveau créneau. Le responsable d'exploitation rapproche une estimation documentée du volume attendu de la capacité de traitement prévue pour les planificateurs. Le responsable technique interprète les démonstrations du parcours et du traitement. Aucun de ces éléments ne valide l'affectation automatique.

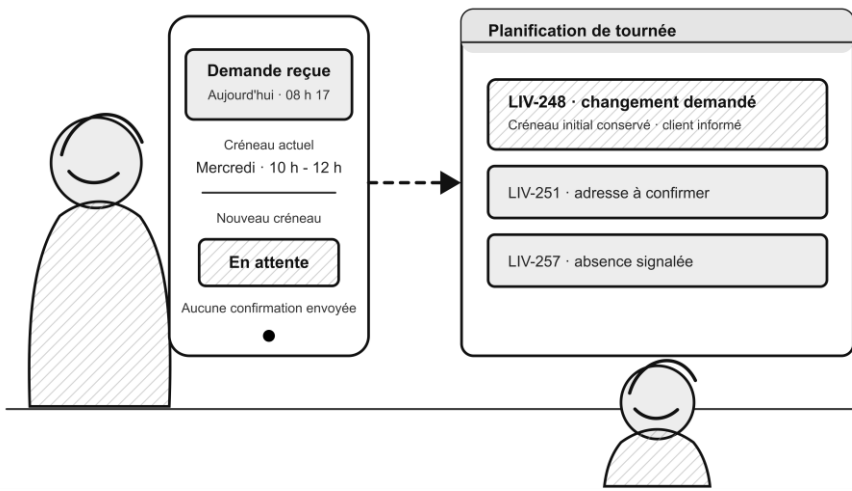


Figure 9.1 - Scénario construit. Le client voit que sa demande a été reçue tandis que le nouveau créneau reste en attente de la décision du planificateur.

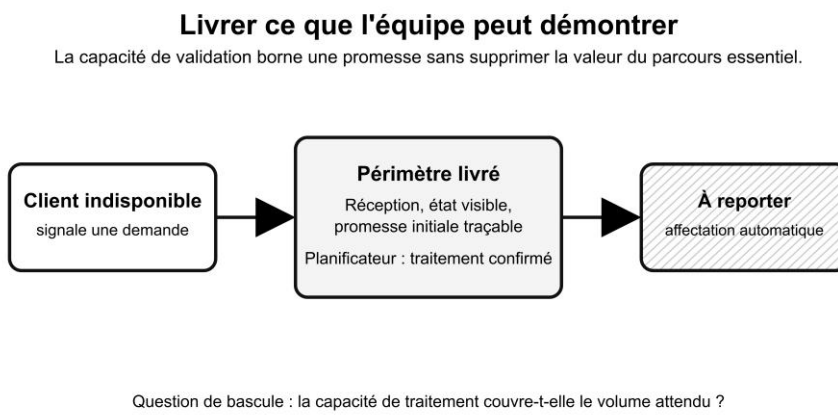


Figure 9.2 - Scénario construit. Le parcours essentiel est livré parce que l'équipe peut le démontrer et en absorber le traitement ; l'automatisation reste hors du périmètre.

La décision conserve une valeur immédiatement perceptible. Le client peut signaler son indisponibilité au lieu de rester sans recours. L'équipe apprend comment les demandes se présentent réellement. Les planificateurs disposent d'un état plutôt que d'une information dispersée. Le coût est lui aussi réel : une partie du traitement reste humaine, le nouveau créneau peut prendre du temps à être confirmé et l'automatisation est reportée.

Une équipe pressée pourrait appeler ce coût un échec de l'IA ou un renoncement à l'ambition initiale. Ce serait confondre l'objectif avec le périmètre que l'équipe sait soutenir à cette date. Le gain local de production a été utilisé pour préparer et livrer un parcours limité, pas pour faire passer une promesse non démontrée pour une capacité acquise.

Objet de décision	État dans ce scénario	Effet sur la livraison
Intention	Aider le client à signaler un changement sans faux engagement.	Le parcours sépare demande et confirmation.
Contrainte	Pas de modification après le départ ; promesse initiale traçable.	Des demandes sont refusées ou conservées sans déplacer le créneau.

Objet de décision	État dans ce scénario	Effet sur la livraison
Connaissance	Interactions d'affectation automatique incomplètes.	L'automatisation est reportée.
Critère	Le planificateur traite la demande et le client voit l'état.	Le parcours minimal peut être démontré.
Preuve	Démonstration du parcours et capacité opérationnelle documentée.	Le périmètre réduit est autorisé.
Mandat	Produit réduit la promesse ; exploitation confirme la capacité ; technique interprète l'intégration.	Aucune réception n'est présentée comme une confirmation.

Quelle information ferait changer cette décision ? La capacité du planificateur pour le volume attendu. Si elle n'est pas confirmée, même le parcours réduit doit être revu. Il ne suffirait pas de constater que l'écran fonctionne : une demande visible sans capacité de traitement pourrait transformer une attente client en nouvelle promesse implicite.

Ce premier cas ne défend donc ni le traitement manuel par principe ni l'automatisation par principe. Il montre qu'une équipe peut poursuivre lorsque la capacité de validation correspond au périmètre annoncé. Elle arrête seulement de confondre une solution techniquement plausible avec une promesse qu'elle sait tenir.

B - Produire plus vite que l'on peut savoir

Une application métier ancienne doit intégrer un nouveau partenaire. Dans le référentiel, deux références clientes apparaissent : `P-0042` et `42`. Une transformation proposée par l'agent IA retire le préfixe et les zéros de tête. Les tests de chaîne passent : dans les deux cas, le résultat devient `42`. La modification est petite, élégante et facile à expliquer dans le code.

Cette facilité est précisément ce qui rend l'arbitrage risqué. La collision de représentation est établie, pas l'équivalence des identités. Un ticket ancien suggère qu'un canal distinguait autrefois certains dossiers, sans confirmer sa portée actuelle. Le propriétaire métier du référentiel, le support et les mainteneurs possèdent chacun une partie de la connaissance. Personne ne peut convertir une transformation de chaîne en qualification métier par simple confiance dans le test.

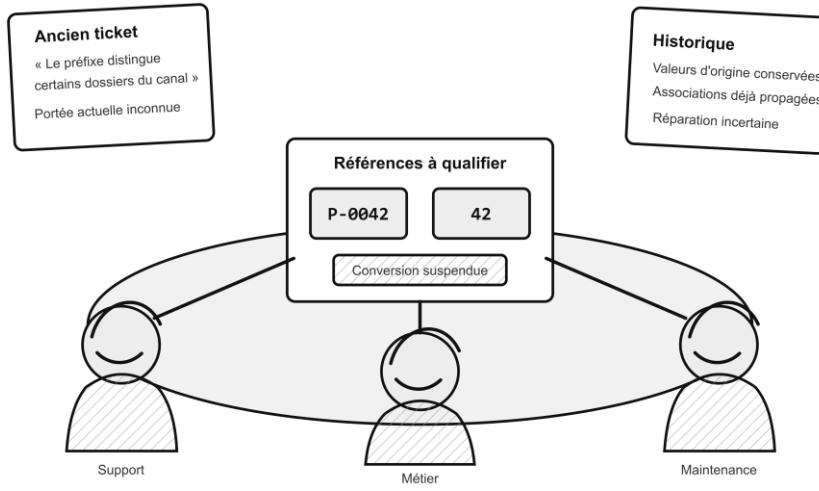


Figure 9.3 - Scénario construit. Le support, le métier et la maintenance réunissent des fragments de connaissance avant de qualifier les références ambiguës.

L'intention reste claire : intégrer le partenaire sans perdre les associations entre clients et dossiers. Une exigence est confirmée : l'identité doit être préservée. La spécification négative est tout aussi importante que la transformation souhaitée : deux références dont l'équivalence n'est pas établie ne doivent pas être fusionnées. Les collisions doivent rester visibles jusqu'à ce qu'une qualification documentée permette une correspondance approuvée.

L'équipe suspend la conversion vers le partenaire sur ce sous-ensemble. Elle conserve les références d'origine et le fonctionnement existant. Elle consigne la condition de reprise : qualifier chaque référence ambiguë, puis approuver une correspondance qui traduit cette qualification. Si les dossiers historiques ne peuvent pas être

retrouvés, une autre décision devra être négociée : réduire le périmètre de l'intégration ou renoncer à la conversion pour les références concernées.

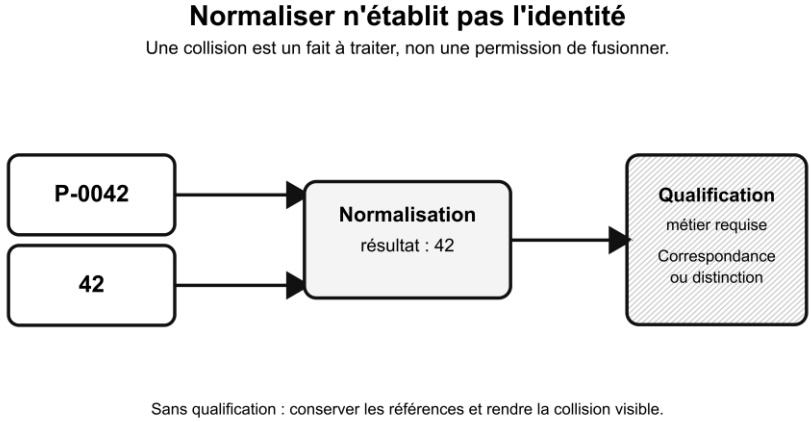


Figure 9.4 - Scénario construit. La normalisation rend une collision visible ; seule une qualification documentée peut autoriser une correspondance ou imposer une distinction.

Le coût de cette décision est difficile à célébrer. La date d'intégration peut être déplacée. Un code déjà écrit peut être repris. Le partenaire peut recevoir un périmètre plus petit. Le bénéfice est moins spectaculaire, mais plus important : l'équipe ne crée pas une fusion de données qu'elle ne saurait pas expliquer ni réparer facilement.

La suspension n'est pas une réponse vague du type « nous devons mieux comprendre ». Elle porte un statut et une condition. La question est connue, les sources de connaissance sont identifiées, l'exigence est explicite et le point de reprise peut être vérifié. Cette précision distingue un arbitrage ouvert d'un report indéfini qui laisserait l'équipe croire que la transformation est prête dès qu'une contrainte de calendrier se présente.

Énoncé	Statut	Conséquence
`P-0042` et 42 deviennent 42 après normalisation.	Établi par la transformation.	Une collision doit être traitée explicitement.

Énoncé	Statut	Conséquence
Les deux références désignent le même client.	À confirmer.	Aucune fusion n'est autorisée.
L'identité des dossiers doit être préservée.	Exigence confirmée.	Le gain de simplification ne permet pas de déroger.
Le partenaire peut intégrer un sous-ensemble.	Option à négocier.	Une voie existe si la connaissance manque durablement.

La personne qui écrit le code reste utile : elle peut montrer les dépendances, isoler les collisions et préparer les tests de correspondance. Elle ne doit pas décider seule de l'identité à partir d'une forme de donnée. Ici, produire plus vite que l'on peut savoir ne justifie pas d'accélérer l'engagement. Il invite à conserver la connaissance manquante comme un objet de décision.

Quelle information ferait changer la décision ? Une qualification qui établit l'identité ou la distinction de chaque référence ambiguë, accompagnée d'une correspondance approuvée. Si cette information ne peut pas être obtenue, l'équipe n'a pas « échoué à terminer ». Elle a appris que la conversion universelle n'est pas une option acceptable pour ce périmètre.

C - Construire plus vite que l'on ne décide

Une équipe prépare un service de réservation destiné à plusieurs organisations. Un prototype produit avec une assistance importante rend les écrans convaincants. La création, la consultation et la modification de réservations fonctionnent pour la première organisation. La démonstration produit l'effet attendu : les responsables produit peuvent discuter de situations concrètes plutôt que d'imaginer un service abstrait.

La tentation est de transformer ce prototype en fondation durable parce qu'il est déjà fonctionnel. Mais une exploration technique révèle que l'espace de stockage est commun et que le filtrage par organisation est assuré seulement dans l'interface. Un utilisateur de l'organisation B qui

connaît l'identifiant d'une réservation de A peut encore la consulter par appel direct à l'API.

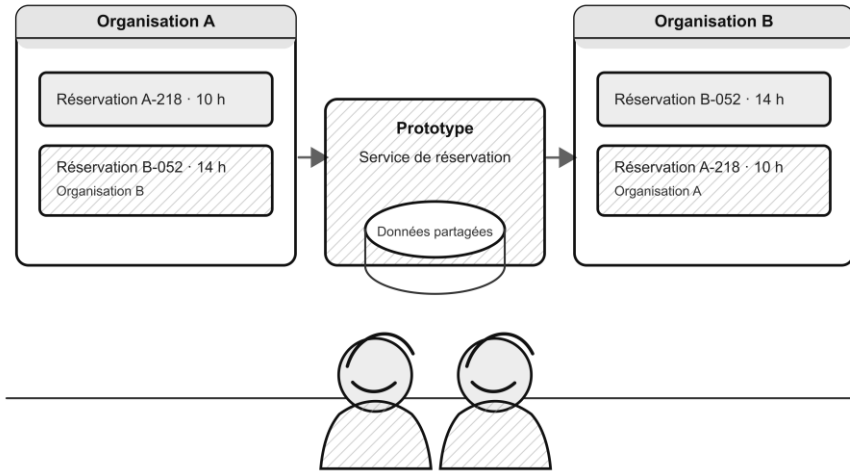


Figure 9.5 - Scénario construit. Le prototype fonctionne pour deux organisations, mais son espace de données commun laisse apparaître les réservations de l'une dans l'interface de l'autre.

L'intention n'est pas seulement de montrer les réservations. Elle est de permettre à chaque organisation de gérer les siennes sans lire ni modifier celles des autres. L'isolation entre organisations est une exigence confirmée avant toute ouverture à plusieurs clients. Elle n'est pas une préférence de conception que l'équipe pourrait remplacer par une interface plus soignée.

La preuve contradictoire est simple et décisive pour le périmètre étudié : deux comptes sont créés, une requête directe est faite avec l'identifiant d'une réservation de A et le prototype renvoie encore la donnée. Cette preuve n'impose ni une base de données particulière ni un découpage universel en services. Elle rend impossible de présenter la persistance et le contrôle actuels comme satisfaisants pour une seconde organisation.

L'équipe conserve les écrans et les apprentissages métier. Elle refuse en revanche de faire du modèle de persistance du prototype une référence durable. Avant l'ouverture, elle reprend le rattachement des

données à une organisation et les contrôles côté service. Les prochaines preuves devront couvrir la lecture, la modification et les accès directs. Le passage d'un test ne garantira pas, à lui seul, toute l'isolation.

Décision possible	Bénéfice immédiat	Conséquence non acceptable
Ouvrir le prototype tel quel	Une seconde organisation accède vite au service.	Le filtrage d'interface ne protège pas les appels directs.
Conserver seulement les écrans	Les apprentissages métier restent disponibles.	La persistance prototype ne devient pas un engagement silencieux.
Reprendre le modèle et les contrôles	L'ouverture est différée.	L'isolation peut être démontrée avant l'exposition.

Le responsable produit porte la décision d'ouverture et le périmètre visé. Le responsable d'architecture mandate la reprise puis interprète les preuves d'isolation. Leurs accords respectifs sont nécessaires : aucun ne peut écarter seul une exigence qui conditionne l'ouverture. Cette répartition ne crée pas une lenteur cérémonielle. Elle reconnaît que le code fonctionnel du prototype a rendu une décision structurante plus urgente, non moins nécessaire.

Quelle information ferait changer la décision de refuser l'ouverture ? Des preuves que B ne peut ni lire ni modifier une réservation de A, y compris par accès direct, pour le périmètre d'ouverture prévu. Tant que ces preuves manquent, l'équipe peut continuer à apprendre sur les écrans. Elle ne doit pas étendre l'exposition du modèle qu'elle sait insuffisant.

Le coût accepté est du code repris et un délai d'ouverture. Le bénéfice est de choisir les engagements de données et de contrôle avant que des données clientes et des contrats externes ne rendent cette reprise beaucoup plus difficile. Construire plus vite n'a pas supprimé la décision. Il a rendu plus visible le moment où l'équipe devait la prendre.

D - Poursuivre ou renforcer selon la preuve requise

Une même organisation envisage deux changements. Le premier ajoute une préférence de tri à une liste d'affichage. Le second doit permettre à une personne d'assistance de consulter, pendant une période limitée, des dossiers appartenant à une autre organisation. Les deux changements peuvent être produits rapidement. Ils ne demandent pas la même preuve, ni le même mandat, ni la même décision.

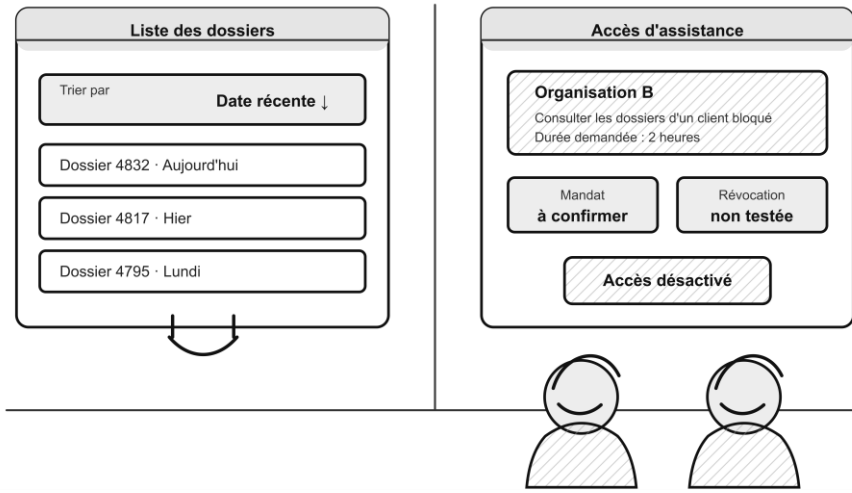


Figure 9.6 - Scénario construit. Une préférence de tri locale et un accès temporaire aux dossiers d'une autre organisation n'engagent ni les mêmes preuves ni les mêmes autorités.

La préférence de tri vise le confort de l'utilisateur. Son critère est borné : elle n'affecte que l'affichage local, peut être réinitialisée, ne change pas l'ordre de traitement métier et ne crée aucun nouvel accès. Une démonstration de réinitialisation, la vérification des requêtes et le contrôle des droits existants établissent ce périmètre. Le responsable de livraison est mandaté pour ce changement local. Le socle obligatoire demeure : changement traçable, vérifications pertinentes et déploiement autorisé. Le contrôle est léger, non absent.

L'accès d'assistance est différent. L'intention peut être légitime : aider un client bloqué. Mais confidentialité et cloisonnement sont impératifs.

L'information déterminante n'est pas seulement « l'accès fonctionne-t-il ? ». C'est le périmètre précis du mandat : quels dossiers, pendant combien de temps, avec quelle condition de révocation et sous quelles autorités. Une exception autorisée peut ouvrir l'accès ; l'absence de mandat ou l'expiration doivent le refuser.

Les tests disponibles ne couvrent encore que le parcours autorisé. Ils ne démontrent pas les refus sans mandat, après révocation ou hors périmètre. Ils ne montrent pas non plus que les accès autorisés laissent une trace attribuable et exploitable. L'accès reste donc désactivé. Le responsable habilité des droits et le responsable métier doivent accepter le périmètre permis une fois les preuves obtenues. Ils ne peuvent pas déclarer conforme un accès interdit ou non démontré.

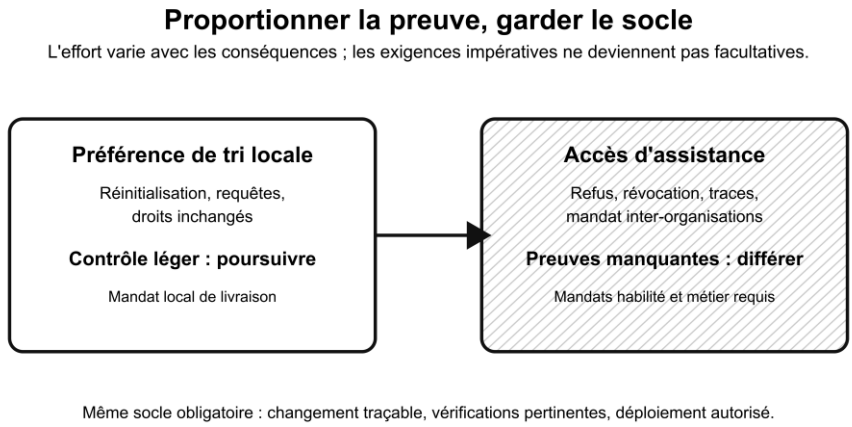


Figure 9.7 - Scénario construit. Proportionner la preuve distingue le tri local de l'accès inter-organisations ; le socle obligatoire ne disparaît dans aucun des deux cas.

Question	Préférence de tri	Accès d'assistance
Intention	Améliorer un affichage local.	Permettre une aide bornée sur un dossier externe.
Limite principale	Ne pas modifier le traitement ni les droits.	Préserver confidentialité et cloisonnement.
Preuve attendue	Réinitialisation, requêtes	Refus sans mandat, après

Question	Préférence de tri	Accès d'assistance
	et absence de nouveaux accès.	révocation et hors périmètre ; traces attribuables.
Mandat	Responsable de livraison.	Responsables habilité et métier dans le périmètre accepté.
Décision	Poursuivre avec un contrôle léger.	Différer l'activation et renforcer la preuve.

Ce contraste évite deux erreurs symétriques. La première consiste à soumettre le tri local à un niveau de preuve qui rendrait toute amélioration mineure disproportionnée. La seconde consiste à traiter l'accès inter-organisations comme une variation plus complexe du même changement. Dans les deux cas, l'équipe perdrait le rapport entre l'effort et l'impact réel.

Proportionner ne signifie pas négocier les obligations. Les contrôles nécessaires au tri ne sont pas facultatifs parce que le changement est local. Les exigences de confidentialité ne deviennent pas optionnelles parce que l'assistance est urgente. La proportion porte sur l'investigation, l'étendue des scénarios, l'autorité mobilisée et le moment d'activation. Elle ne donne pas un pouvoir de dérogation à une exigence impérative.

Quelle information ferait changer les décisions ? Pour le tri, la découverte qu'il modifie l'ordre métier ou un droit d'accès ferait sortir le changement de son périmètre local. Pour l'assistance, les preuves de refus, de révocation et de traces, ainsi que l'acceptation d'un mandat précis, permettraient d'envisager une activation. Ce ne sont pas deux degrés d'un même feu vert. Ce sont deux arbitrages dont les conditions de poursuite diffèrent.

Ce qui converge sans devenir une recette

Les quatre équipes n'ont pas rencontré la même limite. La première a réduit une promesse pour rester dans sa capacité de validation. La

deuxième a refusé de transformer une collision de forme en identité établie. La troisième a conservé un prototype utile tout en refusant d'en faire un engagement d'isolation insuffisant. La quatrième a poursuivi un changement local et différé un accès sensible parce que leurs preuves et mandats n'avaient pas le même poids.

Limite dominante	**Décision prise**	**Coût accepté**	**Ce qui aurait rendu la décision différente**
Capacité de validation	Livrer la demande asynchrone, reporter l'affectation.	Traitement humain et périmètre réduit.	Capacité opérationnelle insuffisante ou preuves d'automatisation.
Connaissance dispersée	Suspendre la conversion ambiguë.	Délai et intégration réduite possible.	Qualification d'identité et correspondance approuvée.
Choix structurant	Reprendre persistance et contrôles avant ouverture.	Code fonctionnel repris et délai.	Preuves d'isolation lecture, modification et API.
Preuves et mandats	Livrer le tri, conserver l'accès désactivé.	Effort de preuve et activation différée.	Refus, révocation, traces et mandat accepté.

La matrice ne doit pas être lue comme un outil de notation. Elle ne dit pas qu'une équipe qui possède six colonnes remplies est mature, ni qu'une réponse est universellement préférable à une autre. Elle rend simplement visible que la production n'est qu'une partie d'une décision. Les mêmes mots - intention, contrainte, connaissance, critère, preuve et mandat - permettent de formuler des décisions différentes sans confondre leurs raisons.

Dans les quatre situations, une capacité d'IA aurait pu multiplier les propositions, accélérer l'implémentation ou écrire une première version des tests. Aucune de ces capacités n'aurait pu répondre de manière autonome à la question qui fait basculer l'arbitrage : la capacité

humaine de traitement est-elle présente ? Deux identités sont-elles établies comme équivalentes ? L'isolation est-elle démontrée pour l'ouverture prévue ? Un mandat permet-il réellement l'accès demandé ?

Cette remarque ne transforme pas le jugement humain en limite mystérieuse ou immuable. Les connaissances, les instruments et les mandats peuvent évoluer. Une équipe peut documenter une règle historique, automatiser une preuve d'isolation ou définir plus précisément un accès d'assistance. Ce qui demeure est la nécessité de savoir ce qui est établi, ce qui doit être vrai et qui peut accepter ce qui reste ouvert.

Lire les coûts sans les effacer

Les quatre cas ont un point commun moins séduisant que leurs solutions : ils coûtent quelque chose au moment où l'équipe choisit de ne pas laisser la production décider à sa place. Le service de livraison conserve du traitement humain. Le référentiel ancien accepte un délai et une intégration possiblement partielle. Le prototype perd une partie de l'élan qui vient d'un produit déjà visible. L'accès d'assistance reste indisponible alors qu'il pourrait résoudre un besoin immédiat.

Nommer ces coûts est nécessaire. Une discipline qui ne reconnaîtrait que les bénéfices de la prudence serait rapidement abandonnée par les équipes soumises à une date, à une attente client ou à une contrainte de disponibilité. Le problème n'est pas que les coûts existent. Le problème est de les masquer jusqu'à ce qu'ils soient payés sous une autre forme : surcharge des planificateurs non prévue, associations de dossiers perdues, exposition de données inter-organisations ou exception de droit d'accès sans trace exploitable.

Cette comparaison révèle un autre point. La décision de réduire n'est pas toujours conservatrice et la décision de poursuivre n'est pas toujours audacieuse. Dans le premier scénario, réduire le périmètre permet de livrer une capacité qui répond directement à l'intention. Dans le quatrième, poursuivre le tri local est plus rigoureux que retarder une amélioration sans raison liée à ses conséquences. Les mots « prudence » et « vitesse » décrivent mal ces cas. Ce qui compte est

l'ajustement entre ce qui est engagé, ce qui est établi et ce que l'équipe peut réellement assumer.

Les coûts peuvent également se déplacer dans le temps. Le traitement humain accepté aujourd'hui peut fournir des connaissances qui rendent l'automatisation future mieux définie. Le délai de qualification des références peut révéler qu'un partenariat doit être segmenté. La reprise du prototype peut éviter un contrat de données dont la réparation aurait été bien plus coûteuse après l'ouverture. Une activation différée peut obliger à produire des traces et des refus qui serviront aussi à d'autres accès sensibles.

Il ne faut pas transformer cette possibilité en promesse. Un coût présent ne devient pas toujours un investissement réussi. L'équipe doit pouvoir constater qu'un périmètre réduit ne produit pas la valeur espérée, que la connaissance historique est perdue ou qu'une preuve requise est disproportionnée à un besoin devenu secondaire. La discipline laisse alors la possibilité d'abandonner une option de façon explicite. Elle ne pousse pas à maintenir une recherche ou un chantier simplement parce qu'il a déjà été documenté.

Le même langage, des questions de bascule différentes

Le lecteur pourrait chercher une question unique qui déclencherait automatiquement une décision. Les scénarios montrent pourquoi cette recherche serait trompeuse. « Avons-nous assez de preuves ? » est une question utile, mais elle reste incomplète tant qu'elle n'indique pas la propriété, le périmètre et le mandat concernés.

Dans le service de livraison, la question de bascule porte sur la capacité opérationnelle : le planificateur peut-il traiter le volume attendu sans transformer l'accusé de réception en promesse vide ? Dans le référentiel, elle porte sur une qualification : que permettent réellement les éléments disponibles au sujet de l'identité ? Dans le prototype, elle porte sur une propriété d'isolation observable par appel direct. Dans l'accès d'assistance, elle porte sur les refus, la révocation, les traces et l'autorité qui accepte le périmètre.

La forme de ces questions aide l'équipe à éviter les réponses trop générales. Une réponse comme « le pipeline est vert » ne confirme pas la capacité des planificateurs. Une réponse comme « le code normalise les identifiants » ne qualifie pas des identités. Une réponse comme « l'interface ne montre pas les données » ne prouve pas l'isolation d'une API. Une réponse comme « l'accès fonctionne pour le support » ne démontre pas les refus qui définissent sa limite.

Cette précision ne demande pas de prévoir toutes les questions à l'avance. Elle demande de conserver une place pour celle qui peut encore modifier la décision. Les équipes apprennent alors à rechercher moins de signaux rassurants et davantage d'informations discriminantes. Une preuve est utile lorsqu'elle distingue des options qui semblaient équivalentes, ou lorsqu'elle empêche de poursuivre sur une hypothèse non reconnue.

Le langage commun sert enfin à la conversation entre équipes. Produit, exploitation, architecture et sécurité ne doivent pas adopter la même solution pour contribuer au même arbitrage. Ils peuvent cependant se comprendre lorsqu'ils rendent explicites l'intention, la limite, le fait manquant, le critère, la preuve et le mandat. Cette grammaire ne remplace pas le désaccord. Elle le rend plus précis et donc plus facile à traiter avant qu'une différence de code ne le transforme en conséquence distribuée.

> **Dans une pratique SDD** Les quatre situations pourraient produire des artefacts très différents : un `spec.md` de parcours réduit, une question issue de `clarify`, un critère d'isolation à exercer par `checklist` ou un mandat d'accès relié à `plan.md`. Leur point commun n'est pas un formulaire à remplir. C'est le lien conservé entre une intention, une limite, une preuve et l'autorité qui choisit de poursuivre, réduire, différer ou reprendre.

La discipline ne promet donc pas l'infaillibilité. Elle rend une erreur plus susceptible d'être rencontrée avant la propagation, une décision plus facile à expliquer et une escalade plus possible lorsque le périmètre dépasse l'équipe. Le dernier chapitre n'ajoutera pas une méthode pour

administrer ces cas. Il examinera la posture qui reste praticable lorsque les outils, les capacités et les formes de production changent encore.

Chapitre 10 - La discipline de la délégation

Les quatre équipes du chapitre précédent ne suivent pas une même procédure. L'une réduit un périmètre de livraison, l'autre suspend une transformation, la troisième reprend un prototype et la quatrième poursuit un changement tout en différant un autre. Cette diversité n'est pas un défaut de la démonstration. Elle est son point de départ : une organisation ne garde pas le contrôle en appliquant la même réponse à toutes les situations.

Ce qui demeure entre ces décisions est une discipline. Elle consiste à garder visibles les conditions sous lesquelles nous délégons une production à une IA générative ou à un agent de code, et acceptons ses conséquences. L'intention donne une direction. Les contraintes disent ce qu'une solution ne peut pas sacrifier. Le statut de connaissance empêche de confondre une hypothèse avec un fait. La spécification énonce ce qui doit être vrai. Les preuves établissent une portée limitée. Les mandats rendent identifiable la personne qui peut accepter, refuser ou faire remonter le risque résiduel.

Cette discipline n'est pas une nouvelle méthode à déployer ni une liste de contrôles que tout changement devrait traverser. Elle n'ajoute pas une couche de gouvernance par-dessus le travail existant. Elle donne un langage pour reconnaître que produire, même vite, n'épuise pas la décision.

Quelle posture reste utile lorsque les outils changent plus vite que les pratiques qui les encadrent ?

Produire n'est pas accepter

Un agent IA peut produire un plan, une migration, une interface, des tests et une explication de code. Une équipe peut les revoir rapidement et constater qu'ils fonctionnent dans les exemples disponibles. Cette production reste précieuse. Elle permet d'explorer davantage d'options, de rendre une intention concrète et de libérer du temps pour des questions qui, autrement, seraient repoussées.

Mais une production d'agent IA n'est pas une acceptation. Elle ne décide pas seule que la demande mérite d'être poursuivie, que les limites sont respectées, que la connaissance est suffisante, que la preuve a la portée demandée ou que le risque résiduel appartient au mandat de l'équipe. Faire comme si ces questions étaient contenues dans le code revient à laisser l'agent IA trancher des arbitrages que personne ne lui a explicitement confiés.

La différence apparaît dans les scènes du livre. Le prototype de réservation produit des écrans crédibles, mais n'accepte pas l'ouverture à une seconde organisation. La transformation de références retourne une valeur cohérente, mais n'accepte pas une fusion d'identités non qualifiées. Les tentatives de notification diminuent certains échecs, mais n'acceptent pas les conséquences produit d'un doublon. Dans chaque cas, un résultat utilisable existe et une décision reste à prendre.

Cette distinction ne demande pas de soupçonner tout résultat généré. Elle invite à préciser la question à laquelle le résultat répond. Une proposition peut être excellente pour rendre un choix visible. Un test peut être excellent pour établir un cas frontière. Une simulation peut être excellente pour réfuter une hypothèse. Leur valeur grandit lorsque l'équipe ne leur attribue pas, par automatisme, le pouvoir d'accepter ce qu'ils n'ont pas établi.

Geste	**Ce qu'il peut produire**	**Ce qu'il ne remplace pas**
Générer une proposition	Des options et une première forme de solution.	L'intention et les contraintes qui rendent une option acceptable.
Écrire des tests	Une vérification sur des propriétés choisies.	Le jugement sur la pertinence et la portée de ces propriétés.
Mesurer un comportement	Une observation dans un périmètre donné.	L'acceptation de ses conséquences dans un produit réel.
Réussir une revue	Une compréhension partagée de la différence étudiée.	Le mandat d'accepter un risque hors du périmètre des relecteurs.

Confier une tâche à un agent IA est donc plus exigeant que le simple transfert d'une tâche. L'organisation doit conserver la faculté de dire pourquoi elle produit, ce qu'elle refuse, ce qu'elle ignore encore, ce qu'elle doit démontrer et qui répond de la suite. Cette faculté n'est pas un ralentissement ajouté après coup. Elle est la condition qui rend la vitesse soutenable lorsque les conséquences ne sont pas toutes visibles dans l'artefact produit.

Diriger, évaluer, accepter ou escalader

On peut rassembler cette posture autour de quatre verbes. Ils ne constituent pas une séquence obligatoire et ne désignent pas quatre rôles. Une même décision peut revenir de l'un à l'autre : une preuve peut obliger à rediriger l'intention, une nouvelle contrainte peut exiger une réévaluation, un mandat absent peut entraîner une escalade avant toute acceptation.

Diriger

Diriger consiste à formuler la direction que doit servir la production. C'est l'intention du chapitre 4 : quel résultat cherchons-nous, pour quelle personne et contre quel problème ? Dans le cas du créneau de livraison, l'intention n'était pas d'automatiser tout déplacement. Elle était de permettre au client indisponible de signaler un changement sans lui promettre un créneau non confirmé.

Diriger consiste aussi à poser les limites. La confidentialité, l'absence de duplication d'un import, la préservation de l'identité ou l'interdiction d'un accès hors mandat ne sont pas des détails à découvrir après la solution. Elles délimitent l'espace où une proposition peut être acceptée. Une intention sans limites peut conduire un générateur à optimiser une forme visible tout en compromettant ce qui rendait le résultat utile.

Évaluer

Évaluer consiste à regarder avec précision ce que nous savons, ce que nous supposons et ce qui doit encore être confirmé. Cette évaluation ne cherche pas une certitude abstraite. Elle cherche la connaissance nécessaire pour la décision à portée réelle. Les références `P-0042` et `42` peuvent produire la même forme après normalisation. Cela n'établit pas leur identité. L'équipe peut tout de même avancer : elle isole la collision, conserve les valeurs d'origine et précise la condition de reprise.

Évaluer inclut la spécification de ce qui doit être vrai. Une phrase comme « le système traite correctement la demande de livraison urgente » ne donne pas de prise à la vérification. Une condition telle que « avant 16 h, avec créneau urgent confirmé et préparation non commencée, l'option de 20 % est appliquée » rend une attente observable. Elle ne prescrit pas la solution. Elle conserve des options tout en empêchant qu'un cas frontière soit décidé par une implémentation silencieuse.

Évaluer inclut enfin l'interprétation de la preuve. Le résultat d'un test, d'une revue ou d'une mesure ne porte pas seul son statut. Il faut savoir quelle propriété il couvre, quel environnement il représente et ce qu'il laisse hors de son champ. La validation utile ne demande pas « avons-nous suffisamment testé ? ». Elle demande « qu'avons-nous établi avant de prendre cette décision, et qu'est-ce qui pourrait encore la contredire ? ».

Accepter ou escalader

Accepter consiste à assumer explicitement un compromis dans un mandat donné. Pour la notification de suivi, une personne habilitée peut accepter deux tentatives dans une fenêtre limitée parce que le doublon est tolérable dans ce cas. Cette acceptation ne vaut ni pour une confirmation de paiement ni pour toutes les équipes qui utilisent le composant. Elle est liée à une classe, à des preuves, à des limites et à une autorité identifiable.

Escalader est l'autre geste de l'acceptation responsable. Lorsque l'effet dépasse le mandat, que les autorités compétentes divergent, qu'une connaissance reste absente ou qu'une exigence impérative n'est pas démontrée, poursuivre comme si le choix était local ne rend pas l'organisation autonome. Cela rend le risque anonyme. L'escalade rend la décision à la personne ou à l'instance qui peut réellement en porter les conséquences.

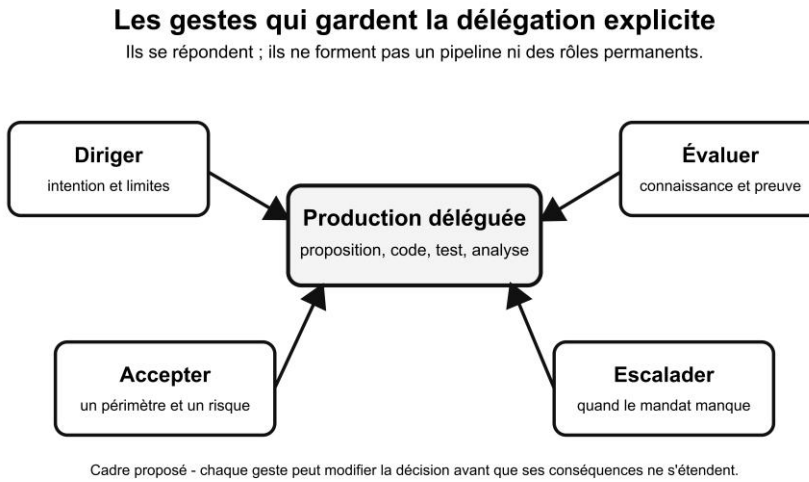


Figure 10.1 - Cadre proposé par le livre. Les quatre gestes se nourrissent mutuellement ; ils ne composent ni une chaîne de validation obligatoire ni un partage figé des rôles.

Ces verbes ne sont pas une grille concurrente des objets du livre. Ils donnent une forme active à ce qui a déjà été établi. Diriger rend visibles intention et contraintes. Évaluer rend visibles connaissance, spécification et preuves. Accepter ou escalader rend visibles mandat, portée et risque résiduel. Leur intérêt est de maintenir une question vivante : quel geste manque avant que nous traitions une production rapide comme une décision achevée ?

Proportionner l'effort, pas les obligations

La discipline ne demande pas la même énergie pour tout changement. Une préférence de tri qui n'affecte que l'affichage local, se réinitialise et ne crée aucun droit nouveau peut avancer avec une vérification bornée.

Un accès d'assistance entre organisations engage confidentialité, cloisonnement, révocation et traçabilité. Il exige davantage d'investigation, de scénarios, de preuves et de mandats avant d'être activé.

Pour décider de cet effort, une équipe peut regarder l'impact d'une erreur, sa propagation, la difficulté à revenir en arrière et ce qu'elle ignore encore. Ces dimensions ne calculent pas une réponse. Elles aident à choisir où placer l'attention. Une erreur locale, visible et facilement annulable permet souvent un apprentissage par une diffusion bornée. Une information qui se propage vers des lecteurs inconnus, un contrat externe ou une donnée difficile à corriger justifie une preuve plus forte avant le premier engagement.

Dimension	**Quand elle augmente**	**Effet possible sur l'effort**
Impact	Une erreur affecte confiance, droit d'accès ou engagement client.	Étendre les scénarios et mobiliser le mandat concerné.
Propagation	Le résultat est consommé par plusieurs produits ou partenaires.	Vérifier la compatibilité avant l'élargissement.
Réversibilité	Les données, messages ou contrats sont difficiles à corriger.	Chercher la preuve plus tôt et borner la diffusion initiale.
Inconnues	Une condition importante n'est pas qualifiée.	Suspendre, réduire ou organiser une recherche avant l'engagement.

Ces dimensions ne permettent jamais de déclarer facultative une exigence impérative. L'isolation entre organisations ne devient pas un objectif souhaitable mais négociable parce que le prototype est presque prêt. La confidentialité ne devient pas une préférence parce qu'un responsable veut aider un client. Le socle obligatoire s'applique même lorsque l'équipe choisit une validation légère pour un changement local.

Cette distinction évite deux caricatures. La première est le contrôle uniforme : exiger une investigation lourde pour une correction visible et

réversible peut décourager les améliorations sans renforcer ce qui compte. La seconde est le pragmatisme sans frontières : invoquer l'urgence pour traiter une exigence impérative comme un compromis ordinaire. Proportionner l'effort permet de rester exigeant sans prétendre que toute décision présente le même coût.

Dire qu'un changement est réversible exige de nommer ce qui peut effectivement être annulé, retrouvé, expliqué et, lorsque c'est pertinent, rejoué. Le contrôle proportionné a lui aussi un coût ; il devient utile lorsqu'il éclaire une décision identifiable avant qu'une conséquence soit plus difficile à réparer.

Des décisions explicites, pas le mythe de l'infaillibilité

Une organisation sérieuse peut se tromper. Elle peut avoir formulé une intention claire, identifié ses contraintes, produit des scénarios pertinents, réuni les mandats nécessaires et découvrir ensuite une information qui modifie l'arbitrage. La discipline de la délégation ne promet pas de supprimer ce risque. Elle refuse seulement de confondre l'incertitude restante avec l'absence de décision.

Une décision explicite conserve plusieurs capacités lorsque le résultat est décevant. Elle permet de comprendre ce qui a été recherché. Elle montre quelles limites étaient connues et lesquelles étaient supposées. Elle relie une preuve à la question qu'elle devait éclairer. Elle rend visible l'autorité qui a accepté le périmètre. Elle permet enfin de réduire, réparer, escalader ou apprendre sans reconstruire artificiellement une histoire où personne n'aurait pris de décision.

Cette possibilité d'apprendre ne doit pas devenir une excuse pour livrer sans preuve. Une diffusion bornée est utile parce qu'elle met une hypothèse à l'épreuve dans un périmètre contrôlé, non parce qu'elle autorise à exposer n'importe quelle contrainte. De même, l'existence d'un rollback ne rend pas acceptable une fuite de données déjà consultées. La faillibilité nous invite à mieux décrire les conditions de révision, pas à abaisser les exigences dont une erreur serait irréparable.

Après un résultat contradictoire	Réponse compatible avec la discipline	Réponse qui efface la décision

Après un résultat contradictoire	Réponse compatible avec la discipline	Réponse qui efface la décision
Une propriété attendue n'est pas établie	Modifier, réduire ou différer le périmètre et conserver la preuve.	Modifier le test pour retrouver un indicateur vert.
Une connaissance historique manque	Garder le statut à confirmer et négocier une alternative.	Supposer l'équivalence pour respecter la date.
Une observation change les conséquences produit	Réexaminer le mandat et l'activation.	Dire que l'approbation initiale couvre toute évolution.
Une exigence impérative est contredite	Arrêter ou rechercher l'exception réellement autorisée.	Présenter une dérogation implicite comme une optimisation.

La rigueur ne réside pas dans l'assurance d'avoir raison. Elle réside dans la capacité de dire ce que nous avions des raisons de croire, ce que nous avons réellement établi et ce que nous allons faire lorsque ces raisons ne suffisent plus. Cette honnêteté protège aussi la vitesse future : une équipe qui comprend pourquoi elle a réduit une décision peut la reprendre plus vite qu'une équipe qui doit d'abord découvrir quel engagement son code avait créé.

Faire de la visibilité une capacité quotidienne

Une posture ne devient pas réelle parce qu'elle est énoncée dans un document de cadrage. Elle apparaît dans des gestes ordinaires. Une demande indique le résultat recherché au lieu de commencer par une solution attendue. Une revue demande quelle contrainte aurait pu être perdue dans une simplification. Un statut « à confirmer » reste attaché à une condition au lieu de disparaître dans une note ancienne. Un test porte le nom de la propriété qui compte. Une décision d'activation indique la classe, le périmètre et la personne qui l'accepte.

Ces gestes paraissent modestes. Ils ont pourtant un effet cumulé lorsque la production est abondante. Plus les propositions sont nombreuses, plus il est facile de comparer leur forme sans comparer

leurs engagements. Plus les tests sont faciles à écrire, plus il est facile de couvrir les comportements déjà supposés sans rencontrer le cas qui contredit la supposition. Plus la revue est assistée, plus le risque existe de confondre une explication fluide avec une conséquence réellement comprise.

La visibilité crée une friction choisie. Elle interrompt brièvement le passage entre « nous avons une solution » et « nous pouvons l'accepter ». Cette friction n'est utile que si elle est reliée à une décision. Demander systématiquement une justification longue pour une correction locale la transforme en rituel. Demander une propriété d'isolation, un refus explicite ou un mandat pour un accès qui franchit des organisations la transforme en protection contre une conséquence difficile à défaire.

Cette distinction peut guider l'amélioration d'un système de travail. Une équipe n'a pas besoin de lancer un programme de transformation pour commencer. Elle peut observer une décision récente qui a surpris quelqu'un : quelle intention n'avait pas été formulée ? Quelle contrainte est apparue tard ? Quel fait avait été pris pour acquis ? Quelle preuve ne répondait pas à la question utile ? Quel mandat était supposé plutôt qu'identifié ? La réponse ne constitue pas un diagnostic général de maturité. Elle donne une occasion concrète de rendre le prochain arbitrage plus lisible.

L'équipe peut aussi chercher les endroits où une décision est déjà prise mais sans vocabulaire pour le dire. Un paramètre de configuration peut contenir une tolérance produit. Une migration de données peut contenir une hypothèse d'identité. Un seuil d'alerte peut contenir une acceptation de délai. Un indicateur de déploiement peut contenir une promesse au client. Les rendre visibles ne signifie pas retirer ces choix du système. Cela signifie relier leur existence à l'intention, à la preuve et au mandat qui les rendent défendables.

La trace doit servir une décision, pas la remplacer

Lorsque les équipes cherchent à rendre leurs choix explicites, elles peuvent tomber dans la collecte de traces. Chaque discussion devient

un ticket, chaque scénario une feuille de calcul, chaque déploiement une suite d'approbations. Cette accumulation peut donner une impression de contrôle tout en rendant plus difficile de retrouver la seule information qui comptait. Une trace qui n'aide plus à comprendre, à décider, à revoir ou à réparer ne remplit plus sa fonction.

La discipline demande une trace proportionnée à la portée du changement. Pour le tri local, le code, les tests de réinitialisation et la revue peuvent suffire à montrer que l'affichage n'a pas étendu ses droits. Pour un accès inter-organisations, la trace doit permettre de retrouver le mandat, la durée, les refus attendus et l'identité de l'accès effectué. Le niveau de détail suit les conséquences possibles, non le désir de produire une preuve administrative de sérieux.

Une trace utile conserve également le lien entre un résultat et la décision qui l'interprète. Un tableau de tests qui indique cent cas passants peut être précieux, mais il n'explique pas nécessairement pourquoi deux tentatives de notification ont été acceptées pour une classe et non pour une autre. Une approbation enregistrée peut être nécessaire, mais elle n'indique pas toujours quelle exigence restait impérative. L'organisation gagne à conserver les éléments qui permettent de répondre à ces questions sans devoir reconstituer l'intention à partir d'indices dispersés.

Cette règle protège les personnes autant que le système. Lorsqu'un résultat inattendu survient, une trace claire évite de chercher un responsable à blâmer avant de comprendre la décision. Elle permet de voir quelles informations étaient disponibles, quel périmètre avait été accepté et quelle condition de reprise existait. La responsabilité devient alors la capacité d'agir et de répondre, non l'exposition d'une personne isolée à une signature dont elle ne maîtrisait pas le sens.

Ne pas confondre autonomie et absence de limites

L'autonomie est souvent invoquée pour défendre la rapidité d'une équipe. Elle est réelle lorsqu'une équipe dispose du mandat, de la connaissance et des preuves nécessaires pour décider dans son périmètre. Elle devient fictive lorsqu'elle oblige l'équipe à accepter des

conséquences produit, réglementaires ou inter-organisationnelles qu'elle ne peut pas connaître ni porter seule.

Rendre l'escalade possible ne retire donc pas l'autonomie. Cela évite que l'équipe doive choisir entre deux formes de dépendance cachée : attendre une approbation générale qui ne comprend pas le cas, ou avancer seule avec une autorité supposée. Une escalade bien formulée demande une décision précise, indique ce qui est établi, nomme la contrainte qui bloque et conserve une option sûre durant l'attente. Elle donne à l'autorité sollicitée les conditions pour répondre sans absorber tout le travail local.

Cette approche demande aussi de respecter les décisions prises plus près du travail lorsque leur périmètre est réel. Une organisation qui réexamine systématiquement un tri local, une correction réversible et déjà démontrée transmet le message inverse : les mandats existent sur le papier, mais personne ne peut les exercer. La discipline ne cherche pas davantage de contrôle central. Elle cherche un contrôle humain explicite à la bonne échelle.

Le même raisonnement vaut pour les agents IA. Leur confier des tâches étroites peut augmenter l'autonomie d'une équipe, parce qu'ils préparent plus vite des options et des preuves. Leur attribuer silencieusement la décision de ce qui compte ou de ce qui est acceptable réduit en réalité l'autonomie humaine : personne ne peut plus expliquer où l'intention, la contrainte ou le mandat ont été choisis. L'enjeu n'est pas de limiter l'outil par réflexe, mais de ne pas lui demander d'endosser une responsabilité que l'organisation n'a pas elle-même décrite.

Une discipline révisable

Ce livre a employé des mots stables pour faciliter la discussion. Ils ne doivent pas devenir un vocabulaire rigide. Une équipe peut découvrir qu'elle nommait « contrainte » une préférence négociable, ou qu'elle traitait comme une connaissance établie un retour d'expérience limité. Elle peut améliorer un critère, déplacer une preuve plus tôt ou réviser

un mandat. Ces corrections ne sont pas des échecs de la discipline. Elles montrent qu'elle reste capable de se réexaminer.

Ce qui devrait résister à cette révision est plus simple. Aucune production ne doit faire disparaître la question de son sens. Aucune vitesse ne doit transformer une exigence impérative en détail optionnel. Aucune preuve ne doit être présentée comme plus large que ce qu'elle établit. Aucun risque résiduel ne doit être accepté par défaut parce que l'autorité qui devrait le porter n'a pas été identifiée.

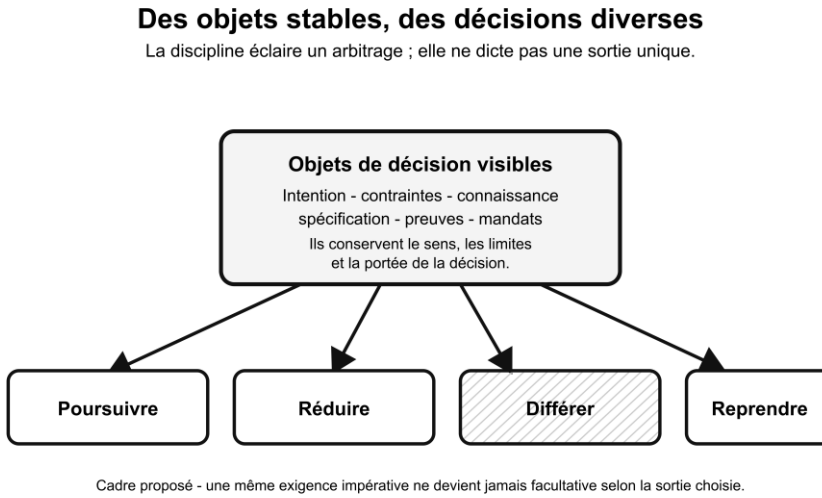


Figure 10.2 - Cadre proposé par le livre. Les mêmes objets de décision éclairent des choix de poursuivre, réduire, différer ou reprendre sans prescrire une sortie unique.

Une équipe n'atteint donc pas un état final où tout travail confié à un agent IA serait parfaitement contrôlé. Elle développe une capacité à reconnaître les objets qui demandent son jugement lorsque le contexte change. Cette capacité est plus durable qu'un ensemble d'outils, de formulaires ou de commandes parce qu'elle peut les réviser sans renoncer à la responsabilité qu'ils étaient censés soutenir.

Une posture qui survit aux outils

Les outils continueront à changer. Ils produiront peut-être des propositions plus complètes, trouveront plus d'incohérences,

exécuteront davantage de validations et expliqueront mieux leurs propres limites. Il serait imprudent de prédire à quelle vitesse ces capacités progresseront ou quelles tâches elles absorberont. Le livre n'a pas besoin de cette prévision pour conserver sa portée.

Plus une production devient facile, plus il devient utile de distinguer l'artefact de la décision qui l'autorise. Une capacité accrue à explorer des options n'établit pas le résultat recherché. Une capacité accrue à générer des scénarios n'établit pas que les scénarios couvrent la contrainte décisive. Une capacité accrue à classer les risques n'établit pas qui peut accepter celui qui reste. Les réponses peuvent être mieux préparées par les outils ; leur portée et leur responsabilité restent à organiser.

Cette posture ne suppose pas que l'humain constitue une limite technique éternelle. Les rôles, les connaissances disponibles et les façons de prouver évolueront avec les systèmes. Une règle jusque-là inconnue peut être documentée. Une preuve manuelle peut devenir automatisée. Un mandat peut être redéfini. Ce qui doit rester visible est le passage lui-même : qui formule le sens, quelle limite s'impose, quelle preuve est suffisante pour un périmètre et qui répond de l'acceptation ou de l'escalade.

La discipline est donc moins un rempart contre l'outil qu'une manière d'employer une IA générative ou un agent de code sans perdre le fil des conséquences. Elle permet de leur confier davantage de travail lorsque les conditions sont explicites. Elle oblige à réduire ou différer lorsque ces conditions ne sont pas réunies. Elle autorise l'apprentissage par une preuve et une diffusion bornées lorsqu'un retour est réellement possible. Elle ne transforme aucune de ces réponses en loi universelle.

> **Dans une pratique SDD** `constitution.md`, `spec.md`, `plan.md` et `tasks.md` donnent une forme partageable aux objets de décision ; `analyze` et `converge` peuvent révéler des écarts entre eux et le code. Leur valeur ne vient ni de leur quantité ni de l'outil qui les produit. Elle vient de leur capacité à garder reliés le sens recherché, les limites, la portée de la preuve et le mandat qui porte la conséquence.

La conclusion pourra revenir à la promesse initiale du livre. Pour l'instant, retenons une posture praticable : confier à des agents IA le code et les tâches qui peuvent l'être, tout en gardant sous contrôle humain explicite le sens recherché, les limites à respecter, les preuves attendues et les mandats qui permettent d'en assumer les conséquences.

Conclusion - Déléguer sans abandonner le jugement

Au début de ce livre, j'ai refusé de traiter la productivité assistée par l'IA générative comme une question à laquelle une moyenne pourrait répondre. Je garde cette réserve. Les agents de code, souvent fondés sur des LLM, peuvent rendre certaines productions plus rapides, plus nombreuses et plus faciles à explorer. Ils peuvent aussi déplacer l'effort vers la recherche de contexte, la vérification, l'intégration et la responsabilité. Aucun de ces effets ne décide, à lui seul, ce qu'une équipe doit accepter.

Ce déplacement change pourtant la nature visible du travail informatique. Lorsque la première implémentation coûte moins cher, produire une réponse n'est plus toujours l'étape qui concentre le plus d'attention. Il faut davantage savoir formuler ce qui mérite d'être produit, comparer plusieurs propositions plausibles, reconnaître ce qu'elles supposent et établir ce qui permet de les intégrer. Écrire, déboguer et maintenir du code restent des activités d'ingénierie profondes. Elles s'inscrivent plus explicitement dans un travail de direction et de jugement.

Je ne vois pas cette évolution comme une opposition entre l'humain et l'agent IA. Un agent peut explorer un dépôt, préparer plusieurs variantes, retrouver des dépendances, produire des scénarios et signaler une incohérence. Ces capacités deviennent plus utiles quand elles servent une décision identifiable. Elles deviennent fragiles lorsque la qualité apparente de la réponse suffit à lui donner le statut de réponse autorisée.

Garder les liens visibles

La valeur ne réside donc pas dans la seule quantité de code qu'une organisation peut produire. Elle réside dans sa capacité à diriger cette production sans perdre le fil de ses conséquences.

Je veux pouvoir retrouver le résultat recherché derrière une demande de fonctionnalité, une migration ou une correction. Une formulation

technique peut être très précise et ne pas dire encore ce qui doit changer pour un utilisateur, une activité ou un système. Un agent IA peut répondre avec compétence à cette formulation. Il ne peut pas choisir par défaut l'engagement que l'organisation entend prendre.

Je veux aussi pouvoir reconnaître les limites qui s'imposent à la solution. Certaines relèvent d'un compromis autorisé. D'autres sont impératives. Une contrainte de confidentialité, un droit d'accès ou un engagement contractuel ne deviennent pas négociables parce qu'une proposition est élégante ou qu'un délai est serré. En revanche, l'effort consacré à les vérifier peut être proportionné à la portée et à la réversibilité réelle du changement.

Je veux enfin distinguer ce que l'équipe sait, ce qu'elle suppose et ce qu'elle a effectivement établi. Une hypothèse utile n'est pas un fait. Une donnée absente n'est pas une confirmation. Un test vert possède une portée, pas une autorité générale. Cette précision ne demande pas la certitude avant toute action. Elle permet de poursuivre dans un périmètre borné, de conserver une question ouverte ou de demander la preuve qui pourrait encore changer la décision.

Ces liens rendent la responsabilité praticable. Une personne ou une équipe peut agir lorsque son mandat, ses informations et ses moyens couvrent les conséquences. Lorsqu'ils ne les couvrent pas, l'escalade n'est pas un abandon de l'autonomie. Elle restitue la décision à l'autorité qui peut réellement accepter le risque, tout en laissant l'équipe poursuivre ce qui demeure sûr dans son propre périmètre.

La vitesse retrouve sa place

Je ne propose pas de ralentir chaque changement. Une exploration plus rapide, un correctif préparé plus tôt ou une preuve automatisée restent des gains réels. Une équipe peut livrer un périmètre réduit qui apporte déjà de la valeur. Elle peut apprendre par une diffusion bornée lorsqu'elle sait ce qu'elle observe et ce qu'elle peut effectivement arrêter ou réparer.

La proportion ne dispense pas du jugement. Elle lui évite de devenir uniforme. Un tri d'affichage local ne demande pas les mêmes preuves

qu'un accès entre organisations. Dans les deux cas, l'équipe doit pouvoir expliquer ce qu'elle a vérifié et pourquoi ce niveau de contrôle correspond aux conséquences possibles. Un retour arrière technique peut arrêter des effets futurs sans retirer une information déjà consultée ni annuler un engagement déjà interprété.

Cette manière de travailler accepte aussi la possibilité de l'erreur. Une décision explicite peut se révéler mauvaise. Sa force n'est pas d'avoir été infaillible, mais de pouvoir être comprise et reprise : l'équipe sait ce qu'elle cherchait, sur quels éléments elle s'est appuyée, quelle limite elle voulait préserver et quelle autorité avait accepté le périmètre. Elle peut alors corriger sans inventer après coup une histoire où personne n'aurait réellement décidé.

Les artefacts restent des moyens

Le développement piloté par les spécifications offre des lieux où conserver ces liens. `constitution.md` peut porter des principes durables. `spec.md` peut rendre visibles l'intention, les critères et les questions ouvertes. `plan.md` expose des choix de conception. `tasks.md` relie le travail distribué à ce qui doit être vrai. La clarification, l'analyse et la convergence donnent des occasions de rencontrer un écart avant qu'il ne devienne une décision silencieuse.

Je ne leur attribue pas un pouvoir qu'ils n'ont pas. Un fichier ne rend pas une information vraie, une checklist ne détermine pas qu'une preuve est suffisante et un plan ne porte pas un mandat. Ces artefacts sont utiles lorsqu'ils aident des personnes à comparer une conséquence, contredire une proposition ou exercer une responsabilité. Leur accumulation n'est pas une preuve de contrôle.

C'est pourquoi je vois le SDD comme un harnais. Il n'immobilise pas l'agent IA et ne choisit pas la destination. Il relie sa capacité d'action à des points d'ancrage humains : le sens recherché, les limites qui s'imposent, les preuves attendues et l'autorité qui accepte le résultat. Si l'un de ces points change, le lien permet de revenir à l'endroit où la décision doit être réexaminée.

Les outils évolueront. Ils produiront sans doute des changements plus complets, rapprocheront mieux les artefacts et automatiseront des contrôles aujourd'hui coûteux. Je n'ai pas besoin de prévoir précisément cette évolution pour conserver le point central : plus nous confions de travail à des agents IA, plus nous devons savoir où le jugement reste porté.

Je reviens ainsi à la formule qui guide ce livre : déléguer le code, garder le contrôle. Garder le contrôle ne signifie ni écrire chaque ligne soi-même, ni refuser l'aide d'un agent, ni multiplier les barrières. Cela signifie rester capable de comprendre ce que la production engage, de demander ce qui pourrait la contredire et de décider en connaissance de cause.

Une équipe qui conserve cette capacité peut accélérer sans confondre l'élan avec la décision. Elle peut réduire un périmètre sans perdre son intention, différer sans enfouir la condition de reprise et escalader sans rendre sa responsabilité anonyme. C'est cette liberté d'agir et de réexaminer, davantage que l'abondance du code, que je vous invite à développer.

Bibliographie de travail

Cette bibliographie réunit les sources citées dans le manuscrit et les références de contrôle conservées dans le registre de preuves. Le format retenu est **auteur-date** : auteur ou organisme, année, titre, publication ou éditeur, volume et article lorsque ces données existent, puis DOI ou adresse canonique. La date de consultation n'est ajoutée qu'aux documentations web susceptibles d'évoluer.

Les notes du manuscrit conservent la limite d'usage de chaque source. Cette bibliographie identifie les documents ; elle ne remplace pas ces précautions.

Références citées dans le manuscrit

- Becker, J., Rush, N., Barnes, B. et Rein, D. 2025. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. METR, juillet 2025.
- Becker, J., Rush, N., Cunningham, T., Rein, D. et Mahamud, K. 2026. *We are Changing our Developer Productivity Experiment Design*. METR, février 2026.
- Borg, M. et al. 2026. *Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability*. *Empirical Software Engineering*, 31, article 161.
- Confluent. 2026. *Schema Evolution and Compatibility for Schema Registry on Confluent Platform*. Documentation du 9 septembre 2026, consultée le 19 septembre 2026.
- Cui, K. Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S. et Salz, T. 2026. *The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers*. *Management Science*, publication en ligne le 27 février 2026.
- Ebrahim, S. et Tanner, M. 2025. *A Systematic Literature Review on the Use of ChatGPT to Support Productivity of Software Developers*. *AMCIS 2025 Proceedings*, article 17.
- GitHub. 2026. *Spec Kit - Spec-Driven Development*. Documentation consultée le 20 septembre 2026.

- Government Digital Service. s. d. *Learning about users and their needs*. Consulté le 19 septembre 2026.
- Microsoft. s. d. *Retry pattern*. Azure Architecture Center, consulté le 19 septembre 2026.
- NASA. s. d. *Systems Engineering Handbook - Appendix C: How to Write a Good Requirement*. Consulté le 19 septembre 2026.
- Paradis, E. et al. 2025. *How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial*. *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice*.
- Peng, S., Kalliamvakou, E., Cihon, P. et Demirer, M. 2023. *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*. arXiv:2302.06590.
- Software Engineering Institute, Carnegie Mellon University. 2016. *Managing Technical Debt in Complex Software Systems*.
- Xu, F., Medappa, P. K., Tunç, M. M., Vroegindeweij, M. et Fransoo, J. C. 2026. *AI-Assisted Programming Decreases the Productivity of Experienced Developers by Increasing the Technical Debt and Maintenance Burden*. Prépublication arXiv:2510.10165, version 3 du 28 janvier 2026.

Références de contrôle conservées dans le registre

- Fu, Y., Liang, P., Tahir, A., Li, Z., Shahin, M., Yu, J. et Chen, J. 2025. *Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study*. *ACM Transactions on Software Engineering and Methodology*, 34(8), article 218, 1-34. Version ouverte.
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J. et Wang, H. 2024. *Large Language Models for Software Engineering: A Systematic Literature Review*. *ACM Transactions on Software Engineering and Methodology*, 33(8), article 220, 1-79.

Contrôles à refaire lors de l'épreuve finale

- Rechercher une version publiée de Xu et al. et, à défaut, conserver explicitement le statut de prépublication.
- Vérifier les dates et versions des documentations Confluent, GitHub, Microsoft, NASA et Government Digital Service.
- Reporter sans ambiguïté les DOI, volumes, numéros d'article et éventuelles paginations définitives dans le gabarit de publication.